

UNIVERSIDAD DE GRANADA

TRABAJO FIN DE GRADO

INGENIERÍA INFORMÁTICA

Predicción del riesgo de cáncer de pulmón mediante aprendizaje automático y redes neuronales

Autor
Manuel Martínez Herrera

Directores
Francisco Cabrerizo Lorite
José Ramón Trillo Vílchez



Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación

—

Granada, Junio de 2026



UNIVERSIDAD DE GRANADA

Predicción del riesgo de cáncer de pulmón mediante aprendizaje automático y redes neuronales

Autor
Manuel Martínez Herrera

Directores
Francisco Cabrerizo Lorite
Jose Ramón Trillo Vílchez

Predicción del riesgo de cáncer de pulmón mediante aprendizaje automático y redes neuronales: Comparación de modelos supervisados y técnicas de balanceo para la clasificación de pacientes a partir de factores de riesgo

Manuel Martínez Herrera

Palabras clave: cáncer de pulmón, aprendizaje automático, análisis exploratorio, balanceo de datos, hiperparámetros

Resumen

El principal objetivo de este trabajo es predecir, a partir de una serie de variables clínicas y personales, si un paciente padece cáncer de pulmón. El interés por esta temática se debe a la gravedad de esta enfermedad, considerada una de las más peligrosas a nivel mundial.

Para llevar a cabo el proyecto, en primer lugar se realizó un análisis exploratorio de los datos, estudiando distribuciones, correlaciones y otras relaciones relevantes entre variables. A partir de este análisis se identificaron aquellas características que podían aportar mayor información al proceso de predicción.

Posteriormente, se desarrolló una fase experimental inicial con distintos modelos de aprendizaje automático. Durante esta etapa se observó que la calidad y distribución de los datos de entrada podían suponer una de las principales limitaciones del proyecto. Por ello, se aplicaron diferentes técnicas de balanceo, tanto de oversampling como de undersampling, con el objetivo de comprobar si era posible mejorar el rendimiento de los modelos.

Tras esta experimentación, se comprobó que las técnicas de balanceo no mejoraban de forma significativa los resultados obtenidos con el conjunto de datos original. Por este motivo, los modelos finales se entrenaron y evaluaron utilizando dicho conjunto original. Finalmente, se analizaron los resultados obtenidos y se redactaron las conclusiones del trabajo.

Prediction of lung cancer risk using supervised learning and neural networks: Comparison of supervised learning models and data balancing techniques for patient classification based on risk factors

Manuel Martínez Herrera

Keywords: lung cancer, machine learning, classification, data balancing, hyperparameters

Abstract

The main objective of this project is to predict whether a patient suffers from lung cancer based on a set of clinical and personal variables. The interest in this topic arises from the severity of this disease, which is considered one of the leading causes of cancer-related deaths worldwide.

To carry out the project, an exploratory data analysis (EDA) was first performed. This included studying variable distributions, correlations, and other relevant relationships within the dataset. Based on this analysis, the variables with the greatest predictive potential were identified.

Subsequently, an initial experimental phase was conducted using different machine learning models. During this stage, it was observed that the quality and distribution of the input data could represent one of the main limitations of the project. Therefore, several data balancing techniques, including both oversampling and undersampling approaches, were applied in an attempt to improve model performance.

After this experimentation, the results showed that these balancing techniques did not significantly improve the performance achieved with the original dataset. Consequently, the final models were trained and evaluated using the original data. Finally, the obtained results were analyzed and the conclusions of the study were drawn.

Yo, **Manuel Martínez Herrera**, alumno de la titulación Ingeniería Informática de la **Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación de la Universidad de Granada**, con DNI 26529013P, autorizo la ubicación de la siguiente copia de mi Trabajo Fin de Grado en la biblioteca del centro para que pueda ser consultada por las personas que lo deseen.

Fdo: Manuel Martínez Herrera

Granada a 3 de Febrero de 2026.

D. **Francisco Javier Cabrerizo Lorite**, Profesor del Departamento de Ciencias de la Computación e Inteligencia Artificial de la Universidad de Granada.

D. **José Ramón Trillo Vílchez**, Profesor del Departamento de Informática e Ingeniería de Sistemas de la Universidad de Zaragoza.

Informan:

Que el presente trabajo, titulado ***Predicción del riesgo de cáncer de pulmón mediante aprendizaje supervisado y redes neuronales, Comparación de modelos supervisados y técnicas de balanceo para la clasificación de pacientes a partir de factores de riesgo***, ha sido realizado bajo su supervisión por **Manuel Martínez Herrera**, y autorizamos la defensa de dicho trabajo ante el tribunal que corresponda.

Y para que conste, expiden y firman el presente informe en Granada a X de Junio de 2026.

Los directores:

Agradecimientos

Con este trabajo finaliza una etapa llena de sacrificio, esfuerzo y dedicación. Durante estos cuatro años he aprendido y crecido tanto a nivel personal como profesional. Aunque he sido yo quien ha tenido que superar los obstáculos para llegar hasta aquí, este logro no habría sido posible sin el apoyo de las personas que me han acompañado en el camino.

Por ello, quiero agradecer especialmente a:

A mis padres, por la oportunidad que me habéis dado y por haberme puesto siempre por delante de vosotros mismos. Gracias por vuestro esfuerzo, confianza y apoyo incondicional.

A mis abuelos, a mis abuelas y a Lola, por confiar ciegamente en mí. Sin vuestro cariño, apoyo y ánimo constante, este logro no habría sido posible.

A mis hermanos, por acompañarme siempre y por hacerme sentir que puedo contar con vosotros en cualquier momento.

A mis primos, tíos y amigos, gracias por todos los momentos compartidos, por las risas, por vuestra compañía y por ayudarme cuando más lo necesitaba.

A mi tutor, gracias por sus consejos, por su ayuda y por haberme exigido siempre un poco más.

A Lorena, gracias por estar presente durante toda esta etapa, por tu apoyo, por tu ayuda y por todo lo que me has enseñado. Gracias por estar.

Por último, como dijo Isaac Newton: «Si he logrado ver más lejos, ha sido porque me he subido a hombros de gigantes». Hoy puedo decir que esos gigantes sois vosotros.

Gracias.

Índice general

I Marco Introductorio **1**

1. Introducción y objetivos **3**

1.1. Motivación 3

1.2. Contextualización 4

1.3. Descripción del problema 5

1.4. Estructura del trabajo 5

1.5. Herramientas utilizadas 6

1.6. Planificación y presupuestos 7

1.7. Requisitos del sistema 9

1.7.1. Requisitos funcionales 9

1.7.2. Requisitos no funcionales 10

1.8. Objetivos 11

1.9. Consideraciones éticas, legales y de sostenibilidad 12

II Estado del Arte y Fundamentos Teóricos **15**

2. Estado del arte **17**

3. Fundamentos teóricos del aprendizaje automático **19**

3.1. Introducción al aprendizaje automático 19

3.1.1. Fases de un problema de machine learning 20
3.2. Modelos de aprendizaje automático 22
3.2.1. Modelos lineales 22
3.2.2. Modelos basados en distancias 23
3.2.3. Máquina de vectores de soporte 24
3.2.4. Modelos basados en árboles 25
3.2.5. Métodos de combinación de modelos 31
3.3. Introducción a las redes neuronales artificiales 32
3.3.1. Neurona Artificial 32
3.3.2. Redes Neuronales feed-forward 34
3.3.3. Propagación del error 35
3.3.4. Descenso de gradiente 36
3.3.5. Funciones de activación 37
3.4. Problema del desbalanceo de datos 38
3.4.1. Técnicas de balanceo de datos 39
3.5. Métricas de evaluación de modelos 40

III Análisis y Desarrollo Experimental 45

4. Análisis exploratorio de los datos 47

4.1. Conjunto de Datos 47
4.2. Análisis de distribuciones 49
4.2.1. Distribuciones univariantes 49
4.2.2. Distribuciones de las variables condicionadas a la variable etiqueta 51
4.3. Análisis de dependencia y asociación 54
4.3.1. Importancia basada en IM e IV de la variables con respecto a la etiqueta 57

4.4. Variables con mayor capacidad predictiva 59

5. Estudio de viabilidad para mitigar el desbalanceo de clases 63

5.1. Oversampling 64

5.2. Undersampling 67

6. Marco experimental 71

6.1. Preprocesamiento 71

6.2. Modelos empleados 72

6.3. Métricas de evaluación 74

6.4. Planteamiento del proyecto 75

7. Experimentación y análisis de resultados 79

7.1. Modelos de aprendizaje automático evaluados 80

7.1.1. Modelos lineales 80

7.1.2. Modelos basados en distancias 83

7.1.3. Máquina de vectores de soporte 86

7.1.4. Modelos basados en árboles 89

7.1.5. Métodos de combinación de modelos 122

7.1.6. Redes Neuronales 127

7.2. Hiperparámetros 134

7.3. Análisis Global 134

7.4. Coherencia de los resultados con la literatura 137

IV Conclusiones 141

8. Conclusiones y trabajo futuro 143

8.1. Conclusiones 143

8.2. Limitaciones 144

8.3. Trabajo futuro 144

Parte I

Marco Introductorio

Capítulo 1

Introducción y objetivos

1.1. Motivación

El interés por el desarrollo de este trabajo surge de la combinación entre dos ámbitos de gran relevancia en la actualidad: por un lado, el impacto creciente del cáncer de pulmón como uno de los principales problemas sanitarios en España, y por otro, el potencial de las técnicas de inteligencia artificial para mejorar el análisis de datos complejos.

A pesar de los avances en oncología, el diagnóstico precoz del cáncer de pulmón continúa siendo un reto, en gran medida debido a la naturaleza inespecífica de sus síntomas, que pueden solaparse con otras enfermedades respiratorias. Estas limitaciones plantean la necesidad de explorar enfoques alternativos, como los programas de cribado apoyados en técnicas de machine learning.

En este sentido, el uso de modelos de aprendizaje automático resulta especialmente interesante, ya que permite analizar simultáneamente múltiples variables clínicas y demográficas, capturando patrones y relaciones complejas que podrían pasar desapercibidos mediante enfoques más convencionales.

El objetivo de este trabajo es evaluar la capacidad de estos modelos para mejorar la clasificación del riesgo de cáncer de pulmón, contribuyendo potencialmente a la detección temprana y al apoyo en la toma de decisiones clínicas.

Asimismo, este enfoque responde al interés por aplicar técnicas avanzadas de inteligencia artificial a problemas reales de alto impacto social, lo que refuerza tanto la relevancia como la aplicabilidad del estudio.

1.2. Contextualización

El cáncer constituye uno de los principales problemas de salud pública a nivel mundial. En España, las estimaciones más recientes elaboradas por la Red Española de Registros de Cáncer (REDECAN) y publicadas por la Sociedad Española de Oncología Médica (SEOM) en el informe *Las cifras del cáncer en España* [1] indican que en 2026 se alcanzarán 301.184 nuevos casos, lo que supone un incremento del 2 % respecto a 2025. Asimismo, dicho informe señala que, por primera vez, el cáncer ha superado a las enfermedades del sistema circulatorio como primera causa de mortalidad en España, y en mujeres, el cáncer de pulmón supera ya al de mama como primera causa de muerte por un tumor.

Simultáneamente, la Inteligencia Artificial (IA) se ha consolidado como una herramienta con capacidad para complementar los enfoques tradicionales en oncología. La literatura científica reciente destaca su impacto significativo en la investigación oncológica en [2]:

- Análisis de imágenes médicas, donde puede detectar lesiones y localizar focos de enfermedad con mayor eficiencia.
- Patología digital, apoyando la predicción de riesgo y la identificación de patrones en grandes volúmenes de muestras.
- Cirugía robótica, favoreciendo procedimientos más precisos y mínimamente invasivos.
- Descubrimiento de fármacos, acelerando la identificación de compuestos candidatos y la predicción de respuestas a tratamientos.

En el caso específico del cáncer de pulmón, diversos estudios recientes [3] han demostrado que los modelos de aprendizaje automático entrenados con datos tabulares pueden mejorar la capacidad de discriminación frente a modelos estadísticos tradicionales. En particular, en estos datos tabulares se incluyen variables como la demográfía, antecedentes clínicos y factores de riesgo como el tabaquismo, entre otros. Este tipo de enfoques basados en aprendizaje automático permite identificar relaciones no lineales y patrones complejos entre múltiples factores, favoreciendo una clasificación más precisa de la población según su nivel de riesgo.

En este contexto, el aprendizaje supervisado emerge como una estrategia prometedora para apoyar la detección precoz, priorizar pacientes según el riesgo y contribuir al diseño de programas de cribado más eficientes y personalizados en cáncer de pulmón.

1.3. Descripción del problema

El objetivo de este trabajo es abordar un problema de clasificación binaria orientado a la predicción del riesgo de cáncer de pulmón a partir de datos tabulares. Se dispone de un conjunto de datos, donde cada fila representa el perfil de un paciente y cada columna recoge un factor de riesgo o una característica relevante. La variable objetivo indica si el paciente presenta o no cáncer de pulmón.

El dataset incluye identificadores y variables clínicas y epidemiológicas habituales en la estimación de riesgo: variables como la edad y la exposición tabáquica medida, junto con variables categóricas relacionadas con el sexo y la exposición a factores ambientales o antecedentes médicos. Estas variables permiten construir una representación del paciente con la que podemos entrenar nuestros modelos.

Para estudiar este problema se evaluarán distintos modelos de aprendizaje supervisado, desde enfoques más clásicos como la regresión logística, hasta métodos más potentes como CatBoost, XGBoost o variantes similares, además de explorar el uso de redes neuronales. El propósito es comparar su capacidad para aprender patrones relevantes y generar predicciones útiles a partir de datos tabulares.

Debido a que el dataset presenta desbalanceo entre clases (más casos positivos que negativos), se analizará el efecto de distintas técnicas de balanceo del conjunto de datos, incluyendo métodos de oversampling y undersampling, para comprobar si contribuyen a mejorar el rendimiento y la generalización.

De la misma manera, se emplearán estrategias de búsqueda de hiperparámetros para optimizar los modelos y obtener mejores configuraciones. La evaluación no se centrará únicamente en el porcentaje de acierto global, sino en métricas más informativas para un contexto sanitario como F1, además de prestar especial atención a la capacidad del modelo para distinguir correctamente entre pacientes con cáncer y sin cáncer.

1.4. Estructura del trabajo

Este trabajo se estructura en 4 partes, cada una de las partes se divide en un conjunto de capítulos.

- **Primera parte:** En el primer capítulo, Introducción, se contextualiza y describe el problema abordado, así como la estructura general del trabajo. Además, se presentan las principales herramientas empleadas

durante el desarrollo del proyecto, junto con los requisitos, la planificación temporal y el presupuesto estimado. Finalmente, se definen los objetivos planteados desde el inicio del trabajo y las consideraciones legales, éticas y de sostenibilidad.

- **Segunda parte:** En esta parte se presenta el estado del arte y los fundamentos teóricos necesarios para comprender el desarrollo del proyecto. Se describen los distintos algoritmos de aprendizaje automático utilizados, así como las bases teóricas de las técnicas de balanceo de datos empleadas durante la experimentación.

- **Tercera parte:** En esta parte se realiza el análisis exploratorio de los datos y el desarrollo experimental del proyecto. En primer lugar, se estudian las distribuciones, las correlaciones y otras medidas informativas relevantes, con el objetivo de identificar aquellas variables con mayor capacidad predictiva. Posteriormente, se aplican diferentes técnicas de balanceo de datos y se desarrollan los distintos modelos de aprendizaje supervisado. Además, se incorporan redes neuronales como alternativa a los modelos tradicionales.

- **Cuarta parte:** En la sección final del trabajo se presentan las conclusiones obtenidas a partir del análisis y la experimentación realizados. Además, se exponen las limitaciones y se plantean posibles líneas de trabajo a futuro.

Además, el código generado durante las fases de análisis y experimentación se encuentra disponible en el siguiente repositorio de GitHub: `https://github.com/manuelmartinz/LUNG-CANCER`.

1.5. Herramientas utilizadas

El trabajo ha sido desarrollado por completo en el lenguaje de programación Python, un lenguaje interpretado, versátil e intuitivo utilizado en el ámbito de la ciencia de datos y el aprendizaje automático. Su dinamismo, flexibilidad, portabilidad y el amplio ecosistema de librerías especializadas lo convierten en una herramienta adecuada para el desarrollo de modelos predictivos y el análisis de datos.

Las principales librerías empleadas son TensorFlow y Keras, empleadas para el desarrollo de los modelos basados en redes neuronales; NumPy y Pandas, fundamentales para el manejo, preprocesamiento y procesamiento de datos; y Matplotlib y Wandb, utilizadas para la visualización de resultados y el análisis exploratorio.

Desde el punto de vista estadístico, se han aplicado distintos métodos para analizar la relación entre variables y evaluar su relevancia predictiva. En particular, se han utilizado coeficientes de correlación como Pearson y Spearman, para medir relaciones lineales entre variables numéricas. Para el análisis de asociación entre variables categóricas se ha empleado el estadístico Cramér's V. Además, se han incorporado medidas como el Information Value y la Información Mutua, con el objetivo de cuantificar la capacidad de una variable para distinguir entre dos clases y cuantificar la dependencia entre dos variables.

1.6. Planificación y presupuestos

El desarrollo de este trabajo comenzó a finales del mes de septiembre. La planificación del proyecto se ha dividido en varias fases que han permitido avanzar de manera progresiva desde la adquisición de conocimientos hasta la experimentación y redacción final del documento.

En una primera fase, el objetivo principal fue adquirir las bases necesarias para poder realizar el proyecto. Para ello, se realizaron dos cursos: uno centrado en el lenguaje de programación Python y otro orientado a la introducción y comprensión de los modelos de machine learning.

Una vez adquiridos los conocimientos básicos, se inició la fase de experimentación inicial. Durante esta etapa se desarrollaron los primeros modelos y se comenzó a trabajar con las diferentes herramientas y librerías que se utilizan a lo largo del proyecto.

Posteriormente, durante el periodo navideño, se llevó a cabo el análisis exploratorio de los datos (EDA). Este análisis permitió estudiar las características del conjunto de datos y determinar qué variables resultaban más relevantes para la predicción y cuáles aportaban una menor información al modelo.

Con el objetivo de mejorar el rendimiento de los modelos obtenidos, se aplicaron posteriormente distintas técnicas de balanceo de datos, para abordar el problema del desbalanceo entre clases presente en el dataset. Sin embargo, la experimentación no fue fructífera.

Finalmente, en la última fase del proyecto se continuó con la experimentación de distintos modelos y con la redacción progresiva del documento del trabajo.

Durante el desarrollo del curso se trabajó de manera constante en el proyecto. Aunque durante el segundo cuatrimestre la dedicación fue mayor, ya que se disponía de más tiempo para avanzar en el trabajo. En promedio, se dedicaron aproximadamente unas 12 horas semanales al desarrollo del TFG.

Si se considera una dedicación aproximada de 384 horas totales al proyecto, y se toma como referencia un salario base promedio de 27.000 euros brutos anuales para un perfil de Junior Data Scientist en España [4], puede estimarse el coste por hora dividiendo dicho salario entre una jornada anual aproximada de 1.880 horas, considerando que se trabaja unas 40 horas a la semana y que de las 52 semanas que tiene el año, aproximadamente 5 no se trabajan, debido a vacaciones, festivos y otros permisos habituales.

De esta forma, se obtiene un coste medio aproximadamente de unos 14,5 euros por hora, por lo que el valor estimado del trabajo realizado ascendería a 5.568 euros.

Tabla 1.1: Estimación del coste del trabajo realizado

Concepto	**Valor**
Salario base anual de referencia	27.000 €
Horas laborales anuales estimadas	1.880 h
Coste por hora estimado	14,5 €/h
Horas dedicadas al proyecto	384 h
Coste total estimado	**5.568 €**

A esta estimación habría que añadir otros costes indirectos asociados al desarrollo del proyecto, como pueden ser el equipamiento informático (portátil, monitor u otros dispositivos).

La planificación temporal de las distintas fases del proyecto puede observarse de forma resumida en el diagrama de Gantt mostrado en la Figura 1.1.

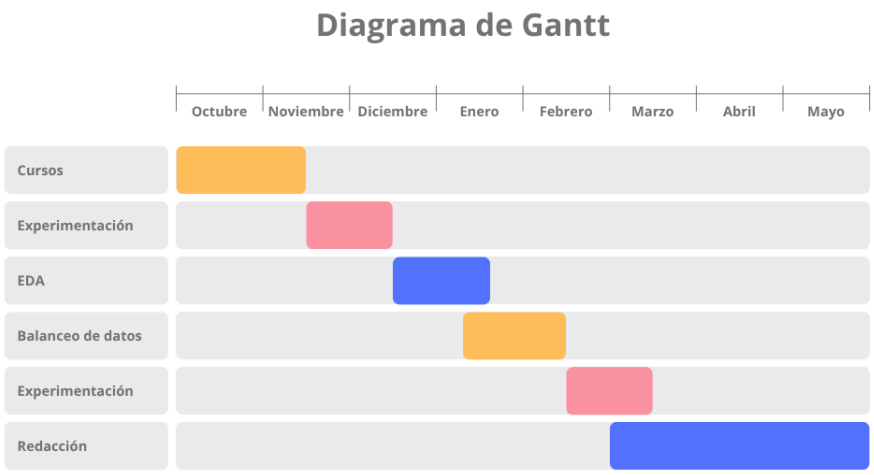


Figura 1.1: Planificación temporal del desarrollo del TFG

Al trabajar de manera constante y mantener reuniones mensuales de seguimiento con mi tutor, las tareas previstas pudieron desarrollarse de forma progresiva y dentro de los plazos establecidos. En general, no se produjeron desviaciones significativas respecto a la planificación inicial, ni fue necesario modificar de forma relevante el alcance del proyecto.

1.7. Requisitos del sistema

En esta sección se describen los principales requisitos funcionales y no funcionales considerados en el desarrollo del proyecto. Los requisitos funcionales definen las acciones que debe ser capaz de realizar el sistema, mientras que los requisitos no funcionales recogen las características de calidad, implementación y mantenimiento que debe cumplir.

1.7.1. Requisitos funcionales

- **RF1. Carga del conjunto de datos:** el sistema debe permitir cargar el conjunto de datos utilizado para el análisis y entrenamiento de los

modelos.

- **RF2. Preprocesamiento de los datos:** el sistema debe realizar las tareas necesarias de limpieza, transformación y preparación de los datos antes del entrenamiento.
- **RF3. Entrenamiento de modelos:** el sistema debe permitir entrenar diferentes modelos de aprendizaje automático para la clasificación del cáncer de pulmón.
- **RF4. Aplicación de técnicas de balanceo:** el sistema debe permitir aplicar técnicas de balanceo de clases, incluyendo métodos de oversampling y undersampling.
- **RF5. Evaluación de modelos:** el sistema debe evaluar el rendimiento de los modelos mediante métricas como Accuracy, Precision, Recall, F1-score y ROC-AUC.
- **RF6. Comparación de resultados:** el sistema debe permitir comparar los resultados obtenidos por los distintos modelos evaluados.
- **RF7. Análisis de importancia de variables:** el sistema debe permitir identificar y analizar la importancia de las variables utilizadas en la clasificación.
- **RF8. Optimización de hiperparámetros:** el sistema debe permitir seleccionar configuraciones de hiperparámetros que mejoren el rendimiento de los modelos.
- **RF9. Generación de gráficos:** el sistema debe generar representaciones gráficas que faciliten la interpretación de los resultados experimentales.
- **RF10. Predicción sobre nuevos registros:** el sistema debe permitir realizar predicciones sobre nuevos registros a partir de modelos previamente entrenados.

1.7.2. Requisitos no funcionales

- **RNF1. Lenguaje de programación:** el sistema debe estar implementado en Python.
- **RNF2. Escalabilidad:** el código debe estar organizado de forma modular, facilitando la incorporación de nuevos modelos o conjuntos de datos.

- **RNF3. Legibilidad del código:** el código debe ser claro, comprensible y estar adecuadamente estructurado.
- **RNF4. Reproducibilidad:** el sistema debe permitir reproducir los experimentos realizados bajo las mismas condiciones de ejecución.
- **RNF5. Eficiencia:** los experimentos deben poder ejecutarse en un tiempo razonable para el tamaño del conjunto de datos utilizado.
- **RNF6. Mantenibilidad:** la estructura del proyecto debe facilitar futuras modificaciones, correcciones o ampliaciones.

1.8. Objetivos

Los objetivos planteados al inicio del trabajo son los siguientes:

- Analizar qué variables presentan mayor influencia en la clasificación de pacientes con cáncer de pulmón.
- Estudiar el impacto del desbalanceo de clases y analizar si la aplicación de técnicas de balanceo mejora el desempeño predictivo.
- Implementar y evaluar distintos modelos de aprendizaje supervisado para la clasificación de pacientes.
- Implementar redes neuronales y comparar su rendimiento frente a modelos clásicos de aprendizaje supervisado.

Estos objetivos han sido cumplidos:

1. El primer objetivo queda cubierto en el capítulo 4, donde se analizan las distribuciones, correlaciones y otras medidas de interés, mostrando finalmente cuáles son las variables más relevantes desde un punto de vista estadístico. Además, algunos modelos como Random Forest 7.1.4 o Extra Trees 7.1.4 también permiten obtener información sobre la importancia de las variables según el propio modelo.
2. El segundo objetivo se cumple en la sección 5, donde se aplican diferentes técnicas de undersampling y oversampling, comparando sus resultados con los obtenidos a partir del conjunto de datos original sin modificar.
3. El tercer objetivo queda cubierto en el capítulo 7, donde se implementan y evalúan diferentes modelos pertenecientes a distintas familias de algoritmos.

4. El último objetivo se aborda también en el capítulo 7, donde se desarrolla una red neuronal y se compara su rendimiento en el conjunto de prueba con el de los distintos modelos implementados.

1.9. Consideraciones éticas, legales y de sostenibilidad

Aunque el trabajo no consiste en el desarrollo de una aplicación comercial ni en el despliegue de un sistema accesible al público, es necesario considerar ciertos aspectos legales, éticos y sociales relacionados con el uso de modelos predictivos en el ámbito sanitario.

Desde el punto de vista legal, el conjunto de datos utilizado no contiene información identificativa directa de pacientes, como nombres, documentos de identidad, direcciones o datos de contacto. Por tanto, el trabajo se desarrolla sobre datos anonimizados o no directamente identificables, reduciendo los riesgos asociados al tratamiento de datos personales. Sin embargo, al tratarse de un problema relacionado con la salud, se reconoce la especial sensibilidad de este tipo de información, por lo que su utilización debe realizarse exclusivamente con fines académicos y de investigación.

Asimismo, el proyecto no recoge datos de usuarios, no almacena información personal nueva y no presta un servicio real de diagnóstico médico. Por este motivo, no resulta necesario incorporar elementos legales propios de aplicaciones o servicios web orientados al público general.

Desde el punto de vista ético, los modelos desarrollados deben interpretarse únicamente como herramientas de apoyo al análisis de datos y no como sistemas destinados a sustituir el criterio de profesionales sanitarios. Las predicciones obtenidas no constituyen diagnósticos médicos y deben entenderse dentro de las limitaciones propias del conjunto de datos y de la metodología empleada.

También debe considerarse la posible existencia de sesgos en los datos utilizados. Si el conjunto de datos no representa adecuadamente a toda la población, los modelos pueden generar resultados menos precisos para determinados grupos de individuos. Por ello, los resultados obtenidos deben analizarse con cautela y teniendo presentes las limitaciones del estudio.

Por otra parte, este trabajo se encuentra alineado con el Objetivo de Desarrollo Sostenible 3 (Salud y Bienestar) [5], al explorar el potencial de las técnicas de aprendizaje automático como herramientas de apoyo para la detección temprana y el análisis de enfermedades. Asimismo, se relaciona

con el Objetivo de Desarrollo Sostenible 10 (Reducción de las Desigualdades) [6], al considerar que los avances tecnológicos en el ámbito sanitario deberían orientarse a favorecer un acceso equitativo al conocimiento y a las herramientas de apoyo a la salud.

Asimismo, en el hipotético caso de que una herramienta basada en los resultados de este trabajo llegara a desarrollarse para su utilización por parte del público general, se considera que debería orientarse a favorecer la accesibilidad y la igualdad de oportunidades en el acceso a recursos de apoyo a la salud. Esta visión se encuentra alineada con los Objetivos de Desarrollo Sostenible mencionados anteriormente, especialmente con aquellos relacionados con la salud y la reducción de las desigualdades. Desde esta perspectiva, los avances tecnológicos en el ámbito sanitario deberían contribuir prioritariamente al bienestar de las personas y a la difusión del conocimiento, procurando que sus beneficios puedan alcanzar al mayor número posible de usuarios con independencia de su situación económica.

En este sentido, el presente trabajo tiene una finalidad exclusivamente académica y divulgativa. Su objetivo es contribuir al estudio de técnicas de inteligencia artificial aplicadas al ámbito sanitario, favoreciendo la difusión del conocimiento y el acceso abierto a los resultados obtenidos.

Parte II

Estado del Arte y Fundamentos Teóricos

Capítulo 2

Estado del arte

La aplicación de técnicas de aprendizaje automático a la predicción del cáncer de pulmón ha experimentado un crecimiento significativo durante los últimos años. El interés por este campo se debe a la posibilidad de identificar patrones complejos en datos clínicos y epidemiológicos que pueden resultar difíciles de detectar mediante métodos estadísticos tradicionales.

Diversos estudios han evaluado el rendimiento de algoritmos supervisados sobre conjuntos de datos estructurados. Entre ellos, Dritsas y Trigka [7] realizaron una comparación de múltiples modelos de clasificación, obteniendo resultados especialmente favorables para los métodos ensemble. Por su parte, Tu et al. [3] analizaron diferentes estrategias de aprendizaje automático para la estimación del riesgo de cáncer de pulmón, concluyendo que la combinación de modelos puede ofrecer mejoras respecto a enfoques individuales.

La investigación reciente también ha puesto de manifiesto la importancia de comprender qué variables influyen en el proceso de predicción. En este sentido, trabajos como el de Ganie y Pramanik [8] muestran el interés creciente por complementar el rendimiento predictivo con análisis que permitan comprender mejor los factores asociados al riesgo de desarrollar la enfermedad.

De forma paralela, la literatura ha identificado diversos retos metodológicos asociados a este tipo de problemas. Entre ellos destaca el desbalanceo de clases, una situación habitual en aplicaciones biomédicas que ha motivado la utilización de distintas estrategias de balanceo con el objetivo de mejorar la capacidad discriminativa de los modelos [7, 9].

Una visión general de estas tendencias puede encontrarse en la revisión sistemática realizada por Islam et al. [9], donde se observa una clara predominancia de modelos basados en árboles de decisión y métodos ensemble,

especialmente Random Forest y XGBoost. Asimismo, la revisión pone de manifiesto la creciente importancia del análisis de variables predictivas y de la validación rigurosa de los modelos desarrollados.

Con el objetivo de situar la propuesta desarrollada en este trabajo respecto a los principales estudios revisados, la Tabla 2.1 resume algunas de las características metodológicas más relevantes de cada uno de ellos.

Tabla 2.1: Comparación de las principales características metodológicas de los trabajos revisados

Trabajo	**Comparación extensa de modelos**	**Comparación sistemática del balanceo**	**Redes neuronales**	**Análisis de variables**	**Análisis integral**
Dritsas y Trigka [7]	Sí	Parcial	Sí	Sí	No
Tu et al. [3]	Sí	No	Sí	Sí	No
Ganie y Pramanik [8]	Sí	No	No	Sí	No
Propuesta de este trabajo	**Sí**	**Sí**	**Sí**	**Sí**	**Sí**

La columna Análisis integral hace referencia a aquellos trabajos que combinan en un mismo marco experimental el análisis exploratorio de los datos, el estudio de la relevancia de las variables, la comparación sistemática de distintas estrategias de balanceo y la evaluación de diferentes familias de modelos de aprendizaje automático. En este sentido, no se considera suficiente la aplicación de una única técnica de balanceo, sino el análisis comparado de varias alternativas y de su impacto sobre el rendimiento de los modelos.

A partir de esta revisión bibliográfica, se observa que los estudios revisados suelen centrarse en aspectos concretos del problema, como la evaluación de determinados algoritmos, el análisis de factores de riesgo o la mejora del rendimiento predictivo. El presente trabajo adopta una perspectiva más amplia, combinando el análisis exploratorio del conjunto de datos, el estudio de la relevancia de las variables, la evaluación de distintas estrategias de balanceo y la comparación sistemática de múltiples familias de modelos de aprendizaje automático, incluyendo modelos clásicos, métodos ensemble y redes neuronales, sobre un mismo conjunto de datos.

Este planteamiento permite obtener una visión global del comportamiento de las diferentes técnicas estudiadas y proporciona una base objetiva para analizar qué estrategias resultan más adecuadas para la predicción del riesgo de cáncer de pulmón en conjuntos de datos tabulares. Además, permite estudiar no solo qué modelo obtiene los mejores resultados, sino también cómo influyen aspectos metodológicos como las técnicas de balanceo en el rendimiento final de los modelos.

Capítulo 3

Fundamentos teóricos del aprendizaje automático

En este capítulo se presentan los fundamentos teóricos del aprendizaje automático y las técnicas de balanceo de datos, siguiendo principalmente el enfoque descrito en el libro de Aurélien Géron[10].

3.1. Introducción al aprendizaje automático

Tal y como se recoge en [11], el ML es una rama de la inteligencia artificial (IA) centrada en algoritmos que aprenden características y patrones a partir de los datos. El aprendizaje automático supuso un cambio de paradigma dentro de la inteligencia artificial, dando lugar a lo que se conoce a día de hoy como IA subsimbólica.

Dentro del ML, podemos encontrar cuatro grandes familias; aprendizaje supervisado, aprendizaje no supervisado, aprendizaje semisupervisado y aprendizaje por refuerzo, en función de la naturaleza de sus objetivos de entrenamiento y a menudo del tipo de datos de entrenamiento que implican.

- Aprendizaje Supervisado: En este paradigma tenemos datos etiquetados. Se entrena un modelo para predecir la salida correcta para una entrada determinada. Algoritmos destacables son las redes neuronales 3.3 y árboles de decisión 3.2.4.
- Aprendizaje no supervisado: En paradigma los datos no disponen de etiquetas. Por lo que se entrena un modelo para discernir patrones intrínsecos, dependencias y correlaciones en los datos.

- Aprendizaje semisupervisado: Es un paradigma intermedio entre el aprendizaje supervisado y el no supervisado, en el que se dispone de un subconjunto de datos etiquetados y otro subconjunto de datos no etiquetados [12].
- Aprendizaje por refuerzo: En este paradigma podemos considerar el modelo como un agente que se entrena con datos que recibe del entorno en el que opera, este entorno es una simulación. El agente o modelo toma una acción que obtendrá la mayor recompensa evaluando su entorno.

Los algoritmos de aprendizaje supervisado entrenan modelos para tareas de predicción. En este caso nuestro trabajo se basa en un problema de clasificación donde queremos predecir la categoría o la clase, es decir valores discretos a diferencia de los modelos de regresión que buscan predecir valores continuos.

Algunos ejemplos de algoritmos de regresión tradicionales son la regresión lineal simple o regresión lineal múltiple. En el otro lado, ejemplos de algoritmos de clasificación tradicionales pueden ser la regresión logística o K-Nearest Neighbors [13]. Aunque muchos algoritmos de ML se pueden utilizar para cualquiera de las dos tareas. Como es el caso de la salida de un algoritmo de regresión puede utilizarse posteriormente para realizar una predicción de clasificación.

Las tareas de clasificación dentro del modelado predictivo se diferencian entre sí según la cantidad de clases posibles y el nivel de exclusividad que existe entre dichas clases. De este modo, tenemos:

- Clasificación binaria, que clasifica los datos en dos categorías exclusivas [14].
- Clasificación multiclase, que clasifica los datos en más de dos categorías exclusivas [15].
- Clasificación multietiqueta, que clasifica los datos en categorías no exclusivas [16].
- Clasificación multisalida, generalización de multietiqueta donde cada etiqueta puede tener más de una clase [10].

3.1.1. Fases de un problema de machine learning

En esta sección se describen las etapas fundamentales que suelen conformar el ciclo de vida de un proyecto de aprendizaje automático [10].

- **Definición del problema y visión global del proyecto.** En esta fase inicial se establece el objetivo del proyecto, se define el problema que se desea resolver y se determina el tipo de algoritmo más adecuado para abordarlo. Asimismo, se seleccionan las métricas que permitirán evaluar el rendimiento del modelo y se realiza una estimación del esfuerzo necesario para su desarrollo, ajuste y validación.

- **Obtención de los datos.** En esta etapa se recopilan los datos necesarios para el desarrollo del modelo. Esto puede implicar la descarga de conjuntos de datos públicos, la extracción de información desde bases de datos o la recopilación mediante APIs u otras fuentes. Además, se configura el entorno de trabajo que permitirá almacenar y procesar dichos datos.

- **Exploración y visualización de los datos.** Una vez obtenidos los datos, se realiza un análisis exploratorio con el objetivo de comprender su estructura, identificar patrones y detectar posibles anomalías. Para ello se utilizan herramientas estadísticas y técnicas de visualización, como gráficos de distribución, diagramas de correlación o representaciones gráficas que facilitan la interpretación de las relaciones entre variables.

- **Preparación y preprocesamiento de los datos.** Los datos obtenidos suelen presentar inconsistencias, valores faltantes o diferentes formas de representar una misma información. En esta fase se llevan a cabo procesos de limpieza y transformación de los datos. Asimismo, muchas técnicas de aprendizaje automático requieren variables numéricas, por lo que es necesario convertir variables categóricas mediante técnicas de codificación. Estas transformaciones pueden automatizarse mediante pipelines de procesamiento de datos.

- **Selección y entrenamiento del modelo.** En esta fase se seleccionan uno o varios algoritmos de aprendizaje automático adecuados al problema planteado. Posteriormente, dichos modelos se entrenan con los datos disponibles y su rendimiento se evalúa mediante técnicas como la validación cruzada. Además, durante este proceso es posible optimizar los hiperparámetros de cada modelo con el fin de maximizar su rendimiento respecto a una métrica de evaluación determinada.

- **Presentación de la solución.** Tras obtener un modelo satisfactorio, se documentan los resultados y se presentan las conclusiones del análisis, destacando las métricas obtenidas, las decisiones adoptadas y las posibles limitaciones del modelo.

- **Despliegue, monitorización y mantenimiento del sistema.** Finalmente, el modelo puede integrarse en un sistema o aplicación (por

ejemplo, una aplicación web o un servicio de predicción). En esta fase es necesario monitorizar su rendimiento y realizar mantenimiento periódico, ya que con el tiempo los datos pueden cambiar o volverse obsoletos, lo que puede afectar a la precisión del modelo.

3.2. Modelos de aprendizaje automático

En esta sección se presentan los fundamentos teóricos de los modelos de aprendizaje automático utilizados, describiendo su funcionamiento interno, sus principales componentes y los principios matemáticos en los que se basan.

3.2.1. Modelos lineales

Los modelos lineales se utilizan para describir la relación entre una variable Y (la etiqueta) y un conjunto de variables predictoras [13]. Estos modelos presentan ventajas notables en términos de interpretabilidad e inferencia y, en muchos problemas reales, pueden resultar sorprendentemente competitivos en comparación con métodos no lineales. Un ejemplo de este tipo de modelos, que se describirá a continuación, es la regresión logística.

Regresión logística

La regresión logística es un modelo que puede utilizarse para estimar la probabilidad de que una observación pertenezca a una determinada clase. Para ello, el modelo calcula una suma ponderada de las características de entrada y posteriormente aplica una función logística sobre el resultado.

En primer lugar, el modelo calcula una puntuación lineal z, que corresponde a la combinación lineal de las variables de entrada junto con un término de sesgo:

$$z = w_0 + w_1 x_1 + w_2 x_2 + \cdots + w_n x_n$$

donde $x_1, x_2, \ldots, x_n$ representan las características de entrada, $w_1, w_2, \ldots, w_n$ son los pesos asociados a cada característica y w_0 es el término independiente o sesgo del modelo.

Posteriormente, este valor z se introduce en la función logística o sigmoide:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

La función sigmoide transforma cualquier número real en un valor comprendido entre 0 y 1. Este valor puede interpretarse como la probabilidad estimada de que la observación pertenezca a la clase positiva.

Finalmente, el modelo realiza la clasificación comparando la probabilidad obtenida con un umbral, que habitualmente se establece en 0.5. De este modo:

$$\hat{y} = \begin{cases} 1 & \text{si } \sigma(z) \geq 0.5 \\ 0 & \text{si } \sigma(z) < 0.5 \end{cases}$$

Por tanto, si la probabilidad estimada es mayor o igual que 0.5, el modelo clasifica la observación como perteneciente a la clase positiva (1); en caso contrario, se clasifica como perteneciente a la clase negativa (0).

En la práctica, aunque el modelo sigue el funcionamiento descrito anteriormente, es necesario ajustar ciertos parámetros que determinan cómo se calculan los coeficientes y cómo se comporta el modelo durante el entrenamiento. Estos parámetros, conocidos como hiperparámetros, permiten adaptar el modelo al conjunto de datos y mejorar su rendimiento.

3.2.2. Modelos basados en distancias

Los modelos basados en distancias son aquellos que realizan predicciones a partir de la similitud entre observaciones, comparando un nuevo punto con los ejemplos más cercanos del conjunto de datos. Estos modelos pueden utilizarse tanto para clasificación como para regresión. Un ejemplo representativo de este enfoque es el algoritmo K-Nearest Neighbors (KNN).

K-Nearest Neighbors

En línea con [17], KNN es un modelo cuyo objetivo es identificar los vecinos más cercanos de un punto de consulta determinado, de modo que podamos asignar una etiqueta de clase a ese punto.

Para calcular la distancia entre puntos se pueden utilizar varias medidas como [18] :

- Distancia euclidiana: es una de las medidas de distancia más utilizadas y se aplica a vectores de valores numéricos. Mediante la siguiente expresión, se calcula la distancia en línea recta entre dos puntos en un espacio n-dimensional.

$$d(x, y) = \sqrt{\sum_{i=1}^{n}(x_i - y_i)^2}$$

- La distancia de Manhattan mide la distancia entre dos puntos como la suma de los desplazamientos necesarios en cada dimensión, en lugar de considerar la distancia en línea recta. Cuya expresión se representa:

$$d(x, y) = \sum_{i=1}^{n} |x_i - y_i|$$

Además hay que definir el valor de k, es decir, cuántos vecinos se verificarán para determinar la clasificación de un punto de consulta específico. Se recomienda un número impar de k para evitar empates y el valor de k dependerá de la naturaleza de los datos ya que si hay mucho ruido, suelen obtenerse mejores resultados con valores altos de k.

3.2.3. Máquina de vectores de soporte

Máquinas de vectores de soporte (SVM)

Como se explica en [19], este modelo clasifica los datos mediante la búsqueda de una línea o un hiperplano óptimo que maximiza la separación entre clases en un espacio N-dimensional.

En general, pueden encontrarse múltiples hiperplanos capaces de separar las clases, pero el objetivo de SVM es seleccionar aquel que maximiza el margen entre los puntos de distintas clases. Esta propiedad favorece una mejor capacidad de generalización sobre datos no vistos. Los puntos más cercanos al hiperplano óptimo se denominan vectores de soporte, ya que determinan tanto la posición del hiperplano como la anchura del margen.

Matemáticamente, el hiperplano de decisión en SVM lineales se define como:

$$\mathbf{w} \cdot \mathbf{x} + b = 0$$

donde $\mathbf{w}$ es el vector de pesos que define la orientación del hiperplano, $\mathbf{x}$ es el vector de características de entrada y b es el término de sesgo.

Para garantizar una correcta clasificación en el caso de margen duro, se impone la siguiente restricción normalizada:

$$y_i(\mathbf{w} \cdot \mathbf{x}_i + b) \geq 1$$

donde $y_i \in \{-1, +1\}$ representa la etiqueta de clase de la muestra $\mathbf{x}_i$. Bajo esta formulación, los hiperplanos de soporte vienen dados por:

$$\mathbf{w} \cdot \mathbf{x} + b = 1 \quad \text{y} \quad \mathbf{w} \cdot \mathbf{x} + b = -1$$

La distancia total entre estos hiperplanos define el margen, que viene dado por:

$$\frac{2}{\|\mathbf{w}\|}$$

En muchos problemas reales los datos no son perfectamente separables. Para abordar esta situación se introduce el concepto de margen blando, que permite cierta tolerancia al error mediante variables de holgura ξ_i. En este caso, la restricción se modifica como:

$$y_i(\mathbf{w} \cdot \mathbf{x}_i + b) \geq 1 - \xi_i$$

El hiperparámetro C controla el equilibrio entre la maximización del margen y la penalización de los errores de clasificación. Valores altos de C penalizan fuertemente los errores, generando márgenes más estrechos, mientras que valores bajos permiten mayor flexibilidad a costa de aceptar más errores.

En situaciones donde los datos no son separables linealmente, se recurre a las SVM no lineales. Estas proyectan los datos a un espacio de características de mayor dimensión donde es posible encontrar una separación lineal. Este proceso se implementa mediante el denominado *kernel trick*, que permite calcular productos escalares en el espacio transformado sin necesidad de realizar explícitamente la transformación. Entre las funciones kernel más utilizadas se encuentran la lineal, la polinómica, la de función de base radial (RBF) y la sigmoide.

3.2.4. Modelos basados en árboles

Los métodos basados en árboles consisten en segmentar el espacio de las variables predictoras en un conjunto de regiones más simples. Para realizar

una predicción sobre una nueva observación, el modelo utiliza la información de las muestras de entrenamiento que pertenecen a la región correspondiente: en problemas de regresión se emplea la media o moda de sus valores [13].

Decision Tree

Los árboles de decisión son algoritmos muy versátiles que permiten realizar tareas de clasificación, regresión e incluso problemas multisalida. Presentan una estructura jerárquica en forma de árbol compuesta por un nodo raíz, nodos internos, ramas y nodos hoja.

Los árboles de decisión se basan en una estrategia de divide y vencerás. A partir de un conjunto de instancias de entrenamiento etiquetadas, el algoritmo comienza con todos los datos en un único nodo raíz y selecciona el atributo que mejor separa las clases. A continuación, divide el conjunto en subconjuntos disjuntos según los valores de dicho atributo y repite el proceso de forma recursiva en cada rama. El objetivo es generar subconjuntos cada vez más homogéneos, hasta que las instancias pertenezcan mayoritariamente a una misma clase o se alcance un criterio de parada [20].

El algoritmo CART, Classification and Regression Tree, disponible en la librería de Scikit-Learn selecciona, en cada nodo, la característica k y el umbral t_k que minimizan la siguiente función de coste:

$$J(k, t_k) = \frac{m_{\text{left}}}{m} G_{\text{left}} + \frac{m_{\text{right}}}{m} G_{\text{right}}$$

donde m es el número total de muestras del nodo, m_{left} y m_{right} representan el número de muestras asignadas a los subconjuntos izquierdo y derecho, respectivamente, y G_{left} y G_{right} corresponden a la impureza de cada uno de esos subconjuntos.

Una de las medidas de impureza más utilizadas es la entropía. En aprendizaje automático, la entropía mide el grado de mezcla entre las clases presentes en un nodo. Si todas las instancias pertenecen a la misma clase, la entropía es igual a cero, lo que indica que el nodo es completamente puro.

Los árboles de decisión suelen considerarse modelos no paramétricos, ya que el número de parámetros no está fijado antes del entrenamiento y depende de la estructura final del árbol. Debido a esta flexibilidad, el modelo puede ajustarse excesivamente a los datos de entrenamiento, fenómeno conocido como sobreajuste. Para evitarlo, es habitual regular el modelo mediante distintos hiperparámetros, como la profundidad máxima del árbol o el número mínimo de muestras por nodo.

Bagging

Es una técnica de aprendizaje en conjunto cuyo objetivo principal es reducir la varianza de los modelos y mejorar su capacidad de generalización. Se basa en entrenar varios modelos del mismo tipo, habitualmente árboles de decisión, sobre diferentes subconjuntos del conjunto de entrenamiento.

Cada uno de estos subconjuntos se genera mediante muestreo con reemplazo(bootstrap), lo que significa que las instancias se seleccionan aleatoriamente del conjunto original permitiendo que una misma muestra pueda aparecer varias veces, mientras que otras pueden no ser seleccionadas. De este modo, cada modelo se entrena con una versión ligeramente distinta de los datos.

Una vez entrenados todos los modelos, sus predicciones se combinan para obtener una respuesta final más robusta. En problemas de clasificación, la salida final suele obtenerse mediante votación mayoritaria, mientras que en regresión se utiliza normalmente la media de las predicciones individuales.

Además, este modelo permite realizar una evaluación interna del modelo mediante las muestras out-of-bag (OOB). Como cada modelo se entrena con una muestra bootstrap, algunas instancias del conjunto original quedan fuera de esa muestra. Estas instancias no utilizadas pueden emplearse posteriormente para evaluar el modelo correspondiente, proporcionando una estimación del rendimiento sin necesidad de utilizar un conjunto de validación adicional.

El bagging es especialmente eficaz en modelos de alta varianza, como los árboles de decisión, porque al combinar múltiples árboles entrenados sobre distintas muestras reduce de forma notable el sobreajuste. Además, suele ofrecer un rendimiento superior al de un árbol individual, especialmente en problemas con atributos numéricos o binarios y, en muchos casos, también en problemas con variables categóricas de múltiples valores [21].

Random Forest

Random Forest es un método de aprendizaje en conjunto basado en árboles de decisión. A diferencia de un único árbol, este modelo construye varios clasificadores y combina sus predicciones para obtener una decisión final más robusta. En problemas de clasificación, esta combinación suele realizarse mediante votación mayoritaria entre los árboles del bosque.

El método puede entenderse como una extensión del bagging, ya que cada árbol se entrena sobre una muestra aleatoria del conjunto de entrenamiento generada con reemplazo. Además, Random Forest introduce una segunda

fuente de aleatoriedad: en cada nodo, la mejor división se busca únicamente entre un subconjunto aleatorio de variables, en lugar de considerar todas las características disponibles.

Esta estrategia aumenta la diversidad entre los árboles y reduce la dependencia entre ellos, lo que ayuda a mejorar la capacidad de generalización del modelo. Por ello, Random Forest suele ser más estable que un árbol de decisión individual. Además, puede proporcionar información sobre la importancia relativa de las características utilizadas en la clasificación [22].

Extra Trees

Extra Trees es un modelo similar a Random Forest, pero introduce una aleatoriedad adicional en la selección de los umbrales de decisión. En cada nodo, además de trabajar con un subconjunto aleatorio de características, las divisiones se realizan a partir de umbrales elegidos aleatoriamente en lugar de optimizados.

Al seleccionarse los umbrales de decisión de forma aleatoria, en lugar de buscarse el óptimo en cada nodo, el sesgo del modelo puede aumentar, pero la varianza se reduce al incrementarse la diversidad entre los árboles. Además, el proceso de entrenamiento suele ser más rápido, ya que se evita el coste computacional asociado a la búsqueda exhaustiva del mejor umbral de partición.

Métodos boosting basados en árboles

Los métodos de boosting basados en árboles consisten en entrenar de forma secuencial un conjunto de árboles débiles, donde cada nuevo modelo se centra en corregir los errores cometidos por los anteriores, combinándose posteriormente sus predicciones para obtener un resultado final más preciso [23].

AdaBoost

AdaBoost es un método de aprendizaje en conjunto basado en el entrenamiento secuencial de predictores débiles, donde cada nuevo predictor intenta corregir los errores cometidos por los anteriores. Para lograrlo, en cada iteración se asigna un mayor peso a las instancias mal clasificadas, obligando al siguiente modelo a centrarse en los ejemplos más difíciles.

Una vez entrenados todos los predictores, el resultado final se obtiene combi-

nando sus predicciones de forma ponderada. A diferencia de métodos como el bagging, en AdaBoost cada predictor recibe un peso en función de su precisión, de modo que aquellos modelos con mejor desempeño tienen mayor influencia en la predicción final.

Inicialmente, todas las instancias del conjunto de entrenamiento reciben el mismo peso. Tras entrenar cada predictor, se calcula su error ponderado sobre los datos de entrenamiento:

$$r_j = \frac{\sum_{i=1}^{m} w_i \, \mathbf{1}(\hat{y}_i^{(j)} \neq y_i)}{\sum_{i=1}^{m} w_i} \tag{3.1}$$

donde m es el número total de instancias, w_i representa el peso asociado a la instancia i, y_i es la etiqueta real y $\hat{y}_i^{(j)}$ es la predicción del predictor j.

A partir de este error se calcula el peso del predictor dentro del conjunto:

$$\alpha_j = \eta \log\left(\frac{1 - r_j}{r_j}\right) \tag{3.2}$$

donde η es la tasa de aprendizaje. Los predictores con menor error obtienen un mayor peso, por lo que tendrán mayor influencia en la predicción final.

Posteriormente, el algoritmo actualiza los pesos de las instancias del conjunto de entrenamiento. Las instancias correctamente clasificadas mantienen su peso, mientras que las mal clasificadas potencian sus pesos:

$$w_i \leftarrow w_i \exp(\alpha_j) \tag{3.3}$$

De este modo, los ejemplos que han sido clasificados incorrectamente adquieren mayor importancia en la siguiente iteración, obligando al próximo predictor a centrarse en los casos más difíciles.

Este proceso se repite de forma iterativa hasta alcanzar el número deseado de predictores. Finalmente, la predicción del modelo se obtiene mediante una votación ponderada entre todos los predictores:

$$\hat{y}(x) = \arg\max_{k} \sum_{j=1}^{N} \alpha_j \, \mathbf{1}(\hat{y}_j(x) = k) \tag{3.4}$$

donde N es el número total de predictores y α_j representa el peso asociado a cada uno de ellos. La clase final será aquella que obtenga la mayor suma de votos ponderados.

Histogram-Based Gradient Boosting

Como se describe en [24], Gradient Boosting es un método de aprendizaje en conjunto basado en la construcción secuencial de predictores débiles, donde cada nuevo modelo se entrena para corregir los residuos o errores del conjunto anterior. Para ello, optimiza de forma aditiva una función de pérdida ajustando nuevos predictores sobre dichos residuos.

Histogram-Based Gradient Boosting es una variante optimizada de Gradient Boosting diseñada para mejorar la eficiencia computacional, especialmente en conjuntos de datos de gran tamaño. La diferencia radica en como procesa las variables de entrada.

En lugar de evaluar todos los posibles puntos de división de manera exacta, este algoritmo agrupa previamente los valores continuos en un número limitado de intervalos, construyendo así histogramas. Gracias a ello, se reduce considerablemente el número de particiones que deben evaluarse en cada nodo, lo que acelera el entrenamiento y disminuye el coste computacional. Esta estrategia puede hacer que el algoritmo sea mucho más rápido que en conjuntos de datos grandes.

Además, HistGradientBoosting incorpora soporte nativo para valores ausentes. Durante el entrenamiento, el algoritmo aprende en cada división si las observaciones con valores perdidos deben enviarse a la rama izquierda o derecha en función de la ganancia obtenida.

XGBoost

De acuerdo con lo expuesto en [25] y en la documentación oficial de XGBoost [26], XGBoost es una biblioteca de software de código abierto, optimizada y distribuida, que implementa algoritmos basados en árboles de decisión potenciados por gradiente. Gracias a ello, puede ofrecer un alto poder predictivo y un buen rendimiento en problemas con relaciones no lineales entre las variables.

Los árboles potenciados por gradiente constituyen un método de boosting en el que los modelos se construyen de forma secuencial y aditiva. Para minimizar el error, en cada iteración se calcula una función de pérdida que mide la diferencia entre las predicciones del modelo y los valores reales.

El término gradiente hace referencia al uso del descenso de gradiente como mecanismo de optimización. En cada etapa, el algoritmo ajusta un nuevo árbol con el objetivo de minimizar la función de pérdida, guiándose por la dirección en la que esta disminuye con mayor rapidez. De este modo, cada

nuevo árbol se entrena para corregir los residuos o errores generados por el conjunto de árboles previamente construido.

CatBoost

CatBoost es un algoritmo de aprendizaje automático basado en gradient boosting, que utiliza árboles de decisión binarios como predictores base. Su objetivo es construir un modelo capaz de minimizar una función de pérdida, ajustando de forma progresiva una secuencia de modelos débiles.

El entrenamiento se realiza de manera iterativa y aditiva. En cada iteración, el nuevo árbol se incorpora al modelo anterior con el propósito de reducir el error cometido hasta ese momento. Para ello, el algoritmo se apoya en métodos de optimización basados en el gradiente, como el descenso por gradiente o aproximaciones de segundo orden como el método de Newton.

Una de las características principales de CatBoost es su capacidad para trabajar de forma eficiente con variables categóricas, reduciendo la necesidad de aplicar transformaciones manuales previas. Esto lo convierte en un modelo especialmente interesante en problemas donde existen características categóricas y relaciones no lineales entre las variables [27].

3.2.5. Métodos de combinación de modelos

Estos métodos se basan en la idea de no buscar el mejor modelo individual, sino en combinar varios modelos con el objetivo de mejorar la capacidad de generalización [23].

Stacking

El stacking es una técnica de ensemble learning en la que varios modelos base, previamente entrenados, generan predicciones que se utilizan posteriormente como entrada de un modelo adicional denominado meta-modelo. Este modelo final aprende a combinar las salidas de los modelos anteriores con el objetivo de obtener una predicción más precisa. De esta forma, el stacking permite aprovechar la información aportada por distintos clasificadores y construir un sistema en varias capas.

Su funcionamiento habitual consiste en entrenar primero los modelos base y, a partir de sus predicciones, crear un nuevo conjunto de datos con el que se entrena el meta-modelo. Este aprende qué peso o relevancia dar a cada predicción en función del comportamiento de los modelos anteriores. Para evitar

sobreajuste, es importante que el meta-modelo se entrene con predicciones obtenidas sobre datos no vistos directamente durante el entrenamiento de la primera capa. En conjunto, el stacking es un método flexible y eficaz para combinar modelos diversos y mejorar el rendimiento predictivo.

Voting Classifier

Un Voting Classifier es un método de ensemble learning que combina varios clasificadores previamente entrenados. Su funcionamiento consiste en agregar las predicciones de todos los modelos y seleccionar como resultado final la clase que recibe la mayoría de votos.

Este enfoque puede mejorar el rendimiento respecto a los modelos individuales, especialmente cuando los clasificadores son suficientemente diversos, de modo que los errores de unos no coincidan con los de otros.

Además, cuando todos los modelos pueden estimar probabilidades de clase, es posible utilizar la variante soft voting, que calcula la probabilidad media predicha por todos los clasificadores y selecciona la clase con mayor probabilidad. En muchos casos, este método ofrece mejores resultados que el voto mayoritario simple.

3.3. Introducción a las redes neuronales artificiales

Una red neuronal artificial es un modelo de aprendizaje automático inspirado en las redes neuronales biológicas del cerebro humano. Son versátiles, potentes y escalables, por lo que son de gran utilidad para tareas complejas y de gran volumen.

Para comprender su funcionamiento, es necesario analizar primero sus componentes básicos, comenzando por la neurona artificial, y posteriormente estudiar cómo estas se organizan en estructuras más complejas y cómo aprenden a partir de los datos.

3.3.1. Neurona Artificial

Una neurona artificial es un modelo matemático inspirado en el funcionamiento de las neuronas biológicas. Este modelo recibe una o varias entradas, las procesa y produce una salida. En términos generales, las entradas se

combinan mediante pesos asociados a cada una de ellas y posteriormente se aplica una función que determina la salida de la neurona.

Uno de los modelos más simples basados en este concepto es el perceptrón. El perceptrón se basa en una neurona artificial denominada unidad lógica de umbral. Esta unidad calcula una suma ponderada de las entradas y posteriormente aplica una función escalonada para producir el resultado.

Matemáticamente, la suma ponderada de las entradas puede expresarse como:

$$z = w_1 x_1 + w_2 x_2 + \cdots + w_n x_n + b \tag{3.5}$$

donde x_i representa las entradas, w_i los pesos asociados a cada entrada y b el término de sesgo.

Tras calcular esta suma ponderada, se aplica una función no lineal. La más común es la función de ReLu:

$$\text{ReLu}(z) = \begin{cases} 0 & \text{si } z < 0 \\ 1 & \text{si } z \geq 0 \end{cases} \tag{3.6}$$

En problemas de clasificación binaria, si la suma ponderada supera el umbral establecido, la instancia se clasifica como perteneciente a la clase positiva; en caso contrario, se clasifica como perteneciente a la clase negativa. Cuando todas las neuronas de una capa están conectadas con todas las neuronas de la capa anterior, se dice que se trata de una capa completamente conectada o capa densa.

Gracias al uso del álgebra lineal, es posible calcular de forma eficiente las salidas de una capa completa de neuronas para múltiples instancias simultáneamente. Este proceso puede representarse mediante la siguiente expresión:

$$h_{W,b}(X) = \phi(XW + b) \tag{3.7}$$

donde:

- X representa la matriz de características de entrada, con una fila por instancia y una columna por característica.
- W es la matriz de pesos de las conexiones entre las neuronas de la capa anterior y las neuronas de la capa actual.

- b es el vector de sesgos asociado a las neuronas de la capa.
- ϕ representa la función de activación aplicada a la suma ponderada de las entradas.

El perceptrón se entrena de forma iterativa utilizando los datos de entrenamiento. En cada iteración se introduce una instancia de entrenamiento y el modelo genera una predicción. Si la predicción es incorrecta, se ajustan los pesos de las conexiones con el objetivo de reducir el error cometido.

La regla de actualización de los pesos del perceptrón puede expresarse mediante la siguiente ecuación:

$$w_{i,j} = w_{i,j} + \eta(y_j - \hat{y}_j)x_i \tag{3.8}$$

donde:

- $w_{i,j}$ es el peso de conexión entre la i-ésima neurona de entrada y la j-ésima neurona de salida.
- x_i es el valor de la i-ésima entrada para la instancia de entrenamiento actual.
- y_j es la salida de la j-ésima neurona de salida para la instancia de entrenamiento actual.
- $\hat{y}_j$ es la salida objetivo de la j-ésima neurona de salida para la instancia de entrenamiento actual.
- η es la tasa de aprendizaje.

3.3.2. Redes Neuronales feed-forward

Aunque el perceptrón constituye una base teórica importante, presenta limitaciones significativas, como su incapacidad para resolver problemas no linealmente separables. Para superar estas limitaciones, se introducen arquitecturas más complejas formadas por múltiples neuronas organizadas en capas, dando lugar a los perceptrones multicapa.

Un perceptrón multicapa, también conocido como red neuronal feed-forward, es un tipo de red neuronal artificial compuesta por una capa de entrada, una o varias capas ocultas formadas por unidades lógicas de umbral (ULU) y una capa de salida, encargada de producir la predicción final.

Las capas más próximas a la entrada se denominan capas inferiores, mientras aquellas capas más cercanas a la salida se conocen como capas superiores. En este tipo de arquitectura, la información fluye en una única dirección, desde la entrada hasta la salida, sin ciclos ni retroalimentación.

Además, todas las capas, excepto la capa de salida, incluyen una neurona de sesgo, la cual está completamente conectada con las neuronas de la capa siguiente y permite ajustar el modelo de forma más flexible.

El entrenamiento de las redes neuronales feed-forward se realiza mediante la minimización de una función de pérdida. En problemas de clasificación binaria, es habitual utilizar la función de pérdida Binary Cross Entropy que cuantifica la diferencia entre las etiquetas reales y las probabilidades predichas por el modelo. Cuanto menor es su valor, mejor se ajustan las predicciones a las etiquetas reales.

Matemáticamente, la entropía cruzada binaria (Binary Cross-Entropy, BCE) se define como:

$$\text{BCE} = -\frac{1}{N}\sum_{i=1}^{N}\left[y_i \log(p_i) + (1-y_i)\log(1-p_i)\right] \tag{3.9}$$

donde:

- N es el número de observaciones.
- y_i es la etiqueta real (0 o 1) de la observación i.
- p_i es la probabilidad predicha de que la observación i pertenezca a la clase 1.

3.3.3. Propagación del error

Para llevar a cabo esta minimización, se emplea el algoritmo de retropropagación (backpropagation), que permite determinar cómo deben ajustarse los pesos de las conexiones y los términos de sesgo con el fin de reducir el error. Para ello, calcula el gradiente de la función de pérdida con respecto a cada parámetro del modelo.

El algoritmo toma un subconjunto del conjunto de entrenamiento, denominado mini-batch. Sus entradas se introducen en la red y se propagan capa a capa hasta obtener la salida. Durante este proceso se calculan y almacenan los valores intermedios de cada neurona, necesarios para la fase posterior.

La propagación hacia delante en una capa de la red puede expresarse como:

$$a^{(k)} = b^{(k)} + W^{(k)} h^{(k-1)} \tag{3.10}$$

$$h^{(k)} = f(a^{(k)}) \tag{3.11}$$

donde $W^{(k)}$ representa la matriz de pesos de la capa k, $b^{(k)}$ el vector de sesgos, $h^{(k-1)}$ la salida de la capa anterior y f la función de activación aplicada a cada neurona [28].

Posteriormente, se evalúa el error del modelo comparando la salida predicha con los valores reales mediante una función de pérdida, en nuestro caso *Binary Cross Entropy*. El error se propaga hacia atrás a través de la red utilizando la regla de la cadena, lo que permite calcular el gradiente de la función de pérdida con respecto a todos los parámetros del modelo:

$$\frac{\partial z}{\partial x_i} = \sum_j \frac{\partial z}{\partial y_j} \frac{\partial y_j}{\partial x_i} \tag{3.12}$$

Esta expresión indica que la variación de la salida z respecto a una variable x_i puede calcularse a partir de las derivadas parciales intermedias de las variables y_j, permitiendo propagar el error desde la salida de la red hasta las capas iniciales.

Se ajustan los pesos y sesgos de la red mediante el algoritmo de descenso de gradiente, desplazándolos en la dirección opuesta al gradiente con el objetivo de reducir el error.

Este procedimiento se repite para todos los mini-batches del conjunto de entrenamiento. Cuando se ha procesado el conjunto completo, se considera que se ha completado una época (*epoch*).

3.3.4. Descenso de gradiente

Para que una red neuronal sea capaz de realizar predicciones útiles, es necesario entrenarla. Este proceso de entrenamiento se lleva a cabo mediante algoritmos de optimización, siendo uno de los más utilizados el descenso de gradiente, con el fin de minimizar la función de coste modificando de manera iterativa los pesos y sesgos del modelo [28].

La idea principal de este método es ajustar iterativamente los parámetros del modelo realizando pequeños desplazamientos en la dirección en la que la

función de pérdida disminuye más rápidamente, con el objetivo de alcanzar un mínimo local.

Sea una función de pérdida $f : \mathbb{R}^n \to \mathbb{R}$, continua y derivable. Su gradiente viene dado por:

$$\nabla f(w_1, \ldots, w_n) = \left(\frac{\partial f}{\partial w_1}, \ldots, \frac{\partial f}{\partial w_n} \right) \tag{3.13}$$

La dirección de máximo descenso corresponde al gradiente negativo:

$$-\eta \nabla f(w_1, \ldots, w_n) \tag{3.14}$$

donde η representa la tasa de aprendizaje (*learning rate*), encargada de controlar el tamaño de los pasos realizados durante la optimización.

A partir de unos valores iniciales, los parámetros del modelo se actualizan iterativamente según la expresión:

$$w_i = w_{i-1} - \eta \nabla f(w_{i-1}) \tag{3.15}$$

Este procedimiento se repite hasta que la función de pérdida converge hacia un mínimo local.

Existen distintas variantes del descenso de gradiente, entre las que destacan SGD, RMSProp y Adam. En este trabajo se ha utilizado Adam (*Adaptive Moment Estimation*).

Este optimizador se ha elegido porque suele ofrecer un entrenamiento estable y eficiente en redes neuronales, especialmente en problemas con muchos parámetros. Además, requiere menos ajuste manual de la tasa de aprendizaje [29].

3.3.5. Funciones de activación

En problemas de regresión lineal, la predicción del modelo se obtiene como una combinación lineal de las variables de entrada, lo que permite generar valores continuos no acotados. Sin embargo, en problemas de clasificación, se requiere predecir etiquetas discretas o probabilidades comprendidas en el intervalo [0,1].

Para abordar este problema, se introduce una función no lineal, denominada función de activación, que transforma la combinación lineal de las entradas

en un valor acotado. En redes neuronales feed-forward modernas, se ha dejado de utilizar la función escalonada de Heaviside, ya que presenta tramos constantes en los que su derivada es nula, lo que impide calcular el gradiente y dificulta el proceso de aprendizaje mediante descenso de gradiente.

Por este motivo, se emplean funciones de activación continuas y diferenciables (o casi en todos sus puntos), como las siguientes:

- **Función sigmoide o logística:** Permite calcular el gradiente de forma efectiva, ya que su derivada es distinta de cero en la mayor parte de su dominio. Esto facilita el aprendizaje del modelo mediante descenso de gradiente. Su expresión es:

$$\sigma(z) = \frac{1}{1 + \exp(-z)}$$

- **Función tangente hiperbólica:** Presenta una forma similar a la sigmoide, pero su rango de salida está comprendido entre -1 y 1. Es continua y diferenciable, lo que favorece el entrenamiento. Además, al estar centrada en cero, puede acelerar la convergencia del modelo. Su expresión es:

$$\tanh(z) = 2\sigma(2z) - 1$$

- **Función ReLU (Rectified Linear Unit):** Se define como:

$$\text{ReLU}(z) = \text{máx}(0, z)$$

  Es continua, aunque no es diferenciable en $z = 0$. Su derivada es nula para valores negativos y constante para valores positivos. A pesar de ello, es la función más utilizada en la práctica debido a su simplicidad y eficiencia computacional.

3.4. Problema del desbalanceo de datos

En numerosos escenarios reales, los conjuntos de datos presentan desbalanceo de clases, es decir, una distribución desigual en el número de instancias pertenecientes a cada clase. Esta situación introduce dificultades significativas en el proceso de aprendizaje, ya que muchos algoritmos están diseñados para optimizar métricas globales como la accuracy, lo que puede derivar en un rendimiento aparentemente elevado mientras se ignora el comportamiento sobre la clase minoritaria.

Además, los modelos de aprendizaje automático suelen incorporar sesgos inductivos que favorecen soluciones simples y la correcta clasificación de los casos más frecuentes. Como consecuencia, los patrones asociados a la clase minoritaria a menudo más complejos y minoritarios quedan ocultos o poco representados durante el entrenamiento. Este fenómeno provoca que los modelos tiendan a clasificar incorrectamente instancias minoritarias, reduciendo su capacidad de generalización en escenarios donde dichas clases son de especial interés.

3.4.1. Técnicas de balanceo de datos

En este apartado se describen las principales técnicas empleadas para abordar el problema del desbalanceo de clases [30]. En particular, se analizarán distintas estrategias de sampling, cuyo objetivo es generar conjuntos de datos más equilibrados. Estas técnicas permiten mejorar la representación de la clase minoritaria y favorecer una mejor aproximación de los límites de decisión entre clases.

Técnicas de Oversampling

Las técnicas de oversampling tienen como objetivo generar nuevas muestras de la clase minoritaria. Una de las técnicas más sencillas es el random oversampling, que consiste en duplicar de forma aleatoria instancias existentes de la clase minoritaria. Este método permite aumentar su representación en el conjunto de datos, evitando que quede "oculta" frente a la clase mayoritaria durante el proceso de aprendizaje.

No obstante, esta técnica presenta limitaciones, ya que la duplicación de instancias puede inducir sobreajuste (overfitting), provocando que el modelo aprenda patrones específicos del conjunto de entrenamiento y no generalice adecuadamente a nuevos datos. Para superar estos inconvenientes, se desarrollaron métodos más avanzados como SMOTE (Synthetic Minority Over-sampling Technique).

SMOTE genera nuevas instancias sintéticas a partir de ejemplos existentes de la clase minoritaria. Para ello, selecciona una instancia de la clase minoritaria de manera aleatoria y calcula sus k vecinos más cercanos dentro de la misma clase. A continuación, se elige uno de estos vecinos y se genera una nueva muestra interpolando entre ambos puntos en el espacio de características, mediante una combinación convexa de sus atributos.

Debido a su efectividad, han surgido múltiples variantes de SMOTE, como Borderline-SMOTE o Safe-Level-SMOTE. En este trabajo, se ha optado

por utilizar SMOTENC, una extensión de SMOTE que permite manejar adecuadamente variables categóricas.

Técnicas de Undersampling

Estas técnicas consisten en eliminar muestras de la clase mayoritaria. Una de las primeras técnicas desarrolladas en este ámbito es el random undersampling, que elimina de forma aleatoria instancias de la clase mayoritaria. Este método permite reducir el tamaño del conjunto de datos y, en consecuencia, el coste computacional del entrenamiento. Resulta especialmente útil en escenarios donde la clase mayoritaria está sobrerrepresentada y contiene información redundante.

Sin embargo, esta técnica presenta importantes limitaciones, ya que la eliminación aleatoria de instancias puede provocar la pérdida de información relevante y alterar los límites de decisión del modelo. Para eliminar estos problemas, se han propuesto métodos más sofisticados, como el Neighborhood Cleaning Rule (NCR).

El método NCR se basa en el análisis del vecindario de cada instancia. En concreto, para cada muestra del conjunto de entrenamiento se calculan sus k vecinos más cercanos. Si una instancia pertenece a la clase mayoritaria y es clasificada incorrectamente por dichos vecinos, se elimina del conjunto de datos. Por el contrario, si la instancia pertenece a la clase minoritaria y es clasificada erróneamente, se eliminan las instancias de la clase mayoritaria presentes en su vecindario. De este modo, NCR permite depurar el conjunto de datos eliminando tanto instancias ruidosas como ejemplos ambiguos cercanos a la frontera de decisión.

3.5. Métricas de evaluación de modelos

En esta seccion estudiaremos las diferentes métricas utilizadas para evaluar nuestros modelos.

Matriz de confusión

La matriz de confusión es una métrica utilizada para evaluar el rendimiento de un modelo de clasificación. Se representa mediante una tabla en la que, normalmente, las filas corresponden a las clases reales y las columnas a las clases predichas. Cada celda indica cuántas instancias de una clase real han sido clasificadas en una clase determinada. De este modo, los valores de la

diagonal principal representan los aciertos del modelo, mientras que el resto de valores reflejan los errores de clasificación.

En este contexto aparecen las siguientes medidas:

- VP: instancias de la clase positiva clasificadas correctamente.
- VN: instancias de la clase negativa clasificadas correctamente.
- FP: instancias de la clase negativa clasificadas incorrectamente como positivas.
- FN: instancias de la clase positiva clasificadas incorrectamente como negativas.

En teoría, el objetivo es minimizar tanto los falsos positivos como los falsos negativos. Sin embargo, en la práctica esto no suele ser posible, ya que existe un compromiso entre ambos tipos de error. Por ello, en función del problema planteado y de las consecuencias asociadas, es necesario priorizar uno frente al otro. En nuestro caso, resulta prioritario minimizar los falsos negativos, dado que no detectar una enfermedad puede tener consecuencias más graves que generar un falso diagnóstico positivo.

Precisión

La matriz de confusión proporciona una gran cantidad de información sobre el rendimiento de un modelo. Sin embargo, en algunos casos puede resultar útil emplear métricas más concisas que sinteticen esta información. Una de ellas es la precisión, que mide la proporción de instancias clasificadas como positivas que realmente pertenecen a la clase positiva. Su expresión se define como:

$$\text{Precision} = \frac{VP}{VP + FP} \tag{3.16}$$

Recall

En la mayoría de los casos, la precisión se analiza conjuntamente con otra métrica denominada recall o sensibilidad. Esta métrica mide la proporción de instancias positivas reales que han sido correctamente identificadas por el modelo. Es decir, indica cuántos de los casos verdaderamente positivos han sido detectados. Su expresión se define de la siguiente manera:

$$\text{Recall} = \frac{VP}{VP + FN} \tag{3.17}$$

F1-score

El F1-score es una métrica utilizada para comparar el rendimiento de distintos clasificadores. Se define como la media armónica entre la precisión y el recall, y se expresa de la siguiente manera:

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \tag{3.18}$$

Esta métrica alcanza valores elevados cuando tanto la precisión como el recall son altos, ya que penaliza aquellos casos en los que una de las dos medidas presenta valores bajos. Por ello, resulta especialmente útil cuando se busca un equilibrio entre ambas.

Accuracy

Accuracy es una de las métricas más utilizadas para evaluar el rendimiento de un modelo de clasificación. Mide el porcentaje de aciertos globales del modelo, es decir, la proporción de instancias, tanto positivas como negativas, que han sido clasificadas correctamente.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \tag{3.19}$$

Es un buen indicador del rendimiento global del modelo. Sin embargo, en determinados casos puede resultar engañosa. Esto ocurre especialmente en conjuntos de datos desbalanceados. Por ejemplo, en un dataset con una distribución 70/30, un clasificador que prediga siempre la clase mayoritaria alcanzaría una accuracy del 70 %, a pesar de no estar capturando adecuadamente la clase minoritaria. Por este motivo, es recomendable analizar esta métrica junto con otras como pueden ser precisión o recall.

Curva ROC

La curva ROC es una representación gráfica utilizada para evaluar la capacidad de un modelo de clasificación binaria para distinguir entre dos clases. En esta gráfica, el eje X representa la tasa de falsos positivos False Positive

Rate, FPR), es decir, la proporción de instancias negativas que han sido clasificadas incorrectamente como positivas, mientras que el eje Y representa la tasa de verdaderos positivos (True Positive Rate, TPR), equivalente al recall, esto es, la proporción de instancias positivas que han sido clasificadas correctamente.La curva ROC permite evaluar la capacidad de un modelo para distinguir entre clases, puesto que un buen clasificador puede identificar una elevada proporción de instancias positivas reales manteniendo, al mismo tiempo, una baja tasa de falsos positivos.

En la práctica, al aumentar el recall, también suele incrementarse la tasa de falsos positivos. Para comparar distintos clasificadores, suele emplearse el área bajo la curva ROC (AUC-ROC). Cuanto más próximo a 1 sea este valor, mejor será la capacidad del modelo para distinguir entre las dos clases. Por el contrario, un valor de 0.5 indica que el clasificador no presenta capacidad discriminativa y su comportamiento es aleatorio.

Parte III

Análisis y Desarrollo Experimental

Capítulo 4

Análisis exploratorio de los datos

En este capítulo se presentará el conjunto de datos y se llevará a cabo un análisis exploratorio de los datos, examinando sus distribuciones, correlaciones y relaciones entre variables. A partir de este estudio, se identifican las variables potencialmente más relevantes para la predicción.

4.1. Conjunto de Datos

El dataset empleado se llama Lung-Cancer-Risk-Dataset [31] disponible en la plataforma Kaggle (ver *enlace*). Representan pacientes y factores de riesgo asociados al cáncer de pulmón, tales como la exposición al tabaco, variables ambientales y determinadas condiciones médicas. Se distribuye bajo licencia CC0 1.0 Universal (Dominio Público), permitiendo así su libre uso con fines académicos y de investigación.

El conjunto de datos cuenta con 50000 pacientes y 11 variables. En la tabla 4.1 se describen las variables.

Nombre de la variable	Descripción	Tipo	Rango/Valores
patient_id	Identificador único del paciente.	Integer	100000–149999
age	Edad del paciente (años).	Integer	18–100
gender	Género del paciente.	String	Male, Female
pack_years	Exposición tabáquica (años × paquetes/día).	Float	0–100
radon_exposure	Exposición residencial a radón.	String	Low, Medium, High
asbestos_exposure	Exposición laboral a asbesto.	String	Yes, No
secondhand_smoke_exposure	Exposición al humo de segunda mano.	String	Yes, No
copd_diagnosis	Diagnóstico de EPOC (COPD).	String	Yes, No
alcohol_consumption	Patrón de consumo de alcohol.	String	None, Moderate, Heavy
family_history	Antecedentes familiares de cáncer de pulmón.	String	Yes, No
lung_cancer	Variable objetivo: diagnóstico de cáncer de pulmón.	String	Yes, No

Tabla 4.1: Variables del dataset *Lung-Cancer-Risk-Dataset*.

4.2. Análisis de distribuciones

En esta sección se analizan las distribuciones individuales de las variables, con el objetivo de obtener una primera visión de los factores que pueden estar más relacionados con la aparición de la enfermedad.

4.2.1. Distribuciones univariantes

Variable etiqueta

Empezamos observando en a Figura 4.1 la variable etiqueta, `lung_cancer`. Claramente se aprecia un desbalanceo hacia la clase positiva. En concreto, el valor *Yes* (presencia de cáncer) corresponde con 34.364 observaciones, lo que equivale al $68,73\,\%$ del total de la población, mientras que la clase *No* (ausencia de cáncer) cuenta con 15.636 registros, correspondientes al $31,27\,\%$. Posteriormente, en la sección 6.3, se propondrán diferentes métricas para abordar este problema.

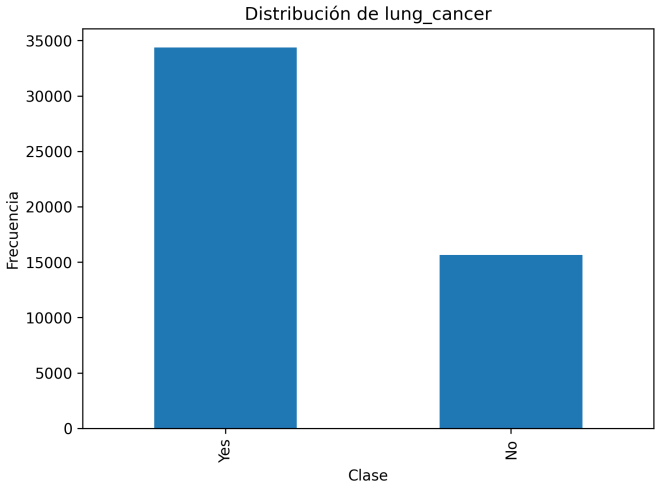


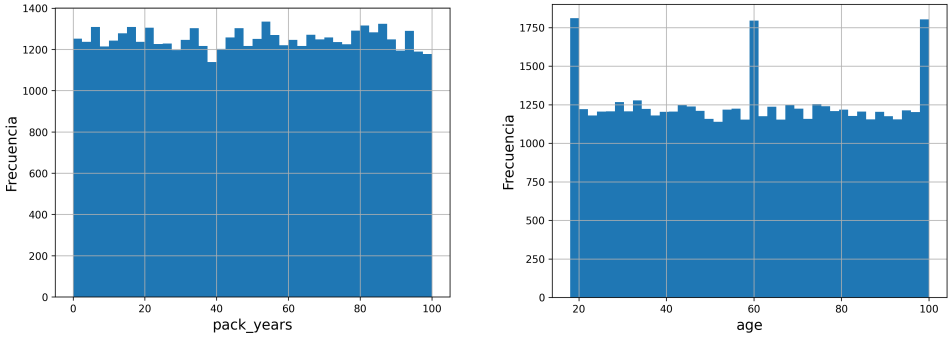
Figura 4.1: Distribución de la variable objetivo (lung_cancer).

Variables numéricas

Como podemos observar tanto en los Histogramas de la Figura 4.2, como en los estadísticos de la Tabla 4.2, las diferentes variables están uniformemente distribuidas, lo que indica que hay tanto personas jóvenes como ancianas. Así como personas con exposición prácticamente nula al tabaco hasta fumadores

muy intensos. La media y la mediana son casi iguales en ambas variables, mostrando una distribución centrada sin asimetrías fuertes.

Tanto la exposición tabáquica como la edad presentan variabilidad moderada. Aunque la mediana está en torno a 50 en packyears, hay un 25 % de personas con menos de 25 y otro 25 % con más de 75. Esto sugiere una población con perfiles de riesgo muy distintos. Respecto a la edad la mediana está en torno a los 59 años , habiendo un 25 % de personas con menos de 38 y otro 25 % de personas mayores de 80. Además ambos tienen valores de dispersión bastante altos.



(a) Distribución de la variable numérica (pack_years)

(b)Distribución de la variable numérica age

Figura 4.2: Distribuciones de las variables numéricas

Estadístico	pack_years	age
Rango	0–100	18–100
Media	49.96	58.86
Mediana	50.18	59
Q1–Q3	24.77–75.07	38–80
Std	28.85	23.96

Tabla 4.2: Estadísticos de variables numéricas

Variables categóricas

Las variables categóricas presentan distribuciones notablemente equilibradas. En particular, `gender`, `asbestos_exposure`, `secondhand_smoke_exposure`, `copd_diagnosis` y `family_history` muestran una proporción cercana al 50 % entre sus dos categorías. Del mismo modo, `radon_exposure` y `alcohol_consumption` se distribuyen de manera prácticamente uniforme entre tres niveles, con aproximadamente un 33 % de frecuencia en cada uno.

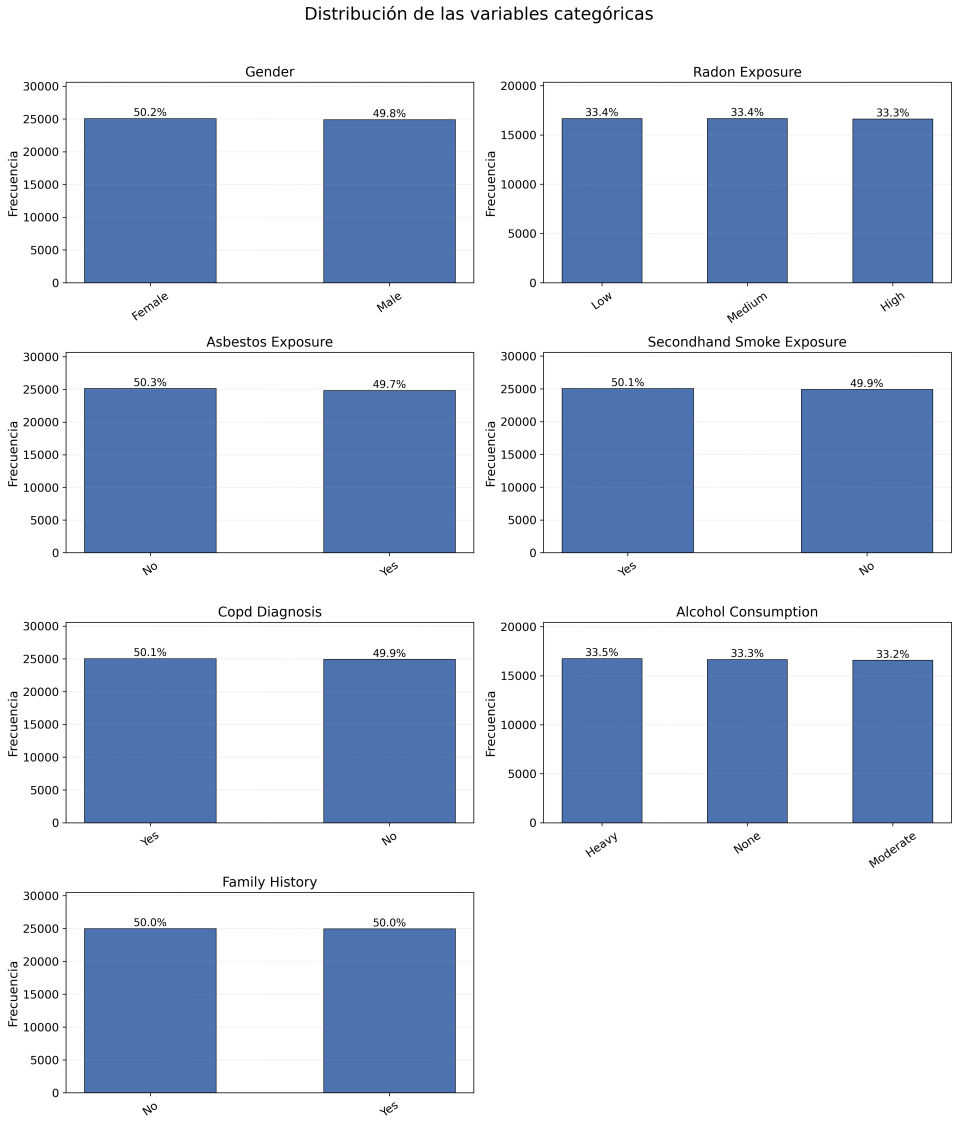


Figura 4.3: Distribución de las variables categóricas.

4.2.2. Distribuciones de las variables condicionadas a la variable etiqueta

Distribuciones de variables categóricas condicionadas a la variable objetivo

En este apartado se presentan tablas de contingencia que muestran, para cada categoría de las variables explicativas, la proporción de pacientes con

y sin diagnóstico de cáncer de pulmón. Estas tablas permiten analizar cómo varía la distribución de la variable objetivo en función de cada factor y serán utilizadas posteriormente para evaluar su relevancia.

Las variables relacionadas con exposición a factores ambientales y antecedentes médicos (Tablas 4.5 , 4.4, 4.6, 4.7, 4.8) muestran una mayor proporción de casos positivos en las categorías de riesgo, lo que indica una asociación descriptiva con la presencia de cáncer en el dataset.

Respecto a la variable alcohol_consumption (Tabla 4.9), se observa una ligera tendencia creciente en la proporción de casos positivos a medida que aumenta el nivel de consumo. Aunque la diferencia entre las categorías es moderada, aproximadamente de cuatro puntos porcentuales, por lo que su capacidad discriminativa parece limitada.

Sin embargo, la variable gender (Tabla 4.3) no muestra diferencias entre categorías: tanto en Female como en Male la proporción de casos positivos y negativos es exactamente la misma. Por ello , esta variable parece no aportar a la clasificación y predicción.

Gender	**No_rate**	**Yes_rate**
Female	0.313	0.687
Male	0.313	0.687

Tabla 4.3: Distribución de gender según la variable objetivo lung_cancer

Radon_exposure	**No_rate**	**Yes_rate**
Low	0.354	0.646
Medium	0.361	0.639
High	0.223	0.777

Tabla 4.4: Distribución de radon_exposure según la variable objetivo lung_cancer

Asbestos_exposure	**No_rate**	**Yes_rate**
No	0.385	0.615
Yes	0.239	0.761

Tabla 4.5: Distribución de asbestos_exposure según la variable objetivo lung_cancer

Copd_diagnosis	**No_rate**	**Yes_rate**
No	0.384	0.616
Yes	0.242	0.758

Tabla 4.6: Distribución de copd_diagnosis según la variable objetivo lung_cancer

Second_hand_smoke_exposure	**No_rate**	**Yes_rate**
No	0.358	0.642
Yes	0.268	0.732

Tabla 4.7: Distribución de second_hand_smoke_exposure según la variable objetivo lung_cancer

Family_history	**No_rate**	**Yes_rate**
No	0.362	0.638
Yes	0.263	0.737

Tabla 4.8: Distribución de family_history según la variable objetivo lung_cancer

Alcohol_consumption	**No_rate**	**Yes_rate**
None	0.328	0.672
Moderate	0.321	0.679
Heavy	0.289	0.711

Tabla 4.9: Distribución de alcohol_consumption según la variable objetivo lung_cancer

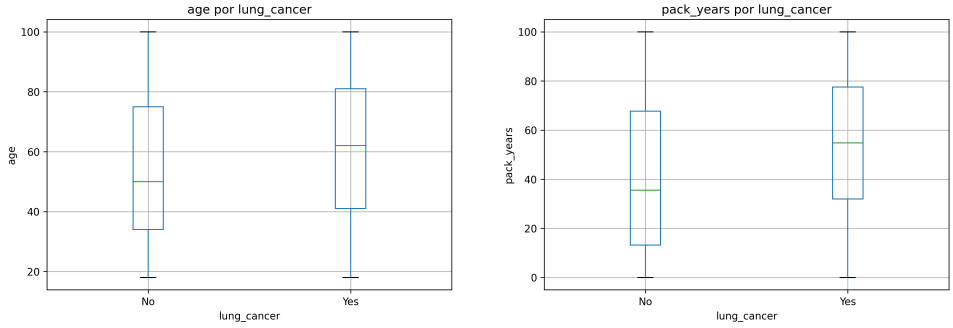
Distribución de las variables condicionadas a la variable objetivo

En este apartado veremos y analizaremos como se distribuyen las variables numéricas, *age* y *pack_years* entre los grupos que tengan cáncer y los que no.

Para ello utilizaremos boxplots que son gráficos que muestran la distribución de una variable. La caja contiene el 50 % de los datos centrales, y la linea interior señala la mediana. Los "bigotes"muestran hasta donde llegan los valores más extremos pero aún habituales.

En los diagramas de caja correspondientes a las variables age y pack_years (Figura 4.4) se observa un desplazamiento de la distribución hacia valores superiores en el grupo de pacientes con diagnóstico positivo respecto al grupo negativo. En ambos casos aunque más notable en la variable pack_years, tanto la mediana como los cuartiles se sitúan en valores más elevados para la clase positiva, lo que sugiere una asociación entre mayores niveles de edad y exposición tabáquica y la presencia de cáncer de pulmón.

Es importante decir que este analisis no implica necesariamente causalidad, sino que indica que estas variables presentan una distribución superior en el grupo diagnosticado con cáncer.



(a) Boxplot de age asociado a lung_cancer

(b) Boxplot de pack_years asociado a lung_cancer

Figura 4.4: Distribuciones de las variables numéricas condicionadas a la variable objetivo

4.3. Análisis de dependencia y asociación

En este apartado se analiza la relación existente entre las variables del conjunto de datos y la variable objetivo, con el objetivo de identificar posibles dependencias y patrones relevantes. Para ello, se emplean distintas medidas estadísticas que permiten cuantificar tanto la asociación lineal como no lineal entre variables, diferenciando entre variables numéricas y categóricas.

Análisis correlaciones variables numéricas

Para cuantificar el grado y la dirección de la asociacion entre las variables

numéricas y la variable objetivo, haremos uso de 2 medidas de correlación, la de Spearman y la de Pearson.

La correlaciónes lineales entre age y pack_years (Figuras 4.5 , 4.6) es prácticamente nula ($r \approx 0.003$), lo que indica ausencia de relación lineal y sugiere que ambas variables aportan información complementaria sin colinealidad.

Además, existe una correlación lineal positiva de la variable objetivo con pack_years ($r \approx 0.21$) y con age ($r \approx 0.12$). Esto sugiere que ambos factores pueden incrementar la probabilidad de cáncer, siendo pack_years la variable numérica con mayor asociación lineal con la etiqueta.

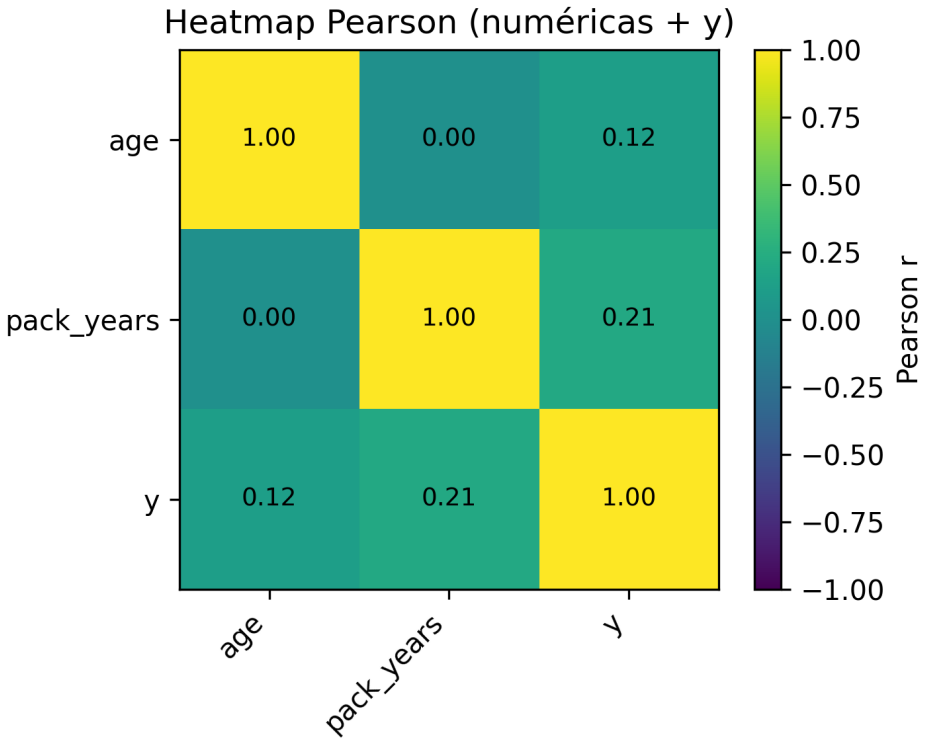


Figura 4.5: Correlación de las variables numéricas y la objetivo mediante Pearson

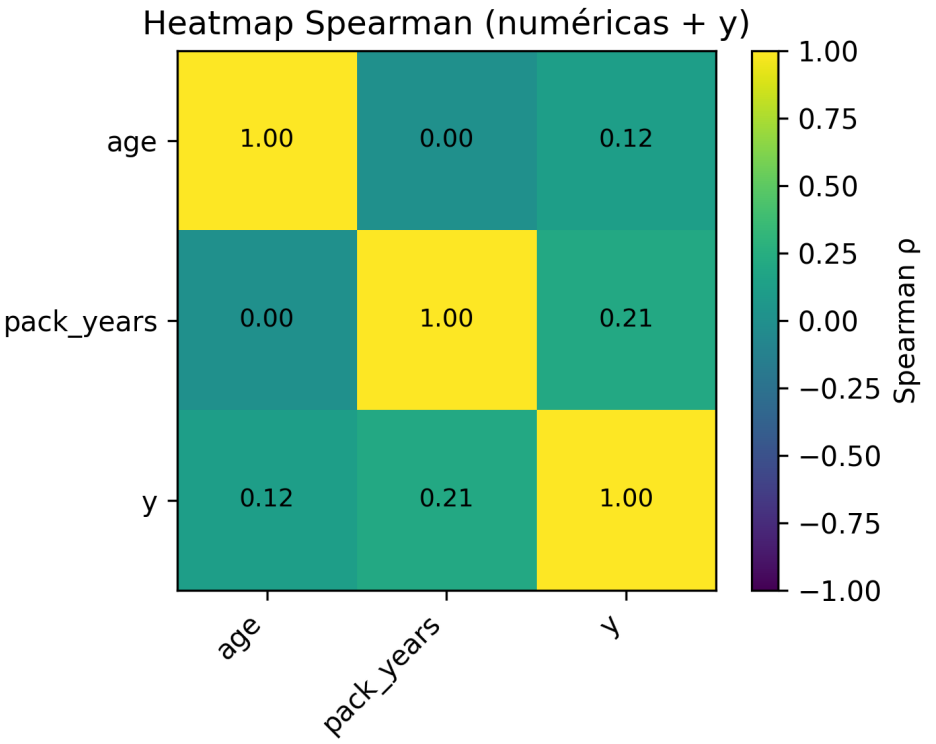


Figura 4.6: Correlación de las variables numéricas y la objetivo mediante Spearman

Análisis correlaciones variables categóricas

Para cuantificar la asociación entre las variables categóricas y la etiqueta se empleó el estadístico Cramers'V. Los resultados (Figura 4.7) muestran una asociación prácticamente nula para gender y *alcohol_consumption*. Las asociaciones más elevadas aparecen en *asbestos_exposure*, *copd_diagnosis* y *radon_exposure*, aunque en todos los casos la relación sigue siendo débil. Otras variables, como *family_history* y *secondhand_smoke_exposure*, presentan asociaciones aún más débiles. Además, los valores obtenidos entre las propias variables categóricas son muy bajos, lo que indica que son prácticamente independientes entre sí.

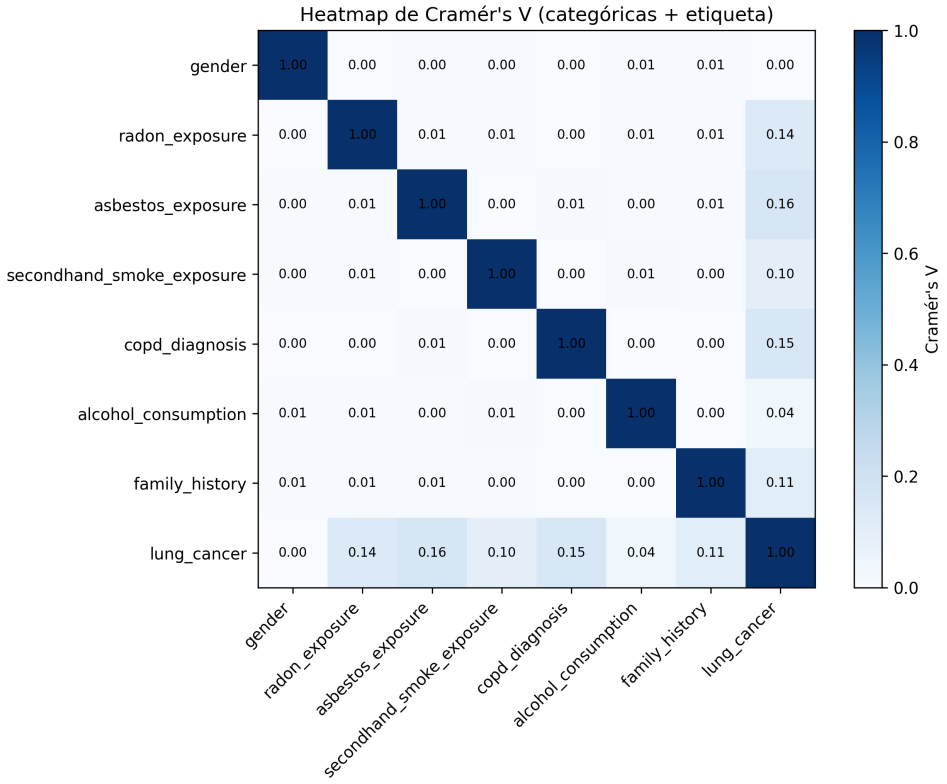


Figura 4.7: Correlación de las variables categóricas y la objetivo mediante el estadístico Cramers' V

4.3.1. Importancia basada en IM e IV de la variables con respecto a la etiqueta

En esta subsección se utilizan medidas como el Information Value (IV) y la Información Mutua (MI) para evaluar la relevancia individual de cada variable respecto a la etiqueta. Al tratarse de análisis univariantes, permiten obtener una primera idea sobre qué factores pueden tener mayor capacidad discriminativa. Sin embargo, no tienen en cuenta posibles interacciones entre variables ni permiten extraer conclusiones de tipo causal.

Información mutua es una medida que cuantifica la dependencia general entre cada variable y la variable objetivo, sin asumir relaciones lineales. Como podemos observar en la Figura 4.8, pack_years es la variable que más información comparte con lung_cancer, seguida de copd_diagnosis y asbestos_exposure, lo que sugiere que son las más relevantes de forma individual, aunque no podemos tenerlas en cuenta del todo ya que se aproximan mucho a 0 todas y su diferencia es escasa.

No obstante, todos los valores obtenidos son cercanos a 0 y las diferencias entre variables son reducidas, por lo que estos resultados deben interpretarse con cautela.

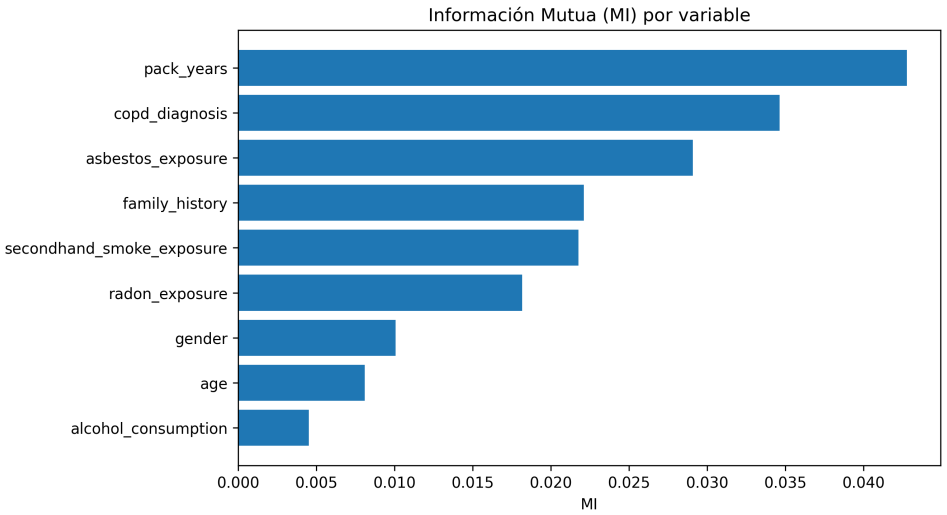


Figura 4.8: Ranking de variables según Mutual Information

El Information Value cuantifica la capacidad de una variable para separar las clases mediante Weight of Evidence. En los resultados obtenidos (Figura 4.9),*pack_years* presenta un IV alto (capacidad discriminativa fuerte), mientras que *asbestos_exposure* y *copd_diagnosis* muestran valores moderados, indicando utilidad adicional pero menor.

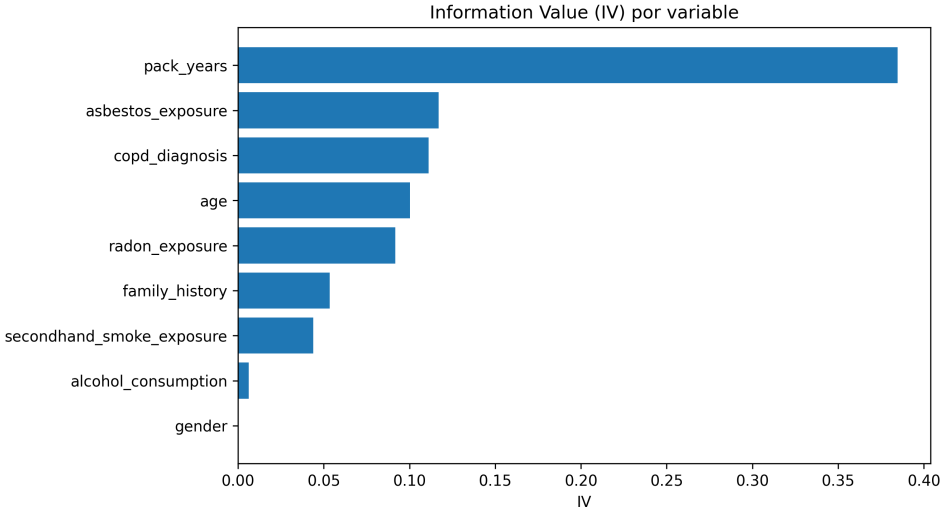


Figura 4.9: Ranking de variables según Information Value

4.4. Variables con mayor capacidad predictiva

A partir del análisis exploratorio (distribuciones , boxplots , gráficas) y las medidas de asociación y relevancia, se puede establecer un tier list razonado de las variables más informativas para predecir la presencia del cáncer de pulmón.

Variables más relevantes

pack_years — Muy relevante

- El boxplot muestra un desplazamiento claro hacia valores mayores en la clase positiva, con mediana e IQR superiores. Aunque existe solapamiento, la separación es la más marcada entre las numéricas.
- Correlación positiva mayor que el resto de numéricas (Pearson ≈ 0.21).
- $IV \approx 0.3846$: capacidad discriminativa fuerte.
- $MI \approx 0.0428$: variable con mayor dependencia respecto a la etiqueta.

copd_diagnosis — Relevante

- Yes_rate ≈ 0.758 en pacientes con COPD, mostrando asociación notoria.
- $IV \approx 0.1111$: capacidad discriminativa moderada.
- $MI \approx 0.0346$: segunda en el ranking de MI.
- Cramér's V ≈ 0.15: asociación débil pero de las más altas.

asbestos_exposure — Relevante

- Yes_rate ≈ 0.761, diferencia notable.
- $IV \approx 0.1169$: separación moderada.
- $MI \approx 0.0291$: tercera más informativa según MI.
- Cramér's V ≈ 0.16: mayor asociación categórica, aunque débil.

Variables informativas complementarias

age — Contribución moderada

- Boxplot desplazado hacia edades mayores en positivos, aunque con un amplio solapamiento.
- Pearson ≈ 0.12: correlación positiva débil.
- $IV \approx 0.1003$: aporta algo de separación, pero no marcada.
- $MI \approx 0.0081$: dependencia global baja.

radon_exposure — Contribución moderada

- La categoría High presenta una mayor proporción de casos ($\approx$ 0.777), lo que sugiere cierta asociación.
- $IV \approx 0.0917$: capacidad moderado-baja.
- $MI \approx 0.0182$: dependencia limitada.
- Cramér's V $\approx$ 0.14: asociación débil pero relevante dentro de categóricas.

Variables con contribución baja

family_history — Baja

- $IV \approx 0.0535$: separación limitada.
- $MI \approx 0.0221$: dependencia pequeña.
- Cramér's V ≈ 0.11: asociación débil.

secondhand_smoke_exposure — Baja

- $IV \approx 0.0439$: separación limitada.
- $MI \approx 0.0218$: dependencia pequeña.
- Cramér's V ≈ 0.10: asociación débil.

Variables con utilidad predictiva prácticamente nula

alcohol_consumption — Muy baja

- $IV \approx 0.0063$ y $MI \approx 0.0045$: contribución prácticamente nula.
- Tablas de contingencia similares entre categorías.
- Cramér's V ≈ 0.04: dependencia casi inexistente.

gender — Nula

- $IV = 0$: no separa las clases.
- $MI \approx 0.0101$: dependencia mínima.
- Cramér's V ≈ 0.00: asociación inexistente.

Capítulo 5

Estudio de viabilidad para mitigar el desbalanceo de clases

En este capítulo se abordará el desbalanceo de clases mediante distintas técnicas de balanceo, con el objetivo de analizar si es posible obtener conjuntos de entrenamiento que mejoren el rendimiento respecto al conjunto original para después aplicarlo al preprocesamiento. Para ello, se evaluarán cuatro modelos utilizando diferentes conjuntos de entrenamiento con distintas distribuciones de etiquetas.

Para las técnicas de oversampling se emplearon RandomOverSampler (ROS) y SMOTENC. RandomOverSampler incrementa el número de instancias de la clase minoritaria replicando muestras existentes de forma aleatoria, mientras que SMOTENC genera nuevas instancias sintéticas.

Respecto a las técnicas de undersampling, se utilizaron RandomUnderSampler (RUS) y NeighbourhoodCleaningRule (NCR). RandomUnderSampler reduce el número de instancias de la clase mayoritaria mediante una eliminación aleatoria de ejemplos. Por su parte, NCR elimina instancias conflictivas o ruidosas basándose en el análisis de vecinos cercanos.

Respecto a las distribuciones de las clases, las diferentes técnicas de balanceo se aplicaron de manera progresiva. Para cada técnica utilizada se generaron distintas versiones del conjunto de entrenamiento, modificando de manera incremental el porcentaje de instancias pertenecientes a la clase minoritaria. De esta forma, fue posible analizar cómo evolucionaba el rendimiento de los modelos conforme variaba el grado de balanceo aplicado.

Para los análisis se han utilizado cuatro modelos pertenecientes a distintas familias: Logistic Regression (LR), Random Forest (RF), Support Vector Machine (SVM) y XGBoost. Para garantizar una comparación homogénea, los modelos se han evaluado con sus configuraciones base, sin aplicar un proceso adicional de selección de hiperparámetros.

5.1. Oversampling

Las Tablas 5.1 y 5.3, además de la Figura 5.1 recogen, respectivamente, la distribución de clases de los distintos conjuntos de entrenamiento generados mediante técnicas de oversampling y los resultados obtenidos por los diferentes modelos evaluados sobre dichos conjuntos.

De forma general, a medida que aumenta el balanceo de clases tanto con RandomOverSampler como con SMOTENC, la accuracy de los modelos tiende a disminuir progresivamente. Esto ocurre porque los modelos dejan de favorecer tanto a la clase mayoritaria original y pasan a realizar predicciones más equilibradas entre ambas clases.

Respecto a la precisión, conforme aumenta la representación de la clase minoritaria en el conjunto de entrenamiento, se observa una reducción en la precisión de la clase negativa y un aumento en la precisión de la clase positiva. Esto indica que los modelos tienden a predecir con mayor frecuencia la clase negativa, lo que mejora parte de su detección, pero reduce la fiabilidad de dicha clase.

En cuanto al recall y F1, se aprecia que aumentan en la clase negativa y disminuyen en la clase positiva. Por otro lado, la ROC-AUC se mantiene relativamente estable en los distintos conjuntos de entrenamiento evaluados, lo que indica que la capacidad global de separación entre clases apenas varía.

Respecto a las matrices de confusión, se observa un aumento de verdaderos negativos y una reducción de falsos positivos conforme aumenta el balanceo del conjunto de entrenamiento. Sin embargo, también aumentan los falsos negativos y disminuyen los verdaderos positivos. Esto confirma que los modelos se vuelven más sensibles a la clase minoritaria, aunque a costa de perder parte de su capacidad para identificar correctamente la clase mayoritaria.

La única excepción destacable se encuentra en el modelo SVM utilizando ROS al 35 % o SMOTENC al 35 %. En este caso, sí se observa una mejora respecto al conjunto original: la accuracy aumenta ligeramente, alcanzando su máximo con SMOTENC_35, y la F1 de la clase minoritaria mejora sin provocar una caída excesiva en la F1 de la clase mayoritaria. Además, este comportamiento se ve respaldado por una mejora en la ROC-AUC.

A pesar de esta excepción puntual, la decisión general es mantener el conjunto de entrenamiento original. Los resultados muestran que las técnicas de oversampling evaluadas no producen una mejora global y consistente en la capacidad predictiva de los modelos. En la mayoría de los casos, el balanceo desplaza el error de una clase a otra, mejorando la detección de la clase minoritaria, pero penalizando el rendimiento sobre la clase mayoritaria. Además, se observa una reducción de los verdaderos positivos y un incremento de falsos negativos, aspecto especialmente crítico en este problema, ya que al trabajar con un contexto relacionado con la salud de los pacientes, minimizar este tipo de error resulta prioritario.

Tabla 5.1: Distribución de clases en los conjuntos de entrenamiento generados mediante técnicas de oversampling

Dataset	**Lung Cancer Yes**	**Lung Cancer No**
train	24055 (0.6872)	10945 (0.3127)
ROS_35	24055 (0.65)	12952 (0.35)
ROS_40	24055 (0.60)	16036 (0.40)
ROS_45	24055 (0.55)	19681 (0.45)
smotenc_35	24055 (0.65)	12952 (0.35)
smotenc_40	24055 (0.60)	16036 (0.40)
smotenc_45	24055 (0.55)	19681 (0.45)

Tabla 5.2: Distribución de clases en los conjuntos de entrenamiento generados mediante técnicas de oversampling. Las notaciones ROS_xx y smotenc_xx indican el porcentaje aproximado de instancias pertenecientes a la clase minoritaria tras aplicar la técnica de balanceo.

Tabla 5.3: Resultados obtenidos por los distintos modelos utilizando técnicas de oversampling, los números rojos y verdes indican el peor y mejor resultado, respectivamente para cada métrica.

Dataset	Modelo	Accuracy	Precision (0/1)	Recall (0/1)	F1 (0/1)	ROC-AUC
train	LR	0.7228	0.6046 / 0.7470	0.3285 / 0.9022	0.4257 / 0.8173	0.7381
ROS_35	LR	0.7171	0.5687 / 0.7583	0.3952 / 0.8636	0.4664 / 0.8076	0.7382
ROS_40	LR	0.7080	0.5360 / 0.7774	0.4931 / 0.8058	0.5137 / 0.7914	0.7381
ROS_45	LR	0.6895	0.5031 / 0.7970	0.5884 / 0.7356	0.5424 / 0.7651	0.7381
smotenc_35	LR	0.7179	0.5706 / 0.7588	0.3963 / 0.8643	0.4677 / 0.8081	0.7382
smotenc_40	LR	0.7079	0.5358 / 0.7740	0.4929 / 0.8057	0.5134 / 0.7913	0.7382
smotenc_45	LR	0.6893	0.5028 / 0.7977	0.5907 / **0.7342**	0.5432 / 0.7646	0.7383
train	RF	0.6987	0.5226 / 0.7584	0.4232 / 0.8241	0.4677 / 0.7899	0.7140
ROS_35	RF	0.6933	0.5112 / 0.7612	0.4438 / 0.8069	0.4751 / 0.7834	0.7128
ROS_40	RF	0.6924	0.5090 / 0.7661	0.4664 / 0.7952	0.4868 / 0.7804	0.7108
ROS_45	RF	0.6861	0.4980 / 0.7692	0.4884 / 0.7760	0.4932 / 0.7726	0.7087
smotenc_35	RF	0.6919	0.5082 / 0.7631	0.4541 / 0.8001	0.4796 / 0.7811	0.7117
smotenc_40	RF	0.6854	0.4970 / 0.7700	0.4926 / 0.7731	0.4948 / 0.7716	0.7085
smotenc_45	RF	**0.6736**	**0.4801** / 0.7751	0.5282 / 0.7397	0.5030 / 0.7570	**0.7080**
train	SVM	0.7315	**0.6718** / **0.7404**	**0.2767** / **0.9385**	**0.3920** / **0.8277**	0.7241
ROS_35	SVM	0.7317	0.6271 / 0.7539	0.3507 / 0.9051	0.4498 / 0.8226	0.7313
ROS_40	SVM	0.7271	0.5766 / 0.7798	0.4788 / 0.8400	0.5232 / 0.8088	0.7415
ROS_45	SVM	0.7011	0.5187 / 0.8078	0.6124 / 0.7414	0.5617 / 0.7732	0.7478
smotenc_35	SVM	**0.7335**	0.6333 / 0.7545	0.3509 / 0.9076	0.4516 / 0.8240	0.7313
smotenc_40	SVM	0.7259	0.5725 / 0.7818	0.4884 / 0.8340	0.5271 / 0.8071	0.7416
smotenc_45	SVM	0.7020	0.5199 / **0.8093**	**0.6163** / 0.7410	**0.5640** / **0.7360**	0.7472
train	XGBoost	0.7291	0.5998 / 0.7633	0.4016 / 0.8781	0.4811 / 0.8167	**0.7539**
ROS_35	XGBoost	0.7253	0.5775 / 0.7732	0.4526 / 0.8494	0.5075 / 0.8095	0.7530
ROS_40	XGBoost	0.7130	0.5436 / 0.7840	0.5131 / 0.8040	0.5279 / 0.7938	0.7507
ROS_45	XGBoost	0.7021	0.5210 / 0.8004	0.5864 / 0.7547	0.5518 / 0.7769	0.7527
smotenc_35	XGBoost	0.7253	0.5764 / 0.7747	0.4592 / 0.8464	0.5112 / 0.8090	0.7538
smotenc_40	XGBoost	0.7135	0.5422 / 0.7909	0.5395 / 0.7927	0.5409 / 0.7918	0.7515
smotenc_45	XGBoost	0.6981	0.5147 / 0.8058	0.6086 / 0.7389	0.5577 / 0.7709	0.7528



Figura 5.1: Matrices de confusión obtenidas por los modelos con técnicas de *oversampling*. Nota: Cada matriz sigue el formato $\begin{bmatrix} VN & FP \\ FN & VP \end{bmatrix}$.

5.2. Undersampling

Las Tablas 5.4 y 5.6, además de la Figura 5.2 recogen, respectivamente, la distribución de clases de los distintos conjuntos de entrenamiento generados mediante técnicas de undersampling y los resultados obtenidos por los diferentes modelos evaluados sobre dichos conjuntos.

En el caso de Neighborhood Cleaning Rule (NCR), se han seleccionado los valores (k = 1), (k = 3) y (k = 5), ya que para valores superiores el rendi-

miento obtenido fue considerablemente peor. Además, se utilizaron valores impares con el objetivo de evitar posibles empates durante el proceso de selección de vecinos.

El parámetro threshold_cleaning actúa como un umbral que determina si debe aplicarse una segunda fase de limpieza adicional sobre clases suficientemente mayoritarias, una vez realizada la limpieza principal basada en vecinos.

En nuestro caso, no tuvo un efecto apreciable sobre el resultado final, ya que la primera fase de limpieza basada en vecinos eliminó la mayoría de instancias ruidosas o conflictivas. Por ello, al reducir este umbral para hacer un segunda limpieza de instancias ruidosas no se observaron cambios relevantes en el número de instancias eliminadas.

Respecto a la accuracy, se observa una disminución progresiva conforme se reduce el número de instancias de la clase positiva en los conjuntos generados mediante undersampling. En cuanto a la precisión, a medida que disminuye la proporción de la clase positiva, aumenta la precisión de dicha clase, mientras que se reduce la precisión de la clase negativa. Este comportamiento es similar al observado con oversampling.

De forma similar, al aumentar la capacidad del modelo para identificar la clase negativa, mejora el recall y F1 de dicha clase, pero empeoran el de la clase positiva. Respecto a la métrica ROC-AUC, los valores se mantienen relativamente estables, siendo el modelo SVM la única excepción destacable, donde sí se aprecia una mejora más consistente.

Este comportamiento también puede apreciarse en las matrices de confusión. Conforme aumenta el undersampling, aumentan los verdaderos negativos y disminuyen los falsos positivos, indicando que los pacientes sin cáncer son identificados de manera más eficaz. Sin embargo, también disminuyen los verdaderos positivos y aumentan los falsos negativos, empeorando la detección de pacientes con cáncer.

En general, todos los conjuntos de entrenamiento evaluados presentan un rendimiento inferior al conjunto original, salvo el caso de rus_35 aplicado sobre SVM, donde se observa una ligera mejora en accuracy y ROC-AUC, manteniendo además valores relativamente equilibrados en precisión, recall y F1.

Por tanto, se mantiene la misma conclusión obtenida en el análisis de oversampling: se conserva el conjunto de entrenamiento original, ya que las técnicas de undersampling no producen una mejora consistente en el rendimiento global. Además, la mejora en la identificación de una clase suele ir acompañada de un empeoramiento en la otra. De la misma manera, se observa un

aumento de los falsos negativos y una reducción de los verdaderos positivos, lo que resulta especialmente desfavorable en este problema.

Tabla 5.4: Distribución de clases en los conjuntos de entrenamiento generados mediante técnicas de undersampling

Dataset	Lung Cancer Yes	Lung Cancer No
train	24055 (0.6872)	10945 (0.3127)
rus_35	20326 (0.65)	10945 (0.35)
rus_40	16417 (0.60)	10945 (0.40)
rus_45	13377 (0.55)	10945 (0.45)
ncr_thr95_k1	17990 (0.62)	10945 (0.38)
ncr_thr95_k3	13058 (0.54)	10945 (0.46)
ncr_thr95_k5	9473 (0.46)	10945 (0.54)

Tabla 5.5: Distribución de clases en los conjuntos de entrenamiento generados mediante técnicas de undersampling. La notación rus_xx indica el porcentaje de instancias pertenecientes a la clase minoritaria tras aplicar la técnica. Para NCR se utiliza la nomenclatura ncr_thrXX_kY, donde XX representa el valor de *threshold_cleaning* e Y el número de vecinos considerados.

Tabla 5.6: Resultados obtenidos por los distintos modelos utilizando técnicas de undersampling, los números rojos y verdes indican el peor y mejor resultado, respectivamente para cada métrica.

Dataset	Modelo	Accuracy	Precision (0/1)	Recall (0/1)	F1 (0/1)	ROC-AUC
train	LR	0.7228	0.6046 / 0.7470	0.3285 / 0.9022	0.4257 / 0.8173	0.7381
rus_35	LR	0.7182	0.5714 / 0.7588	0.3959 / 0.8649	0.4677 / 0.8084	0.7382
rus_40	LR	0.7081	0.5362 / 0.7776	0.4937 / 0.8057	0.5141 / 0.7914	0.7382
rus_45	LR	0.6887	0.5020 / 0.7971	0.5894 / 0.7339	0.5422 / 0.7642	0.7382
ncr_thr95_k1	LR	0.7016	0.5218 / 0.7895	0.5479 / 0.7716	0.5345 / 0.7804	0.7380
ncr_thr95_k3	LR	0.6567	0.4675 / 0.8243	0.7020 / 0.6363	0.5612 / 0.7181	0.7376
ncr_thr95_k5	LR	0.6083	0.4326 / 0.8572	0.8111 / 0.5160	0.5643 / 0.6442	0.7370
train	RF	0.6987	0.5226 / 0.7584	0.4232 / 0.8241	0.4677 / 0.7899	**0.7140**
rus_35	RF	0.6882	0.5016 / 0.7684	0.4820 / 0.7820	0.4916 / 0.7752	0.7152
rus_40	RF	0.6763	0.4845 / 0.7820	0.5508 / 0.7333	0.5156 / 0.7569	0.7145
rus_45	RF	0.6623	0.4693 / 0.7950	0.6116 / 0.6853	0.5311 / 0.7361	0.7145
ncr_thr95_k1	RF	0.6650	0.4728 / 0.7984	0.6195 / 0.6857	0.5363 / 0.7378	0.7227
ncr_thr95_k3	RF	0.6266	0.4422 / 0.8300	0.7421 / 0.5741	0.5542 / 0.6788	0.7299
ncr_thr95_k5	RF	**0.5750**	**0.4118** / 0.8607	0.8380 / **0.4553**	0.5522 / **0.5956**	0.7287
train	SVM	0.7315	**0.6718** / **0.7404**	**0.2767** / **0.9385**	**0.3920** / **0.8277**	0.7241
rus_35	SVM	**0.7339**	0.6337 / 0.7550	0.3532 / 0.9071	0.4536 / 0.8241	0.7312
rus_40	SVM	0.7257	0.5742 / 0.7788	0.4760 / 0.8394	0.5205 / 0.8079	0.7421
rus_45	SVM	0.7009	0.5183 / 0.8086	0.6152 / 0.7398	0.5626 / 0.7727	0.7479
ncr_thr95_k1	SVM	0.7235	0.5599 / 0.7944	0.5412 / 0.8064	0.5504 / 0.8003	0.7448
ncr_thr95_k3	SVM	0.6684	0.4801 / 0.8374	0.7259 / 0.6423	0.5779 / 0.7269	0.7523
ncr_thr95_k5	SVM	0.5943	0.4270 / **0.8878**	**0.8698** / 0.4689	0.5728 / 0.6137	0.7477
train	XGBoost	0.7291	0.5998 / 0.7633	0.4016 / 0.8781	0.4811 / 0.8167	0.7539
rus_35	XGBoost	0.7249	0.5743 / 0.7758	0.4645 / 0.8433	0.5136 / 0.8082	0.7524
rus_40	XGBoost	0.7096	0.5354 / 0.7897	0.5395 / 0.7870	0.5375 / 0.7884	0.7497
rus_45	XGBoost	0.6942	0.5091 / 0.8090	0.6231 / 0.7265	0.5603 / 0.7656	0.7509
ncr_thr95_k1	XGBoost	0.7071	0.5285 / 0.8028	0.5892 / 0.7608	0.5572 / 0.7812	0.7583
ncr_thr95_k3	XGBoost	0.6668	0.4788 / 0.8430	0.7408 / 0.6331	**0.5817** / 0.7231	**0.7590**
ncr_thr95_k5	XGBoost	0.6117	0.4369 / 0.8732	0.8376 / 0.5089	0.5743 / 0.6430	0.7560



Figura 5.2: Matrices de confusión obtenidas por los modelos utilizando técnicas de undersampling. Nota: Cada matriz sigue el formato $\begin{bmatrix} VN & FP \\ FN & VP \end{bmatrix}$.

Capítulo 6

Marco experimental

6.1. Preprocesamiento

El preprocesamiento de los datos se ha centralizado en la clase DataReader, que permite cargar el conjunto de datos y aplicar de forma configurable las transformaciones necesarias antes del entrenamiento de los modelos. Desde el script de cada modelo se indican parámetros como la ruta del archivo CSV, la columna objetivo, las columnas a eliminar, el tipo de codificación de variables categóricas, si se desea aplicar escalado y la semilla de aleatoriedad.

En primer lugar, se carga el conjunto de datos mediante pandas. Para evitar que la categoría None de la variable `alcohol_consumption` sea interpretada erróneamente como valor ausente, se utiliza `keep_default_na=False`. De esta forma, dicha categoría se conserva como un valor válido y no se pierde información durante la lectura del CSV.

Además, durante el análisis exploratorio se comprobó que no existen registros duplicados exactos. La variable `patient_id` es única para cada registro (50.000/50.000), por lo que se considera una variable identificativa y se elimina del conjunto de características. También se verificó que las variables categóricas presentan dominios controlados y coherentes, sin variantes ortográficas ni categorías inesperadas, y que las variables numéricas, como `age` y `pack_years`, se encuentran dentro de rangos válidos, sin evidencias de valores fuera de dominio ni outliers evidentes. Por lo que no se requiere ningún tipo de preprocesamiento en estos casos.

Posteriormente, se separa la variable objetivo `lung_cancer` del resto de variables predictoras. Como la variable objetivo está representada como texto, se transforma a valores numéricos para que pueda ser utilizada por los modelos.

A continuación, se identifican automáticamente las variables numéricas y categóricas. Aunque el módulo permite aplicar imputación mediante SimpleImputer, este paso no modifica los datos en este caso, ya que en el análisis exploratorio se comprobó que no existen valores faltantes. Sin embargo, se mantiene esta funcionalidad para garantizar la robustez del proceso ante posibles cambios futuros en el conjunto de datos.

Después, las variables categóricas se codifican según la estrategia definida. Por defecto se utiliza one-hot encoding, que transforma cada categoría en columnas binarias. Finalmente, si el modelo lo requiere, se aplica StandardScaler para escalar las variables numéricas.

6.2. Modelos empleados

En este trabajo se han empleado varios modelos con el objetivo de comparar su comportamiento ante el problema planteado y contrastar los resultados obtenidos desde distintas perspectivas metodológicas.

Se han seleccionado modelos pertenecientes a distintas familias con el objetivo de analizar el problema desde enfoques diferentes. Cada tipo de modelo aprende los patrones de los datos de una forma distinta: algunos se basan en relaciones lineales, otros en árboles de decisión y otros en combinaciones de varios clasificadores. Por ello, la evaluación experimental resulta más completa y ofrece una visión más fiable del comportamiento de los modelos ante el problema planteado. En la tabla 6.1 se muestran los modelos empleados junto a su justificación:

Tabla 6.1: Resumen de los modelos considerados y su justificación

Familia	**Modelo**	**Justificación breve**
Modelos lineales	Regresión logística	Modelo lineal sencillo e interpretable, adecuado para establecer una referencia clara en la clasificación. Además, presenta un bajo coste computacional y una implementación directa.
Modelos basados en distancias	K-Nearest Neighbors	Método sencillo, intuitivo y fácil de implementar. Requiere pocos hiperparámetros y resulta adecuado cuando el número de variables es moderado.

Continúa en la página siguiente

Familia	Modelo	Justificación breve
Modelos basados en árboles	Árbol de decisión	Modelo versátil e interpretable, capaz de adaptarse a relaciones no lineales. Además, requiere poco preprocesamiento y es la base algoritmos más avanzados.
Máquinas de vectores de soporte	SVM	Modelo potente y versátil, con buena capacidad de generalización. Adecuado en problemas de clasificación y en conjuntos de datos de tamaño pequeño o mediano.
Modelos basados en árboles	Bagging	Permite reducir la inestabilidad de los árboles mediante la agregación de múltiples modelos. Destaca por mejorar la robustez y disminuir la varianza.
Modelos basados en árboles	Random Forest	Reduce el sobreajuste y mejora la generalización mediante la combinación de múltiples árboles con aleatoriedad. También permite estimar la importancia relativa de las variables.
Modelos basados en árboles	Extra Trees	Ofrece robustez y eficiencia computacional gracias a su mayor grado de aleatoriedad. Permite generar árboles de forma rápida y estable.
Boosting basados en árboles	AdaBoost	Destaca por su capacidad para reducir rápidamente el error de entrenamiento. Además, suele mostrar un buen comportamiento de generalización en numerosos problemas.
Boosting basados en árboles	HistGradientBoost	Implementación eficiente de gradient boosting basada en histogramas. Destaca por su rapidez y por el manejo eficiente de variables categóricas.
Boosting basados en árboles	XGBoost	Modelo optimizado para lograr alta eficiencia en el entrenamiento. Incorpora paralelización y técnicas que aceleran significativamente el tiempo de ejecución.
Boosting basados en árboles	CatBoost	Destaca por obtener buenos resultados sin requerir un ajuste exhaustivo de hiperparámetros. Además, mantiene un buen equilibrio entre precisión y eficiencia computacional.

Continúa en la página siguiente

Familia	Modelo	Justificación breve
Combinación de modelos	Stacking	Permite combinar varios modelos y aprovechar sus fortalezas complementarias. Destaca por su robustez y su potencial para mejorar el resultado final.
Combinación de modelos	Voting	Método simple de combinación de modelos que mejora la robustez. Resulta útil por su sencillez y facilidad de aplicación.
Deep Learning	Redes neuronales	Destacan por su capacidad para modelar relaciones complejas. Además, ofrecen versatilidad y escalabilidad en problemas de alta complejidad.

6.3. Métricas de evaluación

En este caso, al trabajar con un conjunto de datos desbalanceado, no resulta adecuado basarse únicamente en la métrica accuracy, ya que un modelo podría obtener un porcentaje elevado de aciertos prediciendo mayoritariamente la clase dominante. Por ejemplo, si más del 65 % de las instancias pertenecen a una misma clase, un modelo que predijera siempre dicha clase alcanzaría una accuracy superior al 65 %, aunque su capacidad de clasificación real fuese deficiente. Por ello, esta métrica puede resultar engañosa si se analiza de forma aislada. Sin embargo, se mantiene en la evaluación final como métrica complementaria, ya que permite conocer el porcentaje global de aciertos del modelo y facilita una primera comparación general entre los distintos clasificadores.

Para la búsqueda de los mejores hiperparámetros se utilizará la métrica F1-weighted, que combina precisión y recall mediante su media armónica, ponderando además el resultado de cada clase según su frecuencia en el conjunto de datos. Esta métrica permite obtener una evaluación más equilibrada del rendimiento global del modelo, penalizando tanto los falsos positivos como los falsos negativos. Además, al existir una mayor proporción de pacientes con cáncer en el conjunto de datos, esta clase tiene un mayor peso en la métrica final, lo que resulta coherente con el objetivo del problema.

Durante el análisis final sobre el conjunto de prueba, los modelos se evaluarán utilizando distintas métricas: accuracy, precision-weighted(media ponderada de la precisión de cada clase según su frecuencia), F1-weighted, ROC-AUC y la matriz de confusión. Esta última permitirá analizar de forma más detallada el tipo de errores cometidos por cada modelo, prestando especial atención a los falsos negativos, ya que en este contexto suponen clasificar como sano a un paciente que realmente presenta cáncer. Por tanto, se buscará un equilibrio entre maximizar el rendimiento global del modelo y reducir en la medida de lo posible este tipo de error, evitando al mismo tiempo clasificar de forma excesiva a todos los pacientes como positivos.

6.4. Planteamiento del proyecto

Para resolver el problema planteado se desarrolló un proyecto organizado en cuatro directorios principales, cuya estructura general se muestra en la Figura 6.1:

- **src:** contiene el código fuente principal del proyecto. Dentro de este directorio se encuentra la clase DataReader encapsulada en el fichero datareader.py, encargada de leer y preprocesar el conjunto de datos, permitiendo configurar aspectos como el escalado, la reproducibilidad, la división entre entrenamiento y test o la técnica de codificación de variables categóricas. También se incluye la clase SequentialGridSearch, encapsulada en grid_search.py, responsable de realizar la búsqueda secuencial de hiperparámetros.

  Esta carpeta incluye el subdirectorio models, donde se encuentran los distintos modelos empleados para resolver el problema. Cada uno de ellos define su propia rejilla de hiperparámetros, carga los datos mediante DataReader, inicializa la ejecución en W&B, crea el modelo correspondiente, realiza el proceso de entrenamiento y finalmente evalúa su rendimiento.

- **results:** almacena los resultados generados tras la ejecución de cada modelo. Para cada uno de ellos se guardan cuatro archivos CSV: un fichero con los mejores hiperparámetros encontrados, otro con los resultados de cada iteración del proceso de búsqueda, un tercero con el desglose de métricas para cada fold de la validación cruzada y, por último, un resumen con las medias y desviaciones estándar obtenidas.

- **wandb:** contiene los registros asociados a las ejecuciones realizadas mediante Weights & Biases. Además, almacena las gráficas y métricas generadas durante la optimización de los modelos.

- **data:** contiene el conjunto de datos que utilizarán los modelos para el entrenamiento y test.

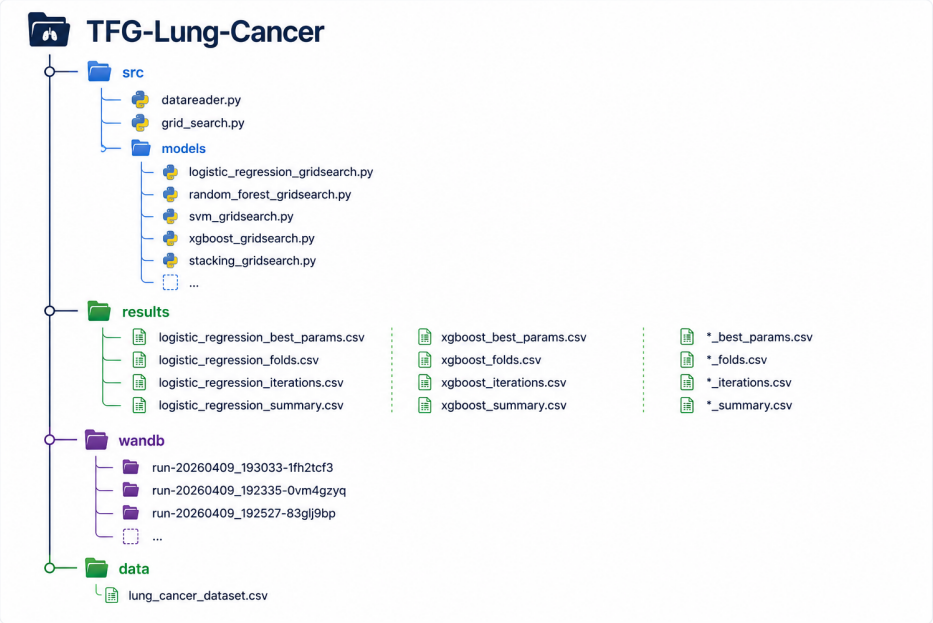


Figura 6.1: Estructura del proyecto: muestra como se disponen las diferentes carpetas y archivos que forman nuestro proyecto

Además, el Algoritmo 1 y la Figura 6.2 resumen el flujo general seguido para el entrenamiento y evaluación de cada modelo. El pseudocódigo recoge los pasos principales de la ejecución, mientras que el diagrama muestra visualmente la metodología aplicada durante el proceso experimental.

Algorithm 1 Flujo general de ejecución de los modelos

1: Definir la rejilla de hiperparámetros del modelo.
2: Leer los argumentos de entrada: dataset, codificación, escalado, semilla y uso de W&B.
3: Crear una instancia de `DataReader`.
4: Cargar y preprocesar el conjunto de datos.
5: Dividir los datos en entrenamiento y test.
6: Crear el modelo correspondiente.
7: Crear una instancia de `SequentialGridSearch`.
8: Ejecutar la búsqueda secuencial de hiperparámetros sobre el conjunto de entrenamiento mediante Validación Cruzada.
9: Obtener el mejor modelo y sus mejores hiperparámetros.
10: Evaluar el mejor modelo sobre el conjunto de test.
11: Calcular las métricas finales: accuracy, precision, recall, F1, ROC-AUC y matriz de confusión.
12: Registrar las métricas obtenidas.
13: Guardar los resultados generados en la carpeta `results`.

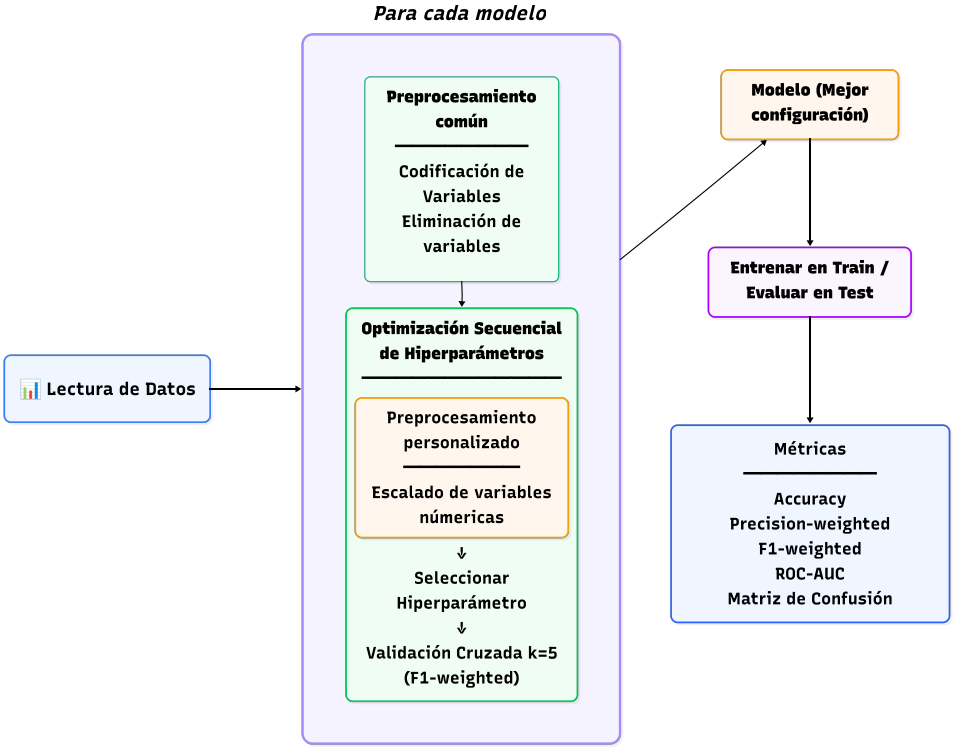


Figura 6.2: Diagrama general de la metodología seguida para el entrenamiento, optimización y evaluación de los modelos.

En primer lugar, se selecciona el conjunto de datos de entrada y se configuran las distintas opciones de preprocesamiento que pueden aplicarse sobre él. A continuación, se realiza la codificación de las variables categóricas, se eliminan aquellas variables que no aportan información relevante para la predicción y, en aquellos modelos que lo requieren, se lleva a cabo el escalado de las variables numéricas.

Posteriormente, se inicia el proceso de optimización secuencial de hiperparámetros. Para cada hiperparámetro se evalúan los distintos valores definidos en su correspondiente rejilla mediante validación cruzada estratificada de 5 particiones, utilizando la métrica F1-weighted como criterio de selección. Una vez identificado el valor que proporciona el mejor resultado, este se incorpora a la configuración del modelo y el proceso continúa con el siguiente hiperparámetro.

Tras completar la optimización, se obtiene la mejor configuración del modelo. Esta configuración se entrena utilizando el conjunto de entrenamiento y posteriormente se evalúa sobre el conjunto de prueba.

Finalmente, sobre el conjunto de prueba se calculan las distintas métricas de evaluación consideradas en este trabajo, entre las que se encuentran Accuracy, Precision-weighted, F1-weighted, ROC-AUC y la matriz de confusión.

Capítulo 7

Experimentación y análisis de resultados

En este capítulo se optimizarán los hiperparámetros de cada modelo de manera individual mediante un proceso de validación cruzada aplicado sobre el conjunto de entrenamiento. Para ello, se empleará un procedimiento de ajuste secuencial, en el que los hiperparámetros se analizarán uno a uno, manteniendo fijados los mejores valores obtenidos en los pasos anteriores.

Para cada hiperparámetro analizado se generarán dos tipos de gráficas. En primer lugar, se representará una gráfica comparativa entre la F1 obtenida en entrenamiento y la F1 obtenida en validación, con el objetivo de estudiar el comportamiento del modelo y detectar posibles indicios de sobreajuste. Cuando ambas curvas presentan valores próximos y una evolución similar, el modelo muestra un comportamiento estable; en cambio, si la F1 de entrenamiento es claramente superior a la de validación, puede existir un ajuste excesivo a los datos de entrenamiento. En estas gráficas, el eje X representa los distintos valores evaluados para el hiperparámetro, siguiendo el orden definido en la rejilla de búsqueda de cada modelo.

En segundo lugar, se generará una gráfica específica para la selección del mejor valor del hiperparámetro. En esta gráfica se mostrará, para cada valor evaluado, la media de la F1 obtenida en los distintos folds de la validación cruzada. De esta forma, se podrá identificar qué valor del hiperparámetro ofrece el mejor rendimiento y, por tanto, cuál será seleccionado para continuar con el proceso de ajuste secuencial.

Una vez obtenida la mejor configuración para cada modelo mediante validación cruzada, dichos modelos se evaluarán sobre el conjunto de prueba. Esta evaluación final permitirá comparar su rendimiento en datos no vistos y seleccionar aquellos modelos que presenten mejores resultados finales.

7.1. Modelos de aprendizaje automático evaluados

7.1.1. Modelos lineales

Regresión Logística

Para el estudio y análisis de este modelo, se han considerado los hiperparámetros y valores recogidos en la Tabla 7.1:

Tabla 7.1: Hiperparámetros evaluados para el modelo de regresión logística

Hiperparámetro	Valores evaluados
C	0.001, 0.01, 0.1, 0.5, 1, 5, 10, 50, 100
solver	lbfgs, liblinear, saga
class_weight	None, balanced
max_iter	200, 500, 1000, 2000

Además de estos hiperparámetros, también se consideró el hiperparámetro penalty. Sin embargo, no se incluyó en la experimentación porque se encuentra obsoleto y está previsto que sea eliminado en futuras versiones de scikit-learn.

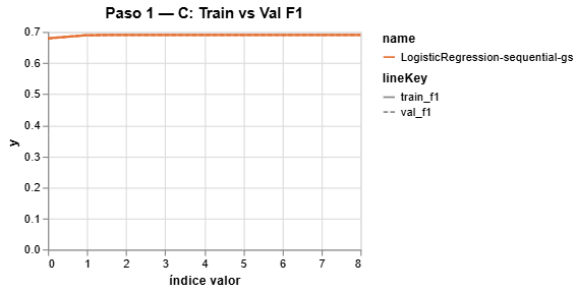
C

El hiperparámetro C, que controla el grado de regularización del modelo, determina cuánto se penalizan los errores de clasificación. Como se observa en la Figura 7.1, los resultados obtenidos para los distintos valores evaluados son muy similares, sin que ninguno de ellos destaque de forma clara sobre el resto.

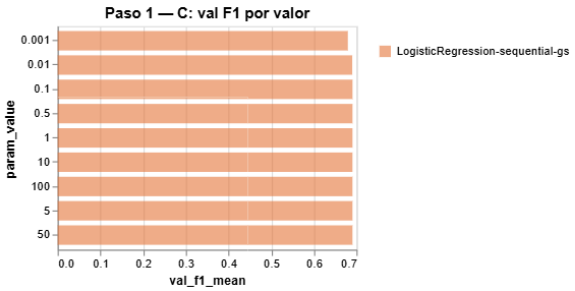
Este comportamiento podría indicar que el modelo es bastante estable frente a cambios en la regularización y que, en este problema, el hiperparámetro C no tiene un impacto decisivo en el rendimiento final.

Respecto a la comparación entre entrenamiento y validación, no se aprecian diferencias significativas en la métrica F1. Por tanto, no se observan indicios claros de sobreajuste en esta etapa.

Seleccionamos el valor 0.1 para C, ya que se obtiene un mejor rendimiento en la métrica F1.



(a) Comparación de la F1 en entrenamiento y validación



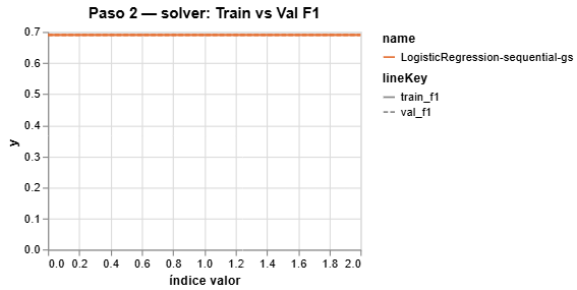
(b) Media de la F1 para cada valor del hiperparámetro

Figura 7.1: Análisis del hiperparámetro C en Regresión Logística

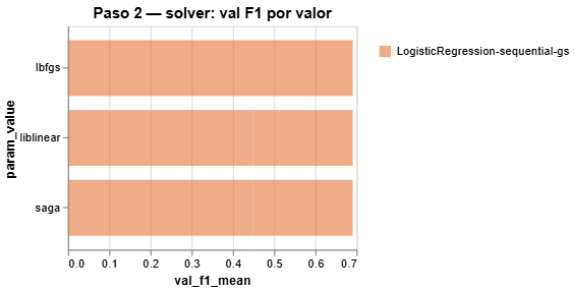
solver

El hiperparámetro solver permite seleccionar el algoritmo de optimización utilizado para ajustar el modelo. Como se observa en la Figura 7.2, los resultados obtenidos son muy similares entre los distintos algoritmos evaluados. Esto sugiere que, en este problema, el modelo presenta un comportamiento estable frente al algoritmo de optimización empleado y que este hiperparámetro no tiene un impacto destacado en el rendimiento.

Además, al comparar las curvas de entrenamiento y validación, no se observan diferencias significativas entre ambas, por lo que no se aprecian indicios claros de sobreajuste en este paso. Finalmente, el valor seleccionado para continuar con el ajuste secuencial ha sido solver = saga.



(a) Comparación de la F1 en entrenamiento y validación



(b) Media de la F1 para cada valor del hiperparámetro

Figura 7.2: Análisis del hiperparámetro solver en Regresión Logística

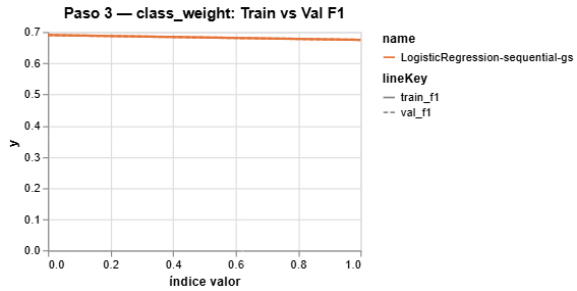
class_weight

El hiperparámetro class_weight asigna un factor de penalización a cada clase con el fin de ajustar su influencia en la función de pérdida. Cuando se establece en None, todos los errores contribuyen por igual al entrenamiento. En cambio, con class_weight='balanced', los pesos se calculan de forma inversamente proporcional al número de instancias de cada

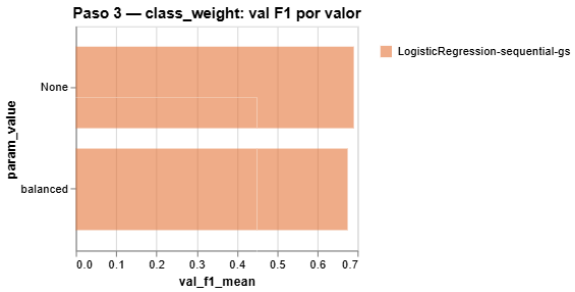
clase, de modo que los errores en la clase minoritaria penalizan más que los de la clase mayoritaria.

Podemos observar en la Figura 7.3, obtenemos un resultado similar para los 2 valores. Este comportamiento podría sugerir que, al tratarse de un desbalanceo moderado (aproximadamente 70/30), la aplicación de pesos de clase no tiene un impacto lo suficientemente significativo como para modificar el rendimiento del modelo de forma notoria. Otra posible explicación es que la regresión logística, al tratarse de un modelo lineal, no sea capaz de capturar completamente la relación entre las variables de entrada y la clase objetivo. En este contexto, el modelo podría haber alcanzado su techo de rendimiento, lo que explicaría que las variaciones en los hiperparámetros apenas produzcan cambios relevantes en la métrica F1.

Este hiperparámetro tampoco presenta indicios apreciables de sobreajuste, ya que las curvas de entrenamiento y validación muestran un comportamiento similar y se mantienen próximas entre sí. Finalmente, seleccionamos class_weight=None.



(a) Comparación de la F1 en entrenamiento y validación



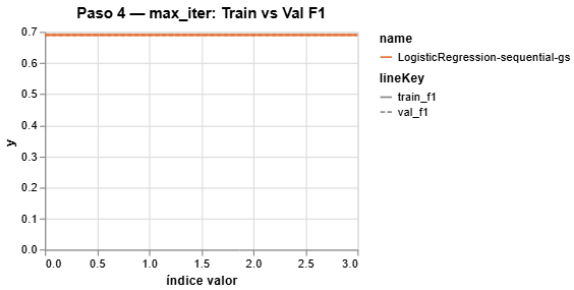
(b) Media de la F1 para cada valor del hiperparámetro

Figura 7.3: Análisis del hiperparámetro class_weight en Regresión Logística

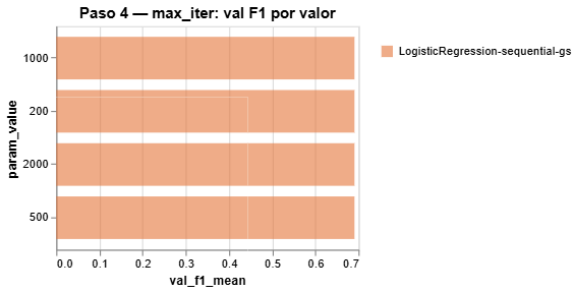
max_iter

Respecto al hiperparámetro max_iter, se observa en la Figura 7.4 que los resultados obtenidos son prácticamente iguales para todos los valores evaluados. Esto indica que el modelo alcanza la convergencia antes de llegar al número máximo de iteraciones, por lo que aumentar este parámetro no aporta mejoras en el rendimiento. En consecuencia, max_iter no resulta determinante en este caso práctico.

Además, al comparar las curvas de entrenamiento y validación, no se observan diferencias significativas entre ambas, por lo que no se aprecian indicios de sobreajuste.



(a) Comparación de la F1 en entrenamiento y validación



(b) Media de la F1 para cada valor del hiperparámetro

Figura 7.4: Análisis del hiperparámetro max_iter en Regresión Logística

De forma global, el modelo ha mostrado un comportamiento muy estable frente a las variaciones en los distintos hiperparámetros evaluados. En particular, se observa que a partir del valor 0.1 en C, la métrica F1 alcanza un nivel prácticamente constante, con mejoras muy ligeras o despreciables al modificar el resto de hiperparámetros.

Este comportamiento sugiere que el modelo ha alcanzado una región de rendimiento estable, en la que ajustes adicionales no permiten obtener mejoras significativas. En consecuencia, es posible que la regresión logística esté limitada por su propia capacidad para modelar el problema, más que por la elección de hiperparámetros concretos.

7.1.2. Modelos basados en distancias

K-Nearest Neighbors

Para el estudio y análisis de este modelo, se han considerado los hiperparámetros y valores recogidos en la Tabla 7.2:

Tabla 7.2: Hiperparámetros evaluados para el modelo k-NN

Hiperparámetro	Valores evaluados
n_neighbors	3, 5, 7, 9, 11, 15, 21, 25, 31, 35, 41
weights	uniform, distance
metric	minkowski, manhattan, euclidean

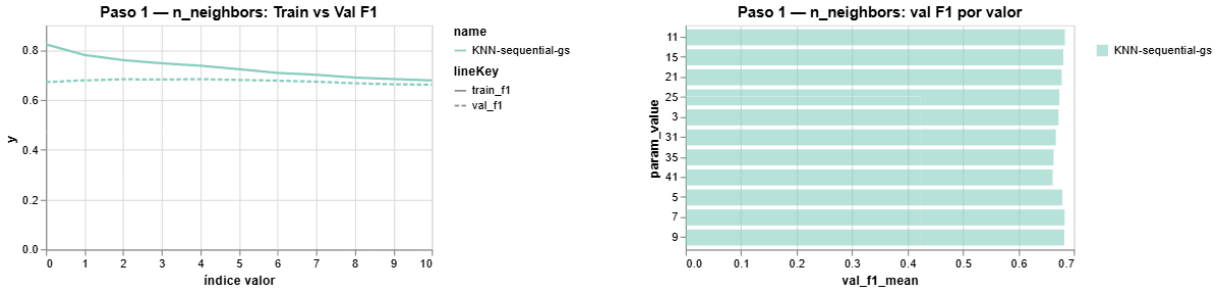
n_neighbors

El hiperparámetro n_neighbors define el número de vecinos utilizados para clasificar un nuevo punto.

Como se muestra en la Figura 7.5, los resultados obtenidos son similares para los distintos valores evaluados, aunque se aprecia una ligera tendencia descendente de la F1 conforme aumenta el número de vecinos. Esto podría sugerir que las clases se encuentran

parcialmente mezcladas entre sí, de forma que considerar un mayor número de vecinos introduce información de puntos pertenecientes a la clase contraria y reduce el rendimiento del modelo. Seleccionamos n_neighbors = 11.

Por otro lado, en la gráfica comparativa entre entrenamiento y validación se observa una ligera diferencia entre ambas curvas, siendo la F1 de entrenamiento superior a la de validación. No obstante, esta diferencia no es lo suficientemente elevada como para considerar que exista sobreajuste significativo.



(a) Comparación de la F1 en entrenamiento y validación

(b) Media de la F1 para cada valor del hiperparámetro

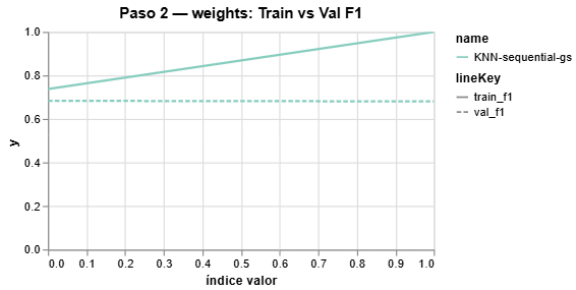
Figura 7.5: Análisis del hiperparámetro n_neighbors en KNN

weights

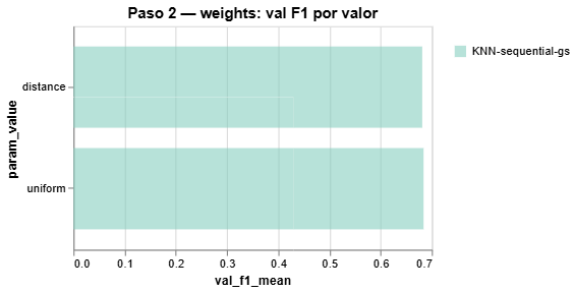
Respecto al hiperparámetro weights, este permite asignar un peso diferente a los vecinos en función de su distancia. En el caso de uniform, todos los vecinos contribuyen por igual, mientras que en distance los vecinos más cercanos tienen mayor influencia en la decisión.

Como se muestra en la Figura 7.6, los resultados obtenidos son muy similares para ambas configuraciones, siendo ligeramente superior el valor uniform. Este comportamiento puede deberse a la presencia de cierto solapamiento entre clases o a la existencia de ruido en los datos, de modo que dar mayor peso a los vecinos más cercanos no aporta una mejora significativa e incluso puede introducir pequeñas inestabilidades en la predicción.

Además, al comparar las curvas de entrenamiento y validación, se observa una diferencia apreciable entre ambas, ya que la F1 en entrenamiento es superior a la de validación. Esta falta de convergencia entre ambas curvas sugiere la presencia de cierto sobreajuste en el modelo. El valor escogido para weights es uniform.



(a) Comparación de la F1 en entrenamiento y validación



(b) Media de la F1 para cada valor del hiperparámetro

Figura 7.6: Análisis del hiperparámetro weights en KNN

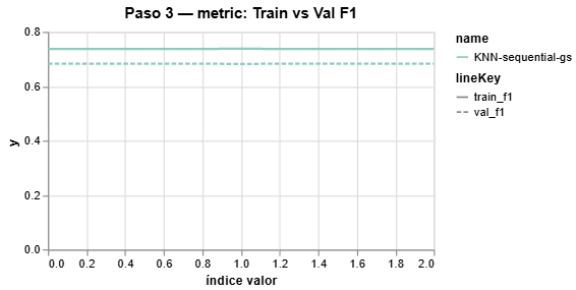
metric

El hiperparámetro metric define el tipo de distancia utilizada para determinar cuáles son los vecinos más cercanos. Las distintas métricas consideradas han sido descritas previamente en los fundamentos teóricos del modelo 3.2.2.

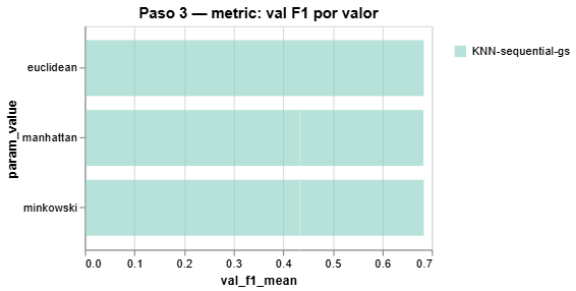
Como se muestra en la Figura 7.7, las tres métricas evaluadas presentan un comportamiento muy similar, con resultados prácticamente iguales. No obstante, la distancia manhattan obtiene un rendimiento ligeramente inferior, con una diferencia aproximada de una milésima respecto a euclidean y minkowski. Además, este hiperparámetro no permite mejorar la F1 obtenida en el paso anterior.

Al comparar las curvas de entrenamiento y validación, se observa una ligera diferencia entre ambas, con valores algo superiores en entrenamiento. Sin embargo, esta diferencia es reducida, por lo que no se considera que exista sobreajuste significativo.

Finalmente, se selecciona metric = minkowski, al ser la primera métrica evaluada y no ser superada por el resto de valores analizados.



(a) Comparación de la F1 en entrenamiento y validación



(b) Media de la F1 para cada valor del hiperparámetro

Figura 7.7: Análisis del hiperparámetro metric en KNN

El modelo podría presentar una ligera tendencia al sobreajuste, dado que obtiene resultados algo mejores en entrenamiento que en validación. Este comportamiento puede deberse

a la naturaleza del propio conjunto de datos. Sin embargo, al tratarse de una diferencia muy reducida, esta interpretación debe tomarse con cautela.

El hiperparámetro más relevante ha sido n_neighbors, puesto que condiciona principalmente el rendimiento del modelo. A partir de ese ajuste, el resto de hiperparámetros apenas modifica la F1, por lo que no aportan mejoras significativas.

7.1.3. Máquina de vectores de soporte

SVM

Para el estudio y análisis de este modelo, se han considerado los hiperparámetros y valores recogidos en la Tabla 7.3:

Tabla 7.3: Hiperparámetros evaluados para el modelo SVM

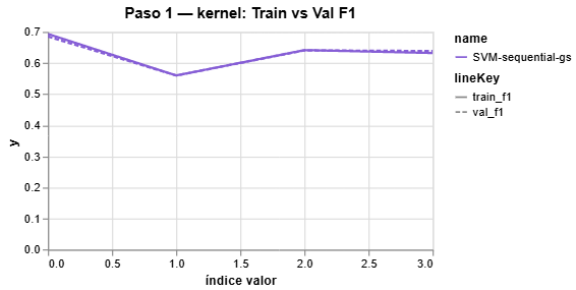
Hiperparámetro	Valores evaluados
kernel	linear, poly, sigmoid, rbf
C	0.01, 0.1, 1
gamma	0.001, 0.01, 0.1, 1, auto, scale
class_weight	None, balanced

Kernel

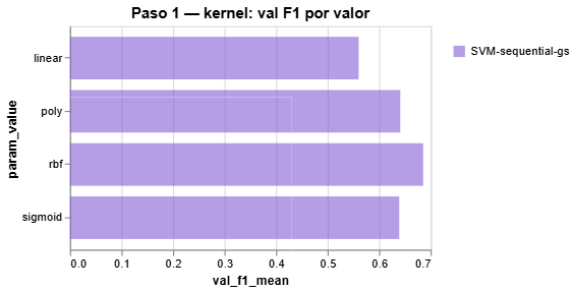
En la Figura 7.8 se analiza el hiperparámetro kernel. Se observa que el mejor resultado se obtiene con el kernel rbf, alcanzando una F1 cercana al 70 %. A continuación, se sitúan los kernels polinómico y sigmoide, mientras que el kernel lineal presenta el peor rendimiento. Este resultado sugiere que los datos no son fácilmente separables mediante una función lineal.

Respecto a la comparación entre entrenamiento y validación, no se aprecian diferencias significativas en la métrica F1. Por tanto, no se observan indicios claros de sobreajuste en esta etapa.

Por lo tanto, se selecciona el kernel rbf para continuar con el siguiente paso de la búsqueda secuencial.



(a) Comparación de la F1 en entrenamiento y validación



(b) Media de la F1 para cada valor del hiperparámetro

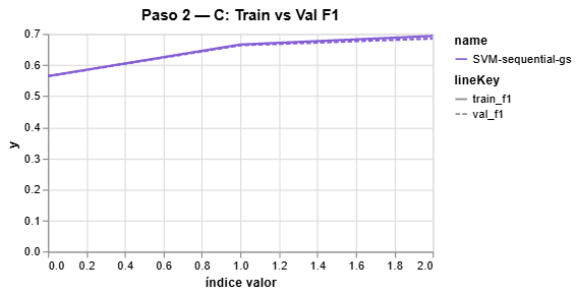
Figura 7.8: Análisis del hiperparámetro kernel en SVM

Regularización (C)

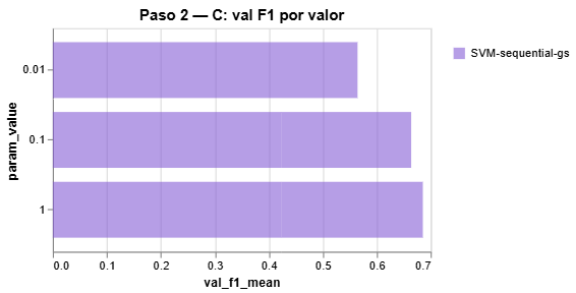
Respecto al hiperparámetro C, que controla el grado de penalización de los errores de clasificación, de forma que valores más altos castigan en mayor medida dichos errores y generan márgenes más estrechos.

Como se muestra en la Figura 7.9, el modelo alcanza un mejor rendimiento según la métrica F1 con el valor 1 de C. Esto indica que, en este caso, una penalización mayor permite ajustar mejor la frontera de decisión, lo que sugiere que las clases no son fácilmente separables mediante un hiperplano con márgenes amplios.

Además, en la misma figura, al comparar las curvas de entrenamiento y validación, no se observan diferencias significativas entre ambas. Por tanto, no se aprecian indicios claros de sobreajuste en este paso.El valor seleccionado ha sido C = 1



(a) Comparación de la F1 en entrenamiento y validación



(b) Media de la F1 para cada valor del hiperparámetro

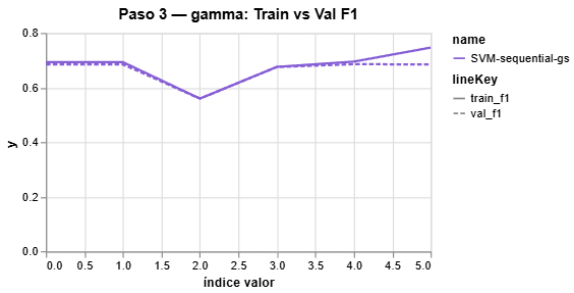
Figura 7.9: Análisis del hiperparámetro C en SVM

Gamma (γ)

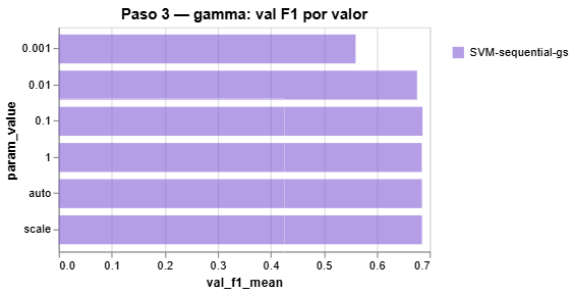
Respecto al hiperparámetro gamma, este representa el grado de influencia que tiene cada punto de entrenamiento sobre los demás en la construcción de la frontera de decisión. Los resultados se muestran en la Figura 7.10.

Los resultados obtenidos son muy similares para la mayoría de los valores evaluados, excepto para el valor 0.001, donde se observa una disminución notable de la F1. Esto sugiere que, en esta etapa del ajuste, el comportamiento del modelo está bastante condicionado por los hiperparámetros seleccionados previamente, por lo que gamma no introduce una mejora significativa adicional. No obstante, los valores muy bajos parecen penalizar el rendimiento, probablemente porque generan una frontera de decisión demasiado suave, al hacer que cada punto tenga influencia sobre zonas más amplias del espacio.

Además, al comparar las curvas de entrenamiento y validación, no se observan diferencias significativas entre ambas para la mayoría de los valores evaluados. El valor seleccionado ha sido $\gamma = 0.1$



(a) Comparación de la F1 en entrenamiento y validación



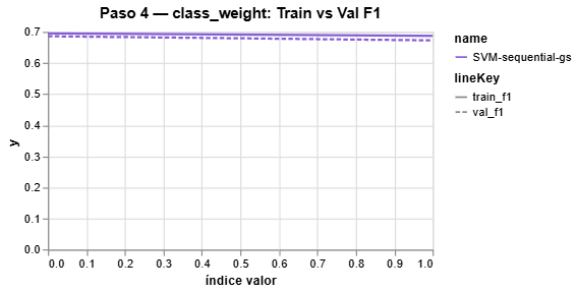
(b) Media de la F1 para cada valor del hiperparámetro

Figura 7.10: Análisis del hiperparámetro gamma en SVM

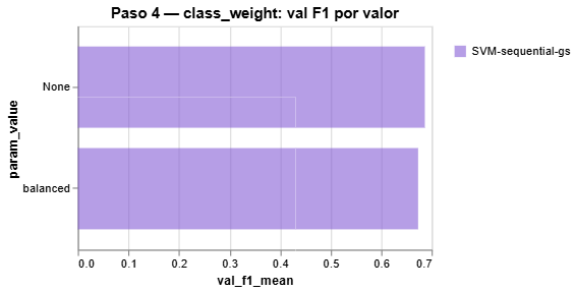
Ponderación a las clases

Respecto al hiperparámetro class_weight, este permite asignar un peso diferente a cada clase durante el entrenamiento. Su objetivo es compensar posibles desbalances, penalizando más los errores cometidos sobre la clase minoritaria.

Como se muestra en la Figura 7.11, los resultados son muy similares tanto al utilizar balanceo como al no aplicarlo. No obstante, el mejor rendimiento se obtiene cuando class_weight no aplica ningún ajuste adicional. Además, las curvas de entrenamiento y validación se mantienen próximas, por lo que no se observan indicios claros de sobreajuste. Por tanto, class_weight = None, será el valor seleccionado.



(a) Comparación de la F1 en entrenamiento y validación



(b) Media de la F1 para cada valor del hiperparámetro

Figura 7.11: Análisis del hiperparámetro class_weight en SVM

En conjunto, el modelo SVM presenta un comportamiento estable y un rendimiento competitivo. El hiperparámetro más influyente es el kernel, siendo rbf el que proporciona mejores resultados, lo que indica que el problema requiere una frontera de decisión no lineal. Además, el resto de hiperparámetros introducen únicamente mejoras marginales.

No se observan diferencias relevantes entre entrenamiento y validación, por lo que no hay indicios claros de sobreajuste.

7.1.4. Modelos basados en árboles

Decision Tree

Para el estudio y análisis de este modelo, se han considerado los hiperparámetros y valores recogidos en la Tabla 7.4:

Tabla 7.4: Hiperparámetros evaluados para el modelo Decision Tree

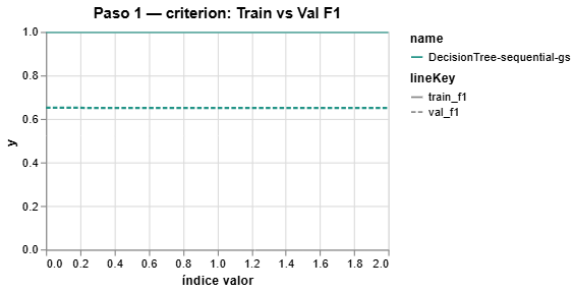
Hiperparámetro	Valores evaluados
criterion	gini, entropy, log_loss
max_depth	3, 5, 8, 10, 15, 20, 30, None
min_samples_split	2, 5, 10, 20
min_samples_leaf	1, 2, 4, 8
max_features	None, sqrt, log2, 0.3, 0.5, 0.7
class_weight	None, balanced

criterion

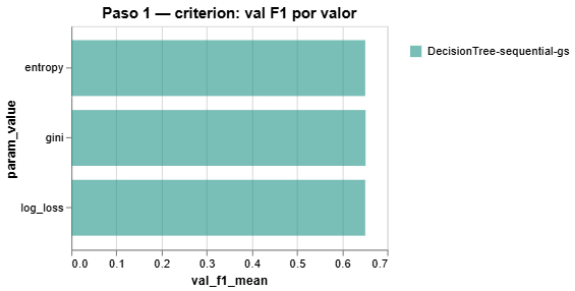
Respecto al hiperparámetro criterion, este define la medida utilizada para evaluar la calidad de cada división del árbol. Su objetivo es medir la impureza de los nodos: cuanto más se aproxima este valor a 0, más homogéneas son las muestras contenidas en el nodo.

Como se observa en la Figura 7.12, los resultados obtenidos son bastante similares entre las diferentes medidas, aunque el criterio gini presenta un rendimiento ligeramente superior.

En esta primera etapa, al evaluar únicamente el criterio de división, se observa una tendencia al sobreajuste, ya que todavía no se han ajustado otros hiperparámetros que permiten regular la complejidad del modelo. Seleccionamos criterion=gini.



(a) Comparación de la F1 en entrenamiento y validación



(b) Media de la F1 para cada valor del hiperparámetro

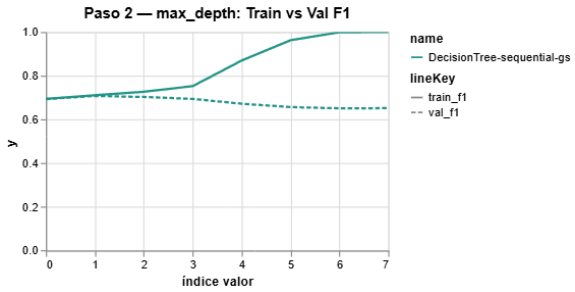
Figura 7.12: Análisis del hiperparámetro criterion en Decision Tree

max_depth

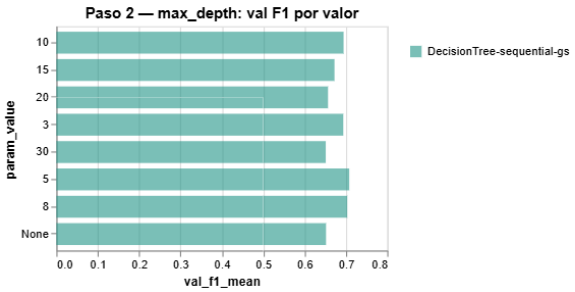
Este hiperparámetro define la profundidad máxima del árbol y permite regular la complejidad del modelo para reducir el riesgo de sobreajuste.

Como se observa en la Figura 7.13, los mejores resultados se obtienen con valores comprendidos entre 3 y 10. A partir de ese rango, el aumento de la profundidad empeora el rendimiento en validación, ya que el árbol tiende a ajustarse en exceso a los datos de entrenamiento.

Además, en la gráfica de entrenamiento y validación se aprecia una diferencia cada vez mayor entre ambas curvas conforme aumenta la profundidad del árbol. Esto confirma que valores elevados de max_depth favorecen el sobreajuste. Seleccionamos el valor 5 para max_depth.



(a) Comparación de la F1 en entrenamiento y validación



(b) Media de la F1 para cada valor del hiperparámetro

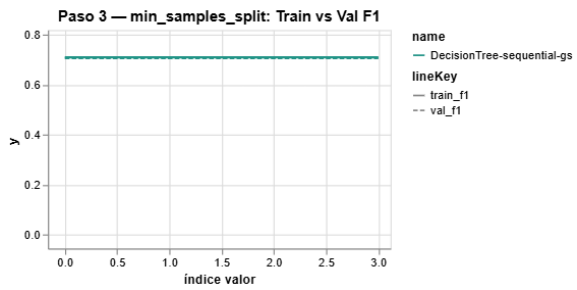
Figura 7.13: Análisis del hiperparámetro max_depth en Decision Tree

min_samples_split

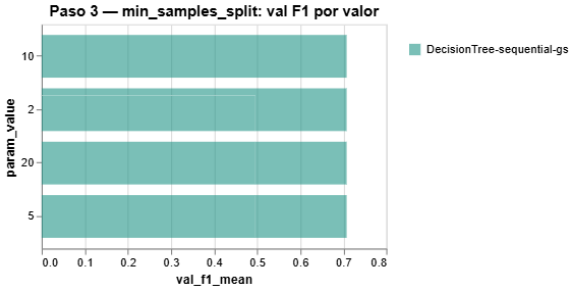
Este hiperparámetro establece el número mínimo de instancias que debe contener un nodo para poder dividirse.

Como se observa en la Figura 7.14, los resultados obtenidos son prácticamente idénticos para los distintos valores evaluados. Esto sugiere que, en este modelo, min_samples_split tiene una influencia muy reducida sobre el rendimiento. Elegimos el valor 2 para min_samples_split.

Además, tras haber limitado previamente la complejidad del árbol mediante max_depth, este hiperparámetro no introduce cambios relevantes. Las curvas de entrenamiento y validación se mantienen próximas, por lo que no se aprecian indicios claros de sobreajuste.



(a) Comparación de la F1 en entrenamiento y validación



(b) Media de la F1 para cada valor del hiperparámetro

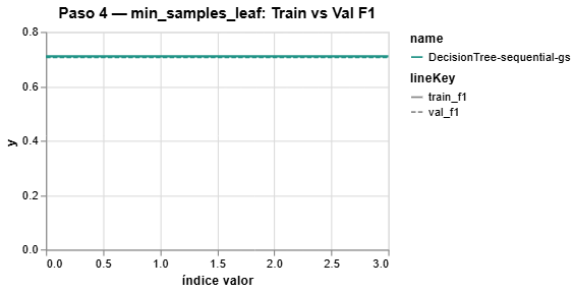
Figura 7.14: Análisis del hiperparámetro min_samples_split en Decision Tree

min_samples_leaf

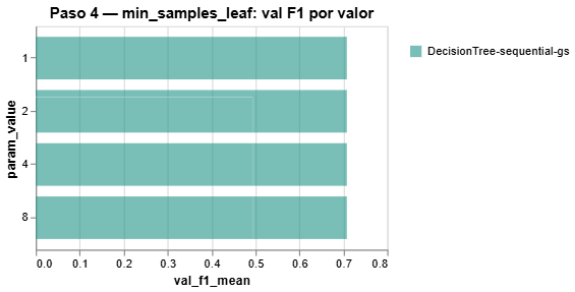
Este hiperparámetro establece el número mínimo de instancias que debe contener un nodo terminal. Se utiliza habitualmente para regular la complejidad del árbol y reducir el riesgo de sobreajuste.

Como se observa en la Figura 7.15, los resultados obtenidos son prácticamente idénticos para los distintos valores evaluados. Esto podría indicar que, en este modelo, min_samples_leaf tiene una influencia muy limitada sobre el rendimiento. El valor seleccionado para min_samples_leaf es 1.

Además, al igual que en el hiperparámetro anterior, tras haber limitado previamente la complejidad del árbol mediante max_depth, min_samples_leaf no introduce cambios relevantes. Las curvas de entrenamiento y validación se mantienen prácticamente unidas, por lo que no se aprecian indicios claros de sobreajuste.



(a) Comparación de la F1 en entrenamiento y validación



(b) Media de la F1 para cada valor del hiperparámetro

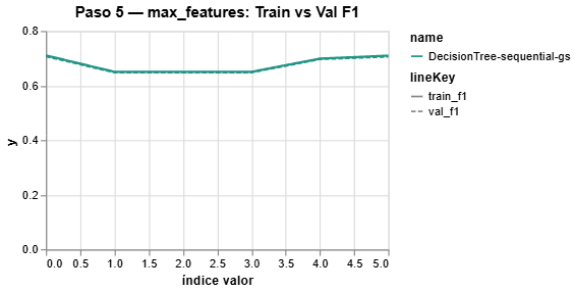
Figura 7.15: Análisis del hiperparámetro min_samples_leaf en Decision Tree

max_features

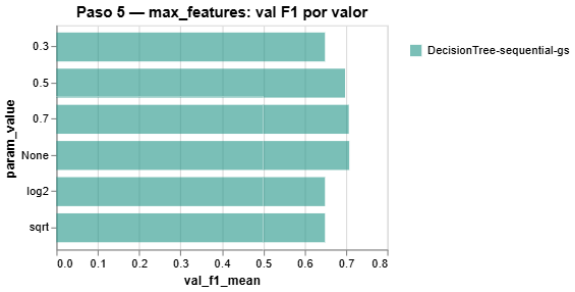
Respecto al hiperparámetro max_features, este establece el número máximo de características que se evalúan en cada nodo para realizar la división.

Como se observa en la Figura 7.16, el rendimiento disminuye cuando se evalúa un porcentaje menor de características. El mejor resultado se obtiene cuando max_features toma el valor None, lo que indica que considerar todas las variables disponibles permite realizar divisiones más adecuadas.

Además, tras haber limitado previamente la complejidad del árbol mediante max_depth, max_features no introduce señales claras de sobreajuste. Las curvas de entrenamiento y validación se mantienen próximas.



(a) Comparación de la F1 en entrenamiento y validación



(b) Media de la F1 para cada valor del hiperparámetro

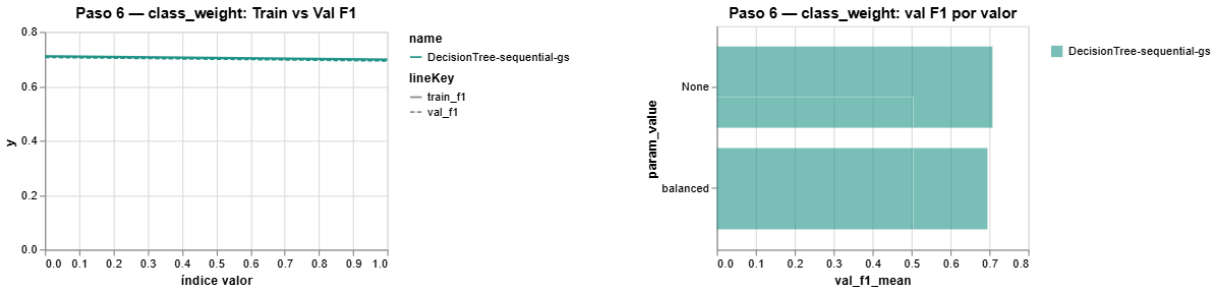
Figura 7.16: Análisis del hiperparámetro max_features en Decision Tree

class_weight

Respecto al hiperparámetro class_weight, este permite asignar un peso distinto a cada clase durante el entrenamiento con el objetivo de compensar posibles desbalances en el conjunto de datos.

Como se observa en la Figura 7.17, el mejor resultado se obtiene cuando class_weight toma el valor None, siendo ligeramente superior al obtenido con la opción balanced. Esto podría indicar que el desbalance presente en los datos no es lo suficientemente significativo como para requerir una ponderación adicional de las clases.

Además, las curvas de entrenamiento y validación se mantienen prácticamente unidas, por lo que no se observan indicios claros de sobreajuste.



(a) Comparación de la F1 en entrenamiento y validación

(b) Media de la F1 para cada valor del hiperparámetro

Figura 7.17: Análisis del hiperparámetro class_weight en Decision Tree

El hiperparámetro más importante en este modelo es max_depth, ya que ha supuesto una mejora notable respecto al valor de F1 obtenido al evaluar criterion. Además, a partir de este ajuste se reduce el sobreajuste y el resto de hiperparámetros apenas incrementan la F1.

En conjunto, el árbol de decisión ofrece un rendimiento estable y constituye una base sólida para los modelos basados en árboles que se utilizarán posteriormente.

Bagging

Para el estudio y análisis de este modelo, se han considerado los hiperparámetros y valores recogidos en la Tabla 7.5:

Tabla 7.5: Hiperparámetros evaluados para el modelo Bagging

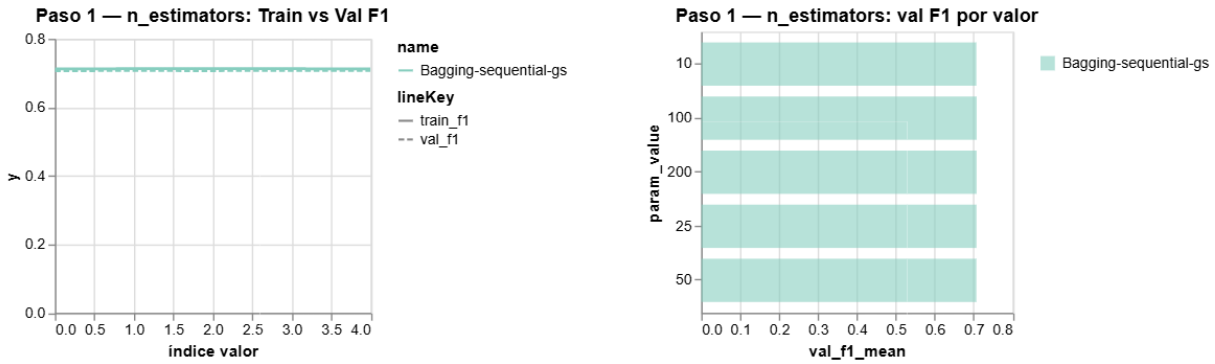
Hiperparámetro	Valores evaluados
n_estimators	10, 25, 50, 100, 200
max_samples	0.5, 0.7, 0.9, 1.0
max_features	0.5, 0.7, 0.9, 1.0
bootstrap	True, False
bootstrap_features	False, True

n_estimators

El hiperparámetro n_estimators permite definir el número de estimadores base que tendrá el modelo. En este caso, los estimadores se corresponden a árboles de decisión.

En la Figura 7.18, se observa que los resultados obtenidos son prácticamente idénticos entre los distintos valores evaluados, con variaciones de apenas unas diezmilésimas. El mejor rendimiento se obtiene con n_estimators = 100, aunque la diferencia respecto al resto de configuraciones es muy reducida.

Además, al tratarse de un método de combinación de árboles, el modelo muestra un comportamiento robusto y estable. Incluso en este primer paso del ajuste, no se observan indicios claros de sobreajuste al comparar las curvas de entrenamiento y validación, pese a que este hiperparámetro no está directamente orientado a reducir la complejidad del modelo.



(a) Comparación de la F1 en entrenamiento y validación

(b) Media de la F1 para cada valor del hiperparámetro

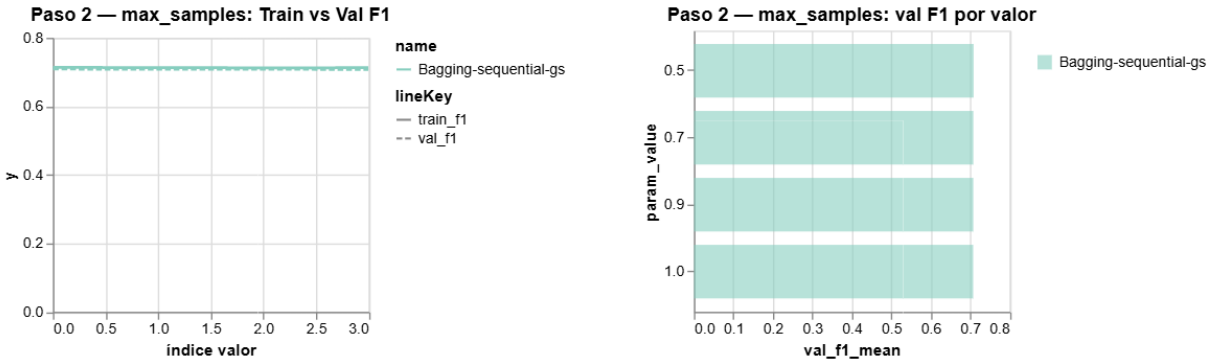
Figura 7.18: Análisis del hiperparámetro n_estimators en Bagging

max_samples

Este hiperparámetro permite definir la proporción de instancias que utilizará cada árbol durante su entrenamiento.

Como se observa en la Figura 7.19, los resultados obtenidos son muy similares entre los distintos valores evaluados, con diferencias de apenas unas pocas milésimas. Esto sugiere que, al tratarse de un ensemble formado por múltiples árboles, pequeñas variaciones en la proporción de instancias utilizadas por cada estimador apenas afectan al rendimiento global del modelo. No obstante, el mejor rendimiento se obtiene con max_samples = 0.5.

En la gráfica comparativa entre entrenamiento y validación tampoco se aprecian diferencias significativas entre ambas curvas, por lo que el modelo mantiene un comportamiento estable y no se observan indicios claros de sobreajuste.



(a) Comparación de la F1 en entrenamiento y validación

(b) Media de la F1 para cada valor del hiperparámetro

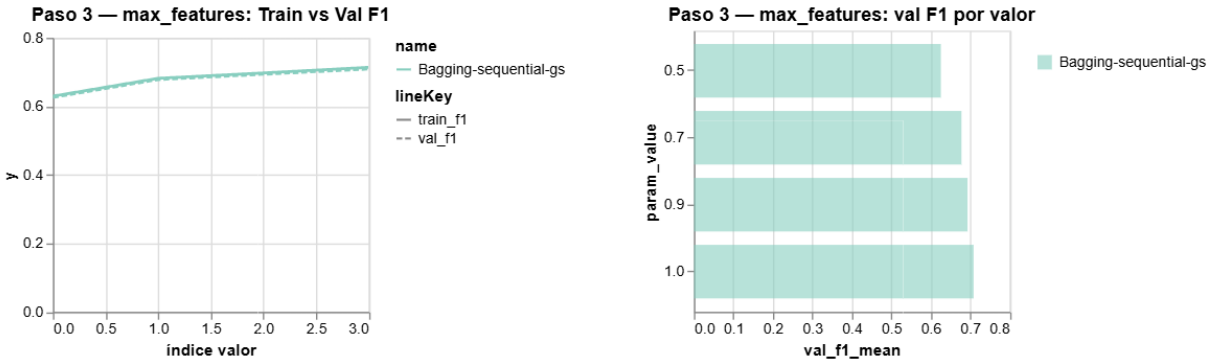
Figura 7.19: Análisis del hiperparámetro max_samples en Bagging

max_features

El hiperparámetro max_features permite definir la proporción de variables predictoras que podrá utilizar cada árbol durante su entrenamiento.

Como se observa en la Figura 7.20, existe una ligera tendencia a obtener peores resultados de F1 en validación conforme disminuye la proporción de variables utilizadas. Esto sugiere que, en este caso, el modelo obtiene mejores resultados cuando cada árbol dispone de una mayor cantidad de información.

Respecto a las curvas de entrenamiento y validación, no se observan diferencias notorias entre ambas, por lo que no se aprecian indicios claros de sobreajuste.



(a) Comparación de la F1 en entrenamiento y validación

(b) Media de la F1 para cada valor del hiperparámetro

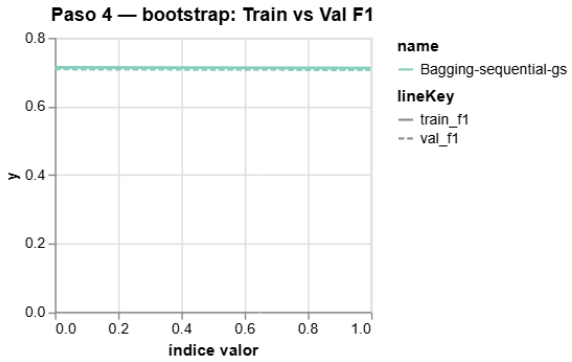
Figura 7.20: Análisis del hiperparámetro max_features en Bagging

bootstrap

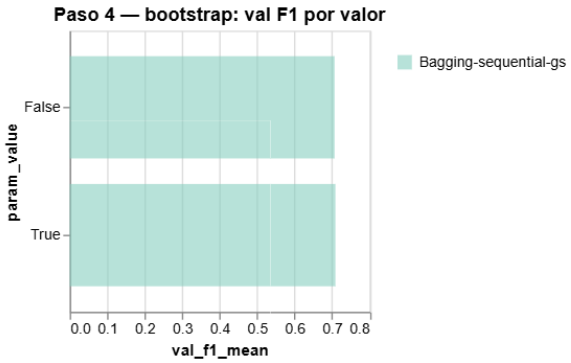
El hiperparámetro bootstrap permite definir si cada árbol se entrena utilizando muestras aleatorias con reemplazo, pudiendo repetirse instancias y aumentando así la diversidad entre los árboles del ensemble, o si, por el contrario, se emplean subconjuntos sin repetición.

Como se observa en la Figura 7.21, los resultados obtenidos para ambos valores evaluados son prácticamente idénticos. Sin embargo, bootstrap = True obtiene un rendimiento ligeramente superior, con una diferencia de apenas unas milésimas. Esto sugiere que el aumento de diversidad entre los árboles puede favorecer sutilmente el rendimiento del modelo.

Las curvas de entrenamiento y validación se mantienen prácticamente unidas, por lo que no se aprecian indicios claros de sobreajuste.



(a) Comparación de la F1 en entrenamiento y validación



(b) Media de la F1 para cada valor del hiperparámetro

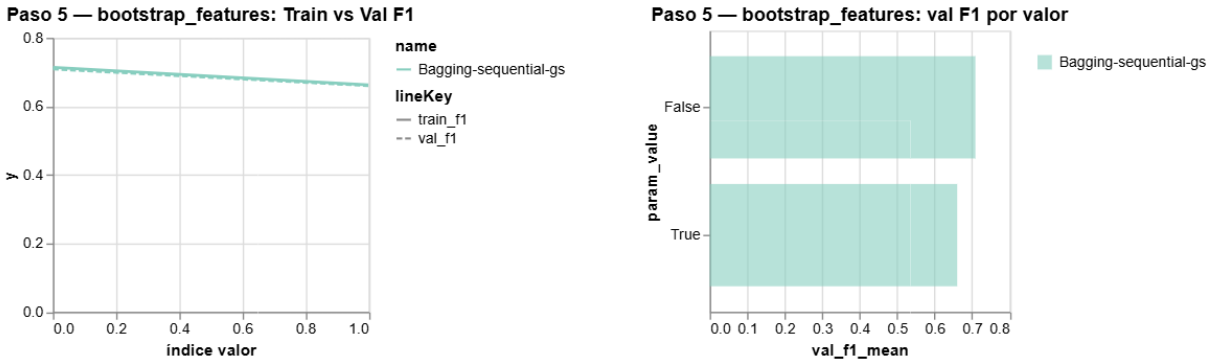
Figura 7.21: Análisis del hiperparámetro bootstrap en Bagging

bootstrap_features

Respecto al hiperparámetro bootstrap_features, este funciona de manera similar a bootstrap, pero aplicado sobre las variables predictoras en lugar de sobre las instancias. Es decir, permite seleccionar las variables utilizadas por cada estimador mediante muestreo con reemplazo.

Como se observa en la Figura 7.22, cuando bootstrap_features = True se obtiene un rendimiento claramente inferior respecto a bootstrap_features = False. Esto sugiere que permitir la repetición de variables predictoras no favorece al modelo en este problema, posiblemente porque reduce la información útil disponible para algunos árboles y dificulta la identificación de patrones relevantes entre las clases.

Las curvas de entrenamiento y validación se mantienen próximas, por lo que no se aprecian indicios claros de sobreajuste.



(a) Comparación de la F1 en entrenamiento y validación

(b) Media de la F1 para cada valor del hiperparámetro

Figura 7.22: Análisis del hiperparámetro bootstrap_features en Bagging

Bagging se muestra como una alternativa sólida y estable dentro de los modelos evaluados, ya que presenta un buen rendimiento respecto a la métrica F1. Además, las curvas de entrenamiento y validación apenas presentan diferencias entre sí durante el proceso de ajuste, por lo que no se observan indicios claros de sobreajuste.

Extra Trees

Para el estudio y análisis de este modelo, se han considerado los hiperparámetros y valores recogidos en la Tabla 7.6:

Tabla 7.6: Hiperparámetros evaluados para el modelo Extra Trees

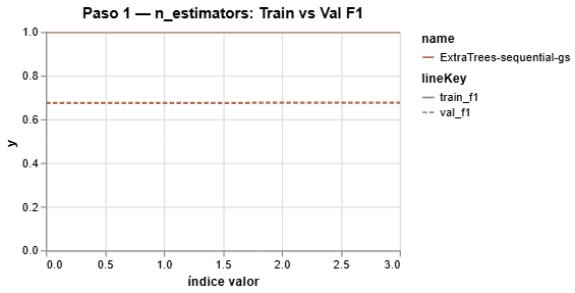
Hiperparámetro	Valores evaluados
n_estimators	50, 100, 200, 300
max_depth	5, 10, 15, 20, 30, None
min_samples_split	2, 5, 10, 20
min_samples_leaf	1, 2, 4, 8
max_features	sqrt, log2, 0.3, 0.5, 0.7
criterion	gini, entropy
class_weight	None, balanced, balanced_subsample

n_estimators

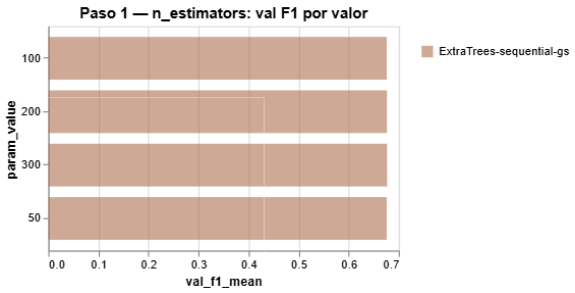
El hiperparámetro n_estimators permite definir el número de árboles que componen el bosque. Como se observa en la Figura 7.23, los resultados obtenidos son prácticamente idénticos entre los distintos valores evaluados, con variaciones de apenas unas milésimas. En este caso, el mejor rendimiento en validación se obtiene con n_estimators = 300.

Al tratarse del primer hiperparámetro evaluado, todavía no se ha regulado completamente la complejidad del modelo, por lo que se aprecia una notoria tendencia al sobreajuste. En

los siguientes pasos se ajustarán otros hiperparámetros con el objetivo de reducirlo en la medida de lo posible.



(a) Comparación de la F1 en entrenamiento y validación



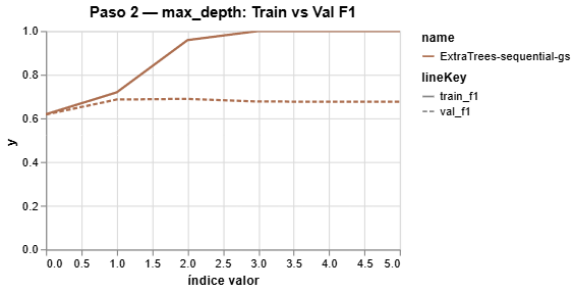
(b) Media de la F1 para cada valor del hiperparámetro

Figura 7.23: Análisis del hiperparámetro n_estimators en Extra Trees

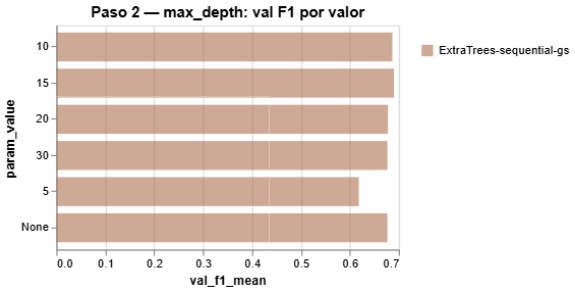
max_depth

Como podemos observar en la Figura 7.24, los valores evaluados presentan una F1 de validación similar, aunque se aprecia cierta tendencia a un menor rendimiento conforme aumenta la profundidad. Del mismo modo, profundidades inferiores a 10 también ofrecen resultados algo peores.

Además, en la comparación entre entrenamiento y validación se observa una separación creciente entre ambas curvas a medida que aumenta la profundidad, lo que indica una mayor tendencia al sobreajuste. Finalmente, seleccionamos max_depth = 15.



(a) Comparación de la F1 en entrenamiento y validación



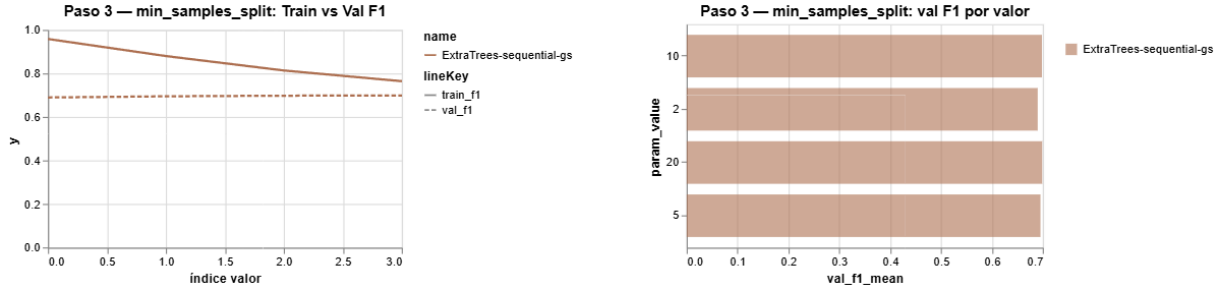
(b) Media de la F1 para cada valor del hiperparámetro

Figura 7.24: Análisis del hiperparámetro max_depth en Extra Trees

min_samples_split

En la Figura 7.25, los resultados obtenidos son prácticamente iguales para los distintos valores evaluados. Esto sugiere que, en este modelo, min_samples_split influye de manera muy limitada sobre el rendimiento. Por ello, se selecciona min_samples_split = 20.

Además, se observa que el sobreajuste tiende a reducirse al aumentar el número mínimo de instancias necesarias para realizar una división, ya que el modelo evita generar particiones demasiado específicas.



(a) Comparación de la F1 en entrenamiento y validación

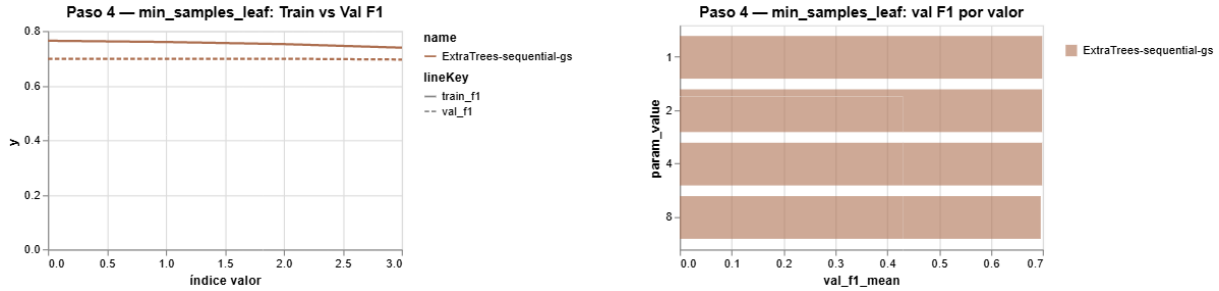
(b) Media de la F1 para cada valor del hiperparámetro

Figura 7.25: Análisis del hiperparámetro min_samples_split en Extra Trees

min_samples_leaf

Respecto al hiperparámetro min_samples_leaf, en la Figura 7.26 se observa que los resultados apenas difieren entre los distintos valores evaluados. Esto indica que min_samples_leaf tiene una influencia muy limitada sobre el rendimiento. El valor seleccionado es min_samples_leaf = 1.

Además, la distancia entre las curvas de entrenamiento y validación es menor que en los hiperparámetros evaluados anteriormente. Esto podría sugerir que, en combinación con los ajustes previos, este hiperparámetro contribuye a reducir el sobreajuste del modelo.



(a) Comparación de la F1 en entrenamiento y validación

(b) Media de la F1 para cada valor del hiperparámetro

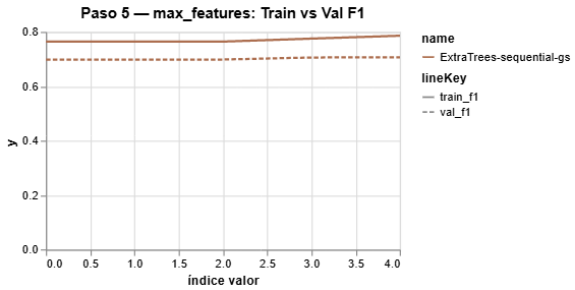
Figura 7.26: Análisis del hiperparámetro min_samples_leaf en Extra Trees

max_features

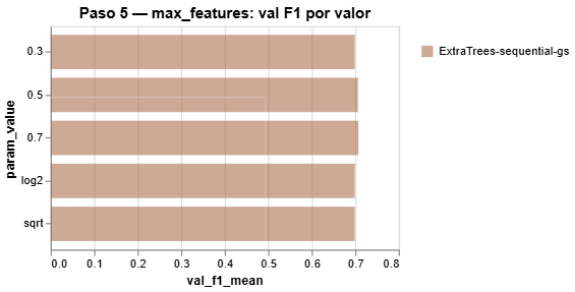
Respecto al hiperparámetro max_features, en la Figura 7.27 se observan resultados muy similares para los distintos valores evaluados. Esto puede explicarse por la naturaleza del

modelo Extra Trees, que ya introduce una elevada aleatoriedad en la selección de los puntos de corte. Se selecciona max_features=0.7.

Asimismo, max_features apenas influye en la reducción del sobreajuste, ya que la distancia entre las curvas de entrenamiento y validación es muy similar a la observada en el hiperparámetro analizado previamente.



(a) Comparación de la F1 en entrenamiento y validación



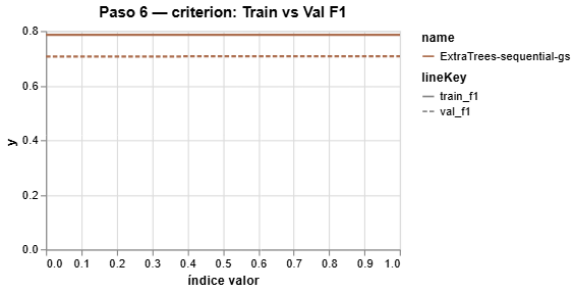
(b) Media de la F1 para cada valor del hiperparámetro

Figura 7.27: Análisis del hiperparámetro max_features en Extra Trees

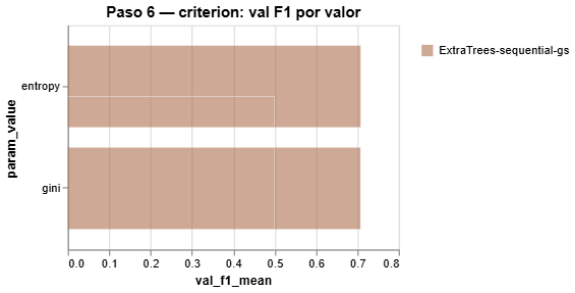
criterion

El hiperparámetro criterion permite definir la medida utilizada para evaluar la homogeneidad de los nodos. En la Figura 7.28, los resultados obtenidos difieren únicamente en unas pocas milésimas, siendo criterion = entropy el valor que ofrece un rendimiento ligeramente superior.

Además, al tratarse de un hiperparámetro relacionado con el criterio de división y no con una restricción directa sobre la complejidad del modelo, no contribuye de forma significativa a reducir el sobreajuste.



(a) Comparación de la F1 en entrenamiento y validación



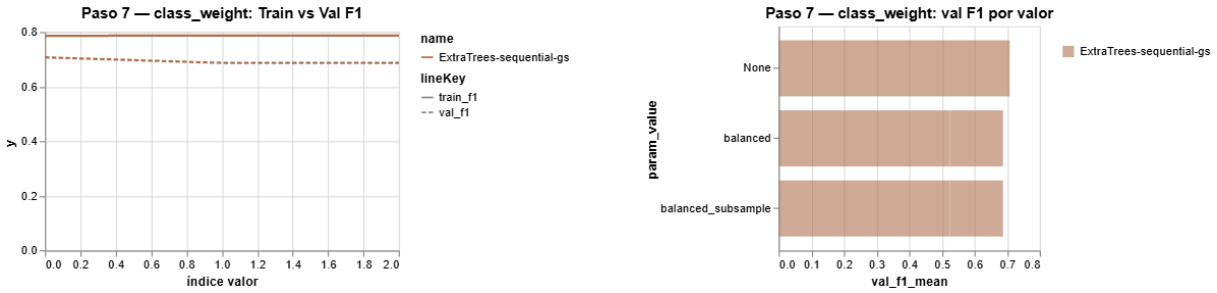
(b) Media de la F1 para cada valor del hiperparámetro

Figura 7.28: Análisis del hiperparámetro criterion en Extra Trees

class_weight

Respecto al hiperparámetro class_weight, en la Figura 7.29 se observa que el mejor resultado se obtiene cuando class_weight toma el valor None, siendo ligeramente superior a las opciones balanced y balanced_subsample. Esto podría indicar que el desbalance presente en los datos no es lo suficientemente significativo como para requerir una ponderación adicional de las clases. Además, al tratarse del último hiperparámetro evaluado, su efecto puede estar condicionado por los ajustes realizados previamente, por lo que no introduce cambios significativos.

Asimismo, las curvas de entrenamiento y validación apenas varían respecto a los hiperparámetros evaluados anteriormente, por lo que no se aprecian cambios relevantes en términos de sobreajuste.



(a) Comparación de la F1 en entrenamiento y validación

(b) Media de la F1 para cada valor del hiperparámetro

Figura 7.29: Análisis del hiperparámetro class_weight en Extra Trees

En teoría, al introducir una mayor aleatoriedad en la construcción de los árboles y aumentar el número de predictores, Extra Trees debería reducir el sobreajuste respecto a un árbol individual. Sin embargo, en este caso no se consigue una reducción clara de este comportamiento.

Esto podría deberse a la propia naturaleza del conjunto de datos, donde la aleatoriedad adicional introducida por el modelo no resulta suficiente para mejorar la generalización.

Finalmente, como se muestra en la Figura 7.30, este modelo permite identificar las variables más relevantes, lo que confirma las hipótesis planteadas sobre la importancia de las variables predictoras, tal y como se describe en la Sección 4.4.

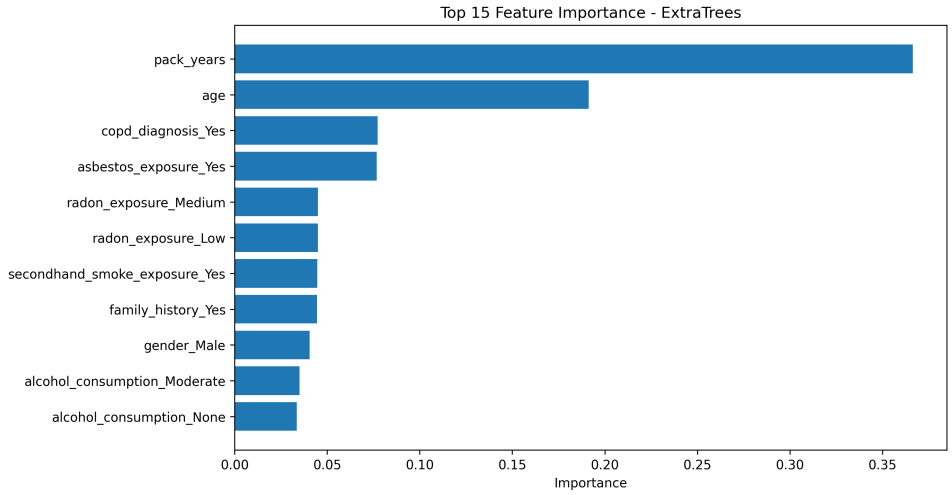


Figura 7.30: Variables más relevantes según el modelo Extra Trees

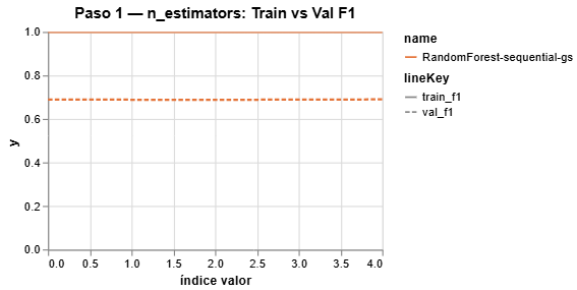
Random Forest

Los modelos Random Forest y Extra Trees comparten los mismos hiperparámetros y valores evaluados. Por este motivo, no se describen nuevamente en esta sección para evitar redundancias. Los valores considerados pueden consultarse en la Tabla 7.6.

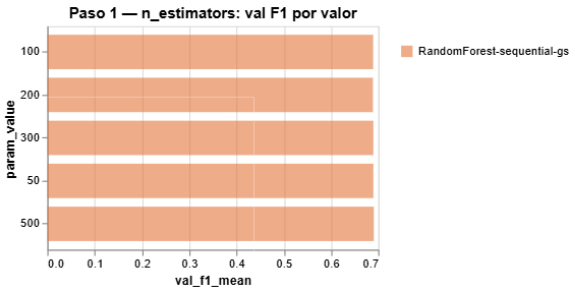
n_estimators

Como se observa en la Figura 7.31, para el hiperparámetro n_estimators los resultados obtenidos son prácticamente iguales entre los distintos valores evaluados, con variaciones mínimas entre ellos. En este caso, el mejor rendimiento en validación se obtiene con n_estimators = 500.

Al tratarse del primer hiperparámetro evaluado, todavía no se han ajustado parámetros orientados a controlar la complejidad del modelo. Por ello, se aprecia una tendencia notable al sobreajuste, ya que la F1 en entrenamiento es superior a la obtenida en validación. Los siguientes hiperparámetros nos permitirán limitar el sobreajuste.



(a) Comparación de la F1 en entrenamiento y validación



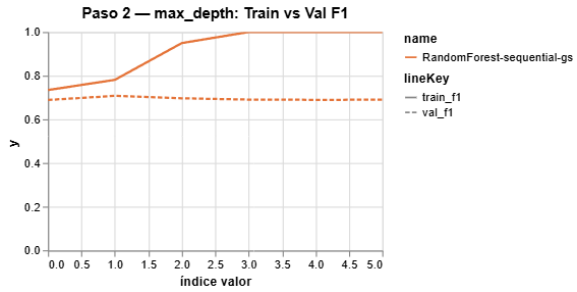
(b) Media de la F1 para cada valor del hiperparámetro

Figura 7.31: Análisis del hiperparámetro n_estimators en Random Forest

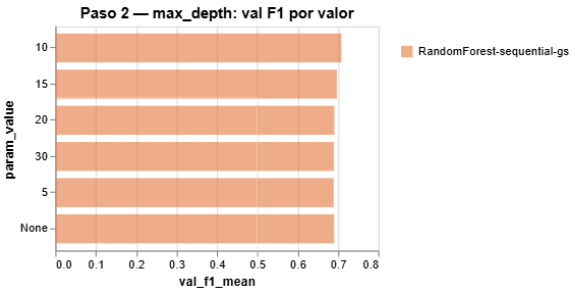
max_depth

Como se observa en la Figura 7.32, los valores evaluados presentan una F1 de validación similar. Sin embargo, a medida que aumenta la profundidad, el rendimiento del modelo disminuye ligeramente. Del mismo modo, profundidades inferiores a 10, como max_depth = 5, ofrecen resultados peores.

Además, en la comparación entre entrenamiento y validación se observa una separación creciente entre ambas curvas conforme aumenta la profundidad, lo que indica una mayor tendencia al sobreajuste. Finalmente, se selecciona max_depth = 10, al ofrecer el mejor rendimiento, además de contribuir a reducir el sobreajuste del modelo.



(a) Comparación de la F1 en entrenamiento y validación



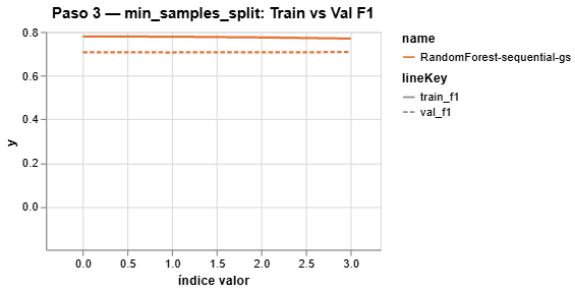
(b) Media de la F1 para cada valor del hiperparámetro

Figura 7.32: Análisis del hiperparámetro max_depth en Random Forest

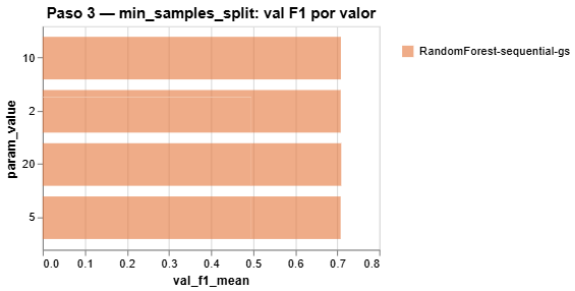
min_samples_split

Como se observa en la Figura 7.33, los resultados obtenidos son prácticamente idénticos para los distintos valores evaluados. No obstante, se selecciona min_samples_split = 20, al obtener un rendimiento ligeramente superior al resto.

Además, se aprecia una reducción clara del sobreajuste respecto al paso anterior, ya que la diferencia entre las curvas de entrenamiento y validación disminuye. Esto indica que este hiperparámetro contribuye a limitar la complejidad del modelo.



(a) Comparación de la F1 en entrenamiento y validación



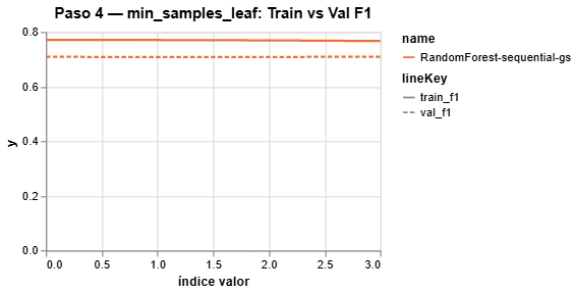
(b) Media de la F1 para cada valor del hiperparámetro

Figura 7.33: Análisis del hiperparámetro min_samples_split en Random Forest

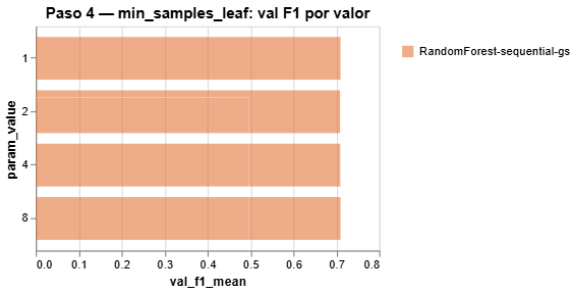
min_samples_leaf

Respecto al hiperparámetro min_samples_leaf, en la Figura 7.34 se observa que los resultados apenas difieren en unas pocas diezmilésimas. Esto indica que su influencia sobre el rendimiento del modelo es bastante limitada. Sin embargo, respecto al paso anterior se obtiene una mejora muy ligera de la F1, por lo que se selecciona min_samples_leaf = 8.

Además, no se aprecia una reducción de la distancia entre las curvas de entrenamiento y validación en comparación con los hiperparámetros evaluados anteriormente.



(a) Comparación de la F1 en entrenamiento y validación



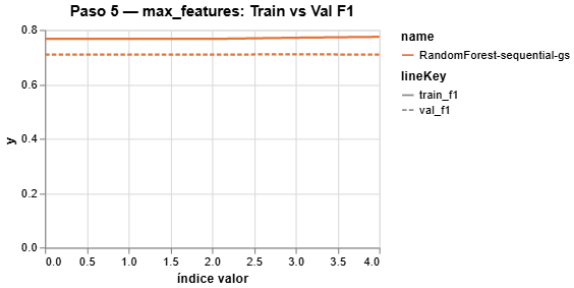
(b) Media de la F1 para cada valor del hiperparámetro

Figura 7.34: Análisis del hiperparámetro min_samples_leaf en Random Forest

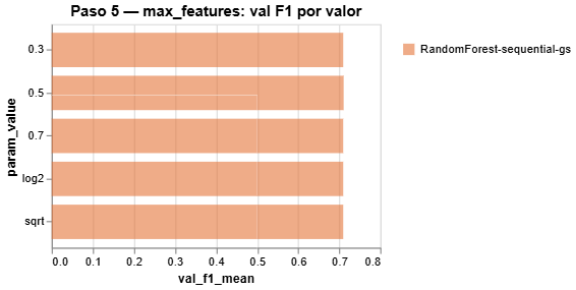
max_features

En la Figura 7.35 se observa que los resultados obtenidos para max_features son muy similares. No obstante, con max_features = 0.5 se obtiene una ligera mejora, aproximadamente de una milésima, por lo que se selecciona este valor. Esto indica que el modelo obtiene un rendimiento ligeramente superior al considerar la mitad de las variables predictoras en cada división.

Por otro lado, max_features apenas influye en la reducción del sobreajuste, ya que la distancia entre las curvas de entrenamiento y validación es muy similar a la observada en el hiperparámetro analizado previamente.



(a) Comparación de la F1 en entrenamiento y validación



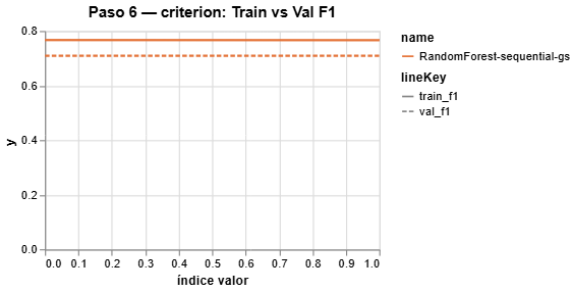
(b) Media de la F1 para cada valor del hiperparámetro

Figura 7.35: Análisis del hiperparámetro max_features en Random Forest

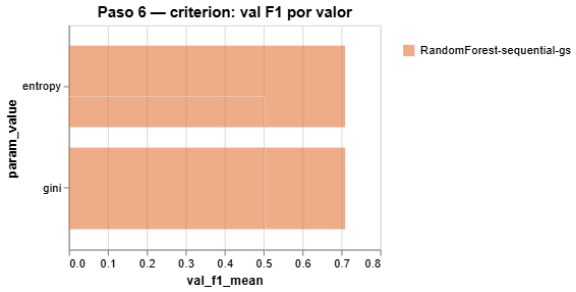
criterion

Como se observa en la Figura 7.36, dada la complejidad del problema y la limitada influencia de este hiperparámetro, su modificación no permite mejorar el rendimiento del modelo. Únicamente se aprecia un pequeño descenso de unas milésimas en la métrica F1. Por ello, se mantiene criterion = gini, al ser el valor que ofrece el mejor resultado.

Además, al tratarse de un hiperparámetro relacionado con la forma de medir la homogeneidad de los nodos, y no con una restricción directa sobre la complejidad del modelo, no contribuye de manera apreciable a reducir el sobreajuste.



(a) Comparación de la F1 en entrenamiento y validación



(b) Media de la F1 para cada valor del hiperparámetro

Figura 7.36: Análisis del hiperparámetro criterion en Random Forest

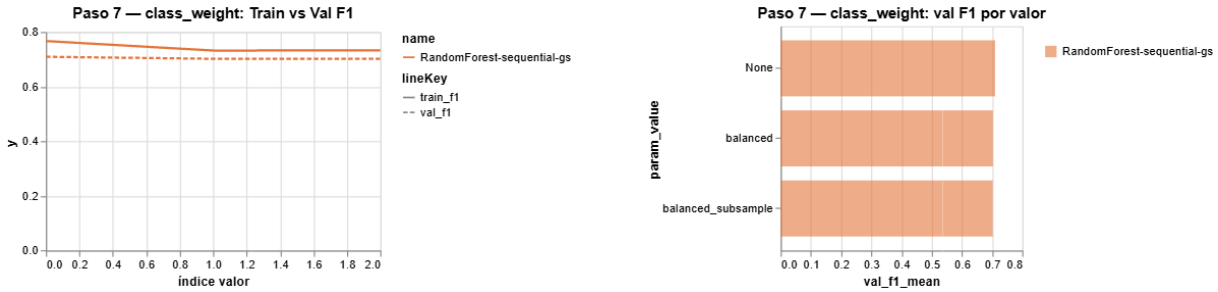
class_weight

Respecto al hiperparámetro class_weight, en la Figura 7.37 se observa que la mejor configuración corresponde a class_weight = None, con una F1 de 0.7093, frente a 0.7019 con

balanced y 0.7014 con balanced_subsample. Aunque las diferencias no son muy elevadas, la opción de no ponderar las clases obtiene un rendimiento superior al resto.

Este resultado puede indicar que el desbalance presente en los datos no requiere una corrección adicional mediante pesos de clase para este modelo. Además, al tratarse del último hiperparámetro evaluado en la búsqueda secuencial, su efecto puede estar condicionado por los ajustes seleccionados en pasos anteriores.

Por otro lado, las curvas de entrenamiento y validación apenas varían respecto a las etapas previas, por lo que no se observan cambios relevantes en términos de sobreajuste.



(a) Comparación de la F1 en entrenamiento y validación

(b) Media de la F1 para cada valor del hiperparámetro

Figura 7.37: Análisis del hiperparámetro class_weight en Random Forest

Random Forest es una opción adecuada para este problema, ya que ofrece un comportamiento estable y obtiene buenos resultados en términos de F1. Además, las curvas de entrenamiento y validación apenas difieren tras el ajuste de hiperparámetros, por lo que no se observan indicios relevantes de sobreajuste.

Otra ventaja de este modelo es que permite analizar la importancia de las variables predictoras, facilitando una interpretación adicional de los resultados. En la Figura 7.38 se muestran las características más relevantes identificadas por el modelo. Estos resultados son coherentes con las hipótesis planteadas previamente sobre la importancia de determinadas variables predictoras, descritas en la Sección 4.4.

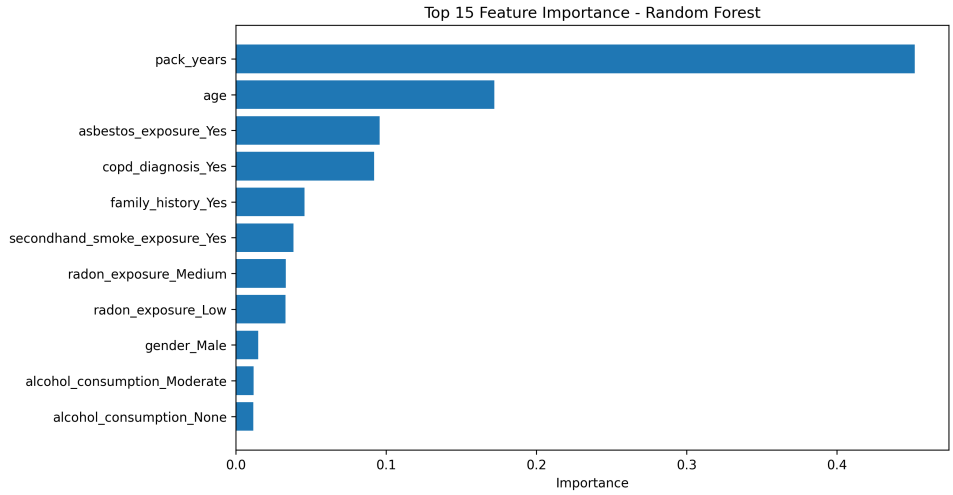


Figura 7.38: Variables más relevantes según el modelo Random Forest

Métodos boosting basados en árboles

AdaBoost

Los valores considerados para los diferentes hiperparámetros pueden consultarse en la Tabla 7.7.

Tabla 7.7: Hiperparámetros evaluados para el modelo AdaBoost

Hiperparámetro	Valores evaluados
n_estimators	50, 100, 200, 300, 500
learning_rate	0.001, 0.01, 0.05, 0.1, 0.5, 1.0

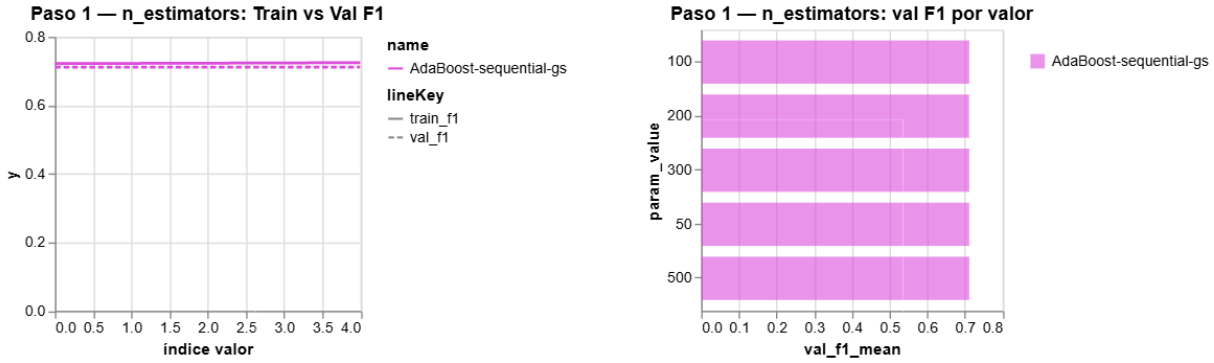
Además, para este modelo se parte del mejor árbol obtenido previamente, es decir, del árbol de decisión configurado con los hiperparámetros seleccionados en la subsección correspondiente a Decision Tree.

n_estimators

El hiperparámetro n_estimators define el número máximo de estimadores utilizados durante el proceso de boosting. Aunque AdaBoost puede detener el entrenamiento antes de alcanzar este límite si consigue un ajuste perfecto.

En la Figura 7.39 se observa que los distintos valores evaluados presentan resultados muy similares. Aunque con n_estimators = 100 ya se obtiene un buen rendimiento, el mejor resultado se alcanza con n_estimators = 500. Los valores intermedios presentan pequeñas reducciones en la F1, lo que sugiere que aumentar el número de estimadores no implica necesariamente una mejora constante del rendimiento.

Además, al partir de un árbol base que apenas presentaba sobreajuste, las curvas de entrenamiento y validación se mantienen próximas. Por tanto, no se observan indicios claros de sobreajuste en esta primera etapa.



(a) Comparación de la F1 en entrenamiento y validación

(b) Media de la F1 para cada valor del hiperparámetro

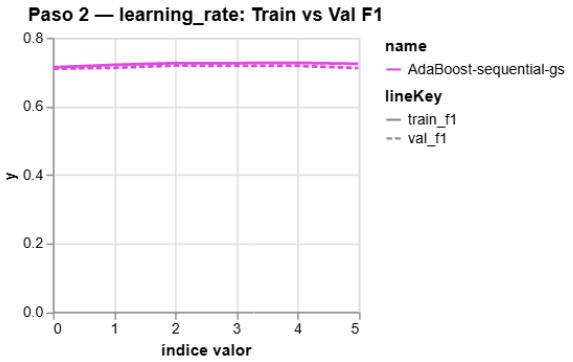
Figura 7.39: Análisis del hiperparámetro n_estimators en AdaBoost

learning_rate

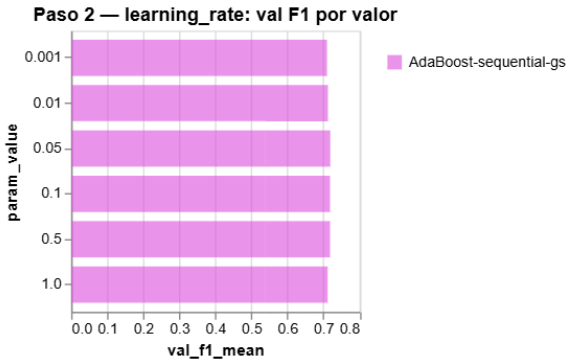
El hiperparámetro learning_rate controla la influencia que tiene cada clasificador en cada iteración del proceso de boosting. Un valor alto hace que cada nuevo estimador tenga más peso en la predicción final.

En la Figura 7.40 se observa que los distintos valores evaluados presentan resultados similares. Sin embargo, existe una ligera tendencia a obtener peores valores de F1 cuando la tasa de aprendizaje es demasiado baja o demasiado alta, mientras que los valores intermedios ofrecen un mejor rendimiento. Seleccionamos learning_rate = 0.05.

Además, este hiperparámetro no desestabiliza el entrenamiento, ya que las curvas de entrenamiento y validación se mantienen muy próximas. Por tanto, no se observan indicios claros de sobreajuste.



(a) Comparación de la F1 en entrenamiento y validación



(b) Media de la F1 para cada valor del hiperparámetro

Figura 7.40: Análisis del hiperparámetro learning_rate en AdaBoost

En conjunto, AdaBoost mejora los resultados obtenidos por el árbol de decisión, pasando de un F1 en validación de 0.7075 a 0.7189. Además, el modelo presenta un comportamiento estable y no muestra indicios claros de sobreajuste.

Histogram-Based Gradient Boosting

Los valores considerados para los diferentes hiperparámetros pueden consultarse en la Tabla 7.8.

Tabla 7.8: Hiperparámetros evaluados para el modelo HistGradientBoosting

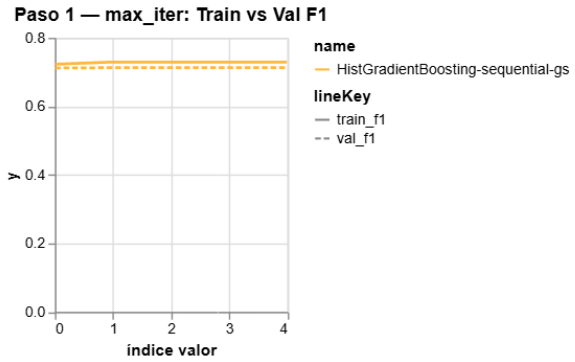
Hiperparámetro	Valores evaluados
max_iter	50, 100, 200, 300, 500
learning_rate	0.01, 0.05, 0.1, 0.2, 0.3
max_depth	3, 5, 7, 10, None
min_samples_leaf	1, 2, 4, 8
max_leaf_nodes	15, 31, 63
max_bins	64, 128, 255

max_iter

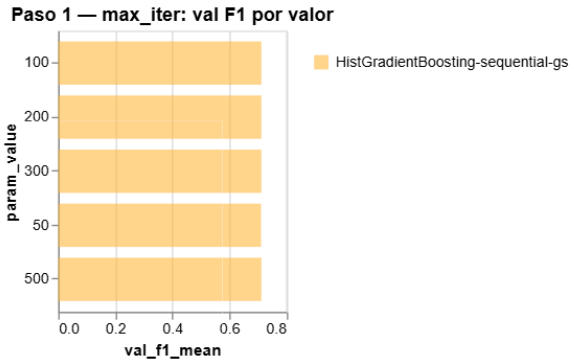
El hiperparámetro max_iter define el número máximo de árboles que pueden añadirse al modelo durante el proceso de boosting.

Como se observa en la Figura 7.41, los distintos valores evaluados presentan resultados muy similares. El mejor rendimiento se obtiene con max_iter = 100, mientras que valores superiores no modifican la F1. Esto sugiere que, a partir de este punto, añadir más árboles no aporta mejoras relevantes al rendimiento del modelo.

Además, no se observan indicios claros de sobreajuste, ya que las curvas de entrenamiento y validación se mantienen muy próximas entre sí.



(a) Comparación de la F1 en entrenamiento y validación



(b) Media de la F1 para cada valor del hiperparámetro

Figura 7.41: Análisis del hiperparámetro max_iter en HistGradientBoost

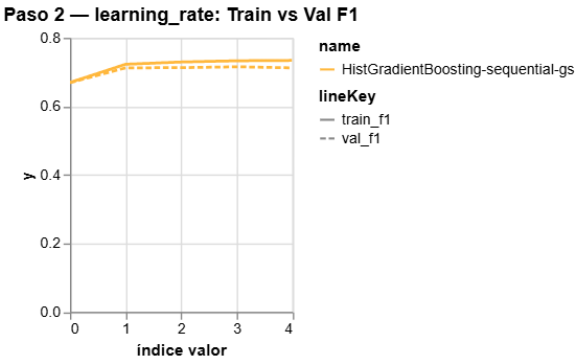
learning_rate

El hiperparámetro learning_rate actúa como un factor multiplicativo aplicado a los valores de las hojas de cada árbol. Esto influye en la intensidad con la que cada nuevo estimador corrige los errores cometidos previamente: cuanto mayor es el learning_rate, mayor es la corrección aplicada por cada árbol.

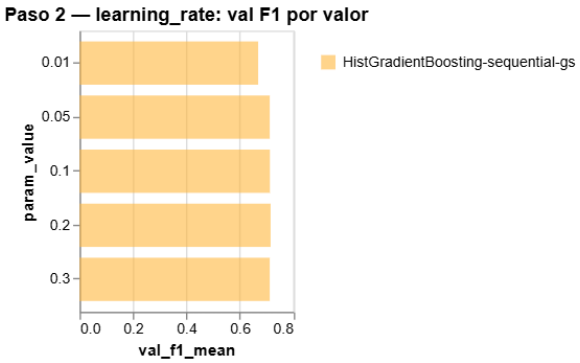
Como se observa en la Figura 7.42, el valor más bajo de learning_rate obtiene el peor resultado, lo que puede deberse a que el aprendizaje del modelo es demasiado lento. A medida que aumenta la tasa de aprendizaje, la F1 mejora progresivamente hasta alcanzar su mejor valor con learning_rate = 0.2.

A partir de este punto, el rendimiento vuelve a disminuir ligeramente, lo que sugiere que correcciones demasiado grandes pueden afectar negativamente a la capacidad de generalización del modelo.

Respecto al sobreajuste, no se aprecian indicios claros debido a la proximidad entre las curvas de entrenamiento y validación.



(a) Comparación de la F1 en entrenamiento y validación



(b) Media de la F1 para cada valor del hiperparámetro

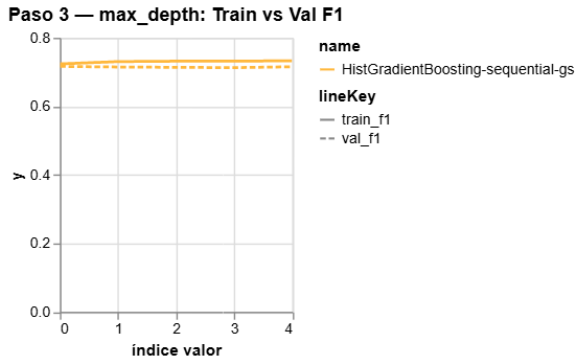
Figura 7.42: Análisis del hiperparámetro learning_rate en HistGradientBoost

max_depth

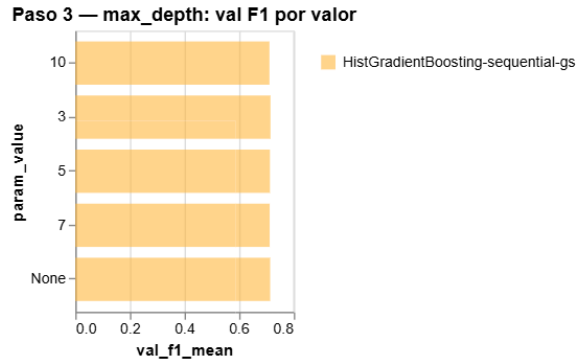
El hiperparámetro max_depth se encarga de definir la profundidad máxima de cada árbol.

Como se observa en la Figura 7.43, los resultados obtenidos son muy similares entre los distintos valores evaluados. No obstante, se aprecia una ligera tendencia a reducir la F1 conforme aumenta la profundidad. El mejor valor encontrado es max_depth = 3. Esto podría sugerir que árboles más profundos tienden a ajustarse más a los datos de entrenamiento y a generalizar ligeramente peor, aunque esta interpretación debe tomarse con cierta cautela, ya que las diferencias son de apenas unas pocas milésimas.

En esta etapa tampoco se aprecian indicios claros de sobreajuste, ya que las curvas de entrenamiento y validación se mantienen próximas entre sí.



(a) Comparación de la F1 en entrenamiento y validación



(b) Media de la F1 para cada valor del hiperparámetro

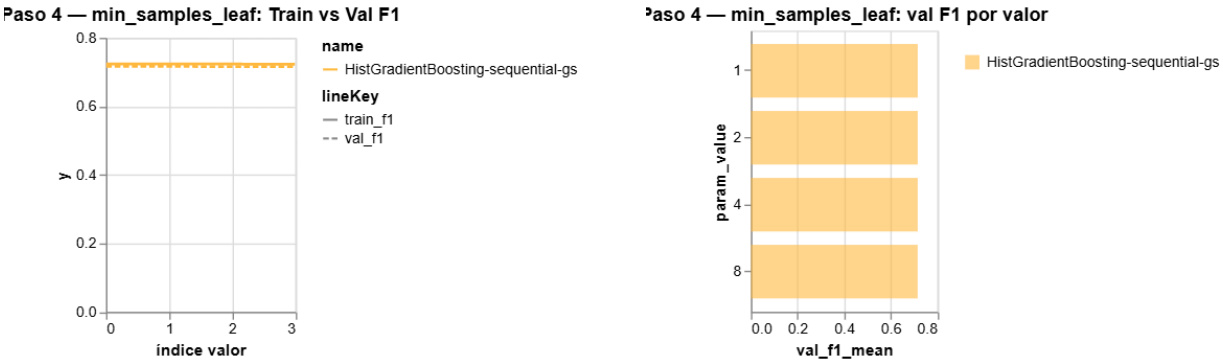
Figura 7.43: Análisis del hiperparámetro max_depth en HistGradientBoost

min_samples_leaf

Con el hiperparámetro min_samples_leaf se define el número mínimo de instancias que debe contener cada hoja. Al igual que max_depth, este parámetro permite controlar la complejidad del modelo y puede ayudar a reducir el sobreajuste.

Como se observa en la Figura 7.44, los resultados obtenidos son prácticamente idénticos entre los distintos valores evaluados. No obstante, se aprecian pequeñas reducciones residuales en la F1 conforme aumenta el número mínimo de instancias por hoja. Dado que las diferencias son muy reducidas, no puede afirmarse que un mayor valor de min_samples_leaf empeore de forma significativa el rendimiento del modelo. Finalmente seleccionamos min_samples_leaf=1.

El modelo sigue mostrando un comportamiento estable, sin indicios notorios de sobreajuste.



(a) Comparación de la F1 en entrenamiento y validación

(b) Media de la F1 para cada valor del hiperparámetro

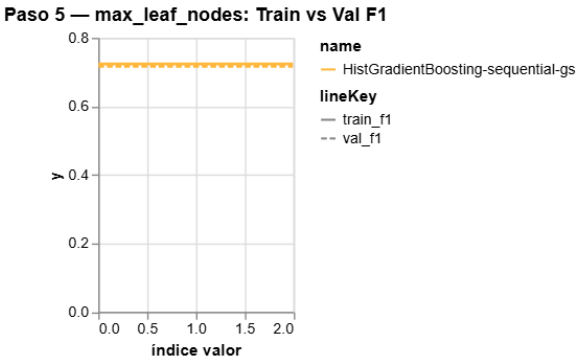
Figura 7.44: Análisis del hiperparámetro min_samples_leaf en HistGradientBoost

max_leaf_nodes

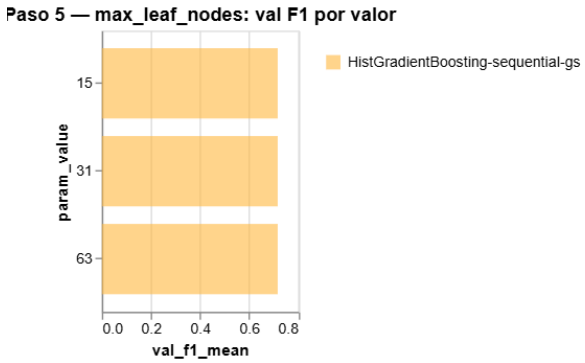
Con el hiperparámetro max_leaf_nodes se define el número máximo de hojas que puede tener cada árbol.

Como se observa en la Figura 7.45, los resultados obtenidos son prácticamente idénticos para todos los valores evaluados. Esto indica que, una vez fijados los hiperparámetros anteriores, este parámetro no introduce mejoras respecto al rendimiento obtenido en el paso previo. Seleccionamos max_leaf_nodes=15.

Del mismo modo, al no producir cambios relevantes en el comportamiento del modelo, las curvas de entrenamiento y validación se mantienen próximas, sin variaciones notorias ni indicios claros de sobreajuste.



(a) Comparación de la F1 en entrenamiento y validación



(b) Media de la F1 para cada valor del hiperparámetro

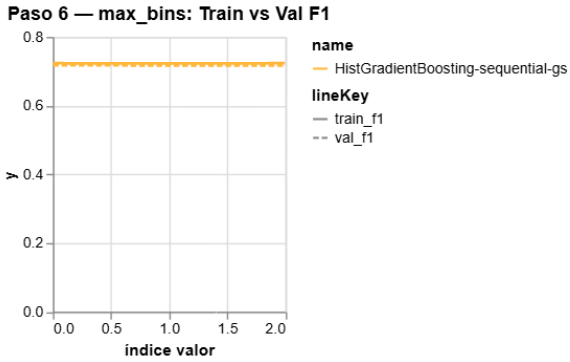
Figura 7.45: Análisis del hiperparámetro max_leaf_nodes en HistGradientBoost

max_bins

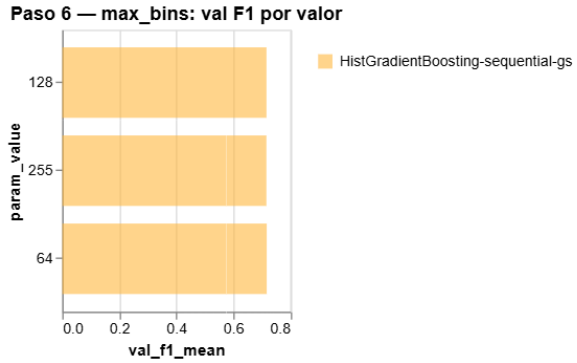
El hiperparámetro max_bins permite definir el número máximo de intervalos en los que se dividirán las variables numéricas antes del entrenamiento del modelo.

Como se observa en la Figura 7.45, los resultados obtenidos son muy similares, con diferencias de apenas unas pocas diezmilésimas entre los valores evaluados. Al tratarse del último hiperparámetro analizado, su efecto puede estar condicionado por las decisiones tomadas en las etapas anteriores del ajuste secuencial. Finalmente, se selecciona max_bins = 64, al obtener una F1 ligeramente superior.

Además, el modelo no muestra indicios claros de sobreajuste, ya que las curvas de entrenamiento y validación se mantienen próximas durante las distintas etapas del ajuste.



(a) Comparación de la F1 en entrenamiento y validación



(b) Media de la F1 para cada valor del hiperparámetro

Figura 7.46: Análisis del hiperparámetro max_bins en HistGradientBoost

Este modelo representa una solución rápida, eficiente y estable para el problema tratado.

Obtiene una buena F1 y mantiene un comportamiento equilibrado entre entrenamiento y validación, sin mostrar indicios claros de sobreajuste.

XGBoost

Los valores considerados para los diferentes hiperparámetros pueden consultarse en la Tabla 7.9.

Tabla 7.9: Hiperparámetros evaluados para el modelo XGBoost

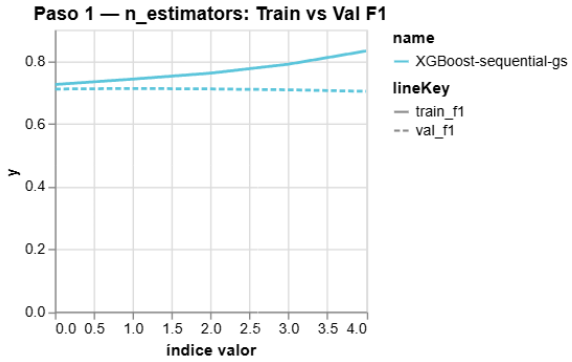
Hiperparámetro	Valores evaluados
n_estimators	10, 25, 50, 100, 200
max_depth	3, 5, 10, 15, 20, None
learning_rate	0.01, 0.1, 0.2, 0.3
subsample	0.5, 0.6, 0.7, 0.8, 0.9, 1.0
colsample_bytree	0.5, 0.6, 0.7, 0.8, 0.9, 1.0

n_estimators

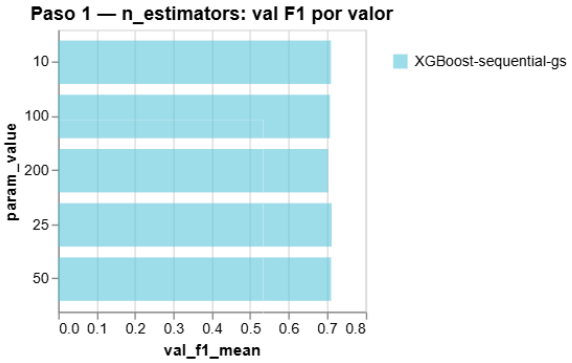
El hiperparámetro n_estimators define el número máximo de estimadores utilizados durante el proceso de boosting.

En la Figura 7.47 se observa que los distintos valores evaluados presentan resultados muy similares. El mejor rendimiento se obtiene con n_estimators = 25. A partir de este valor, se aprecia una ligera tendencia descendente, del orden de unas pocas milésimas, en la métrica F1 conforme aumenta el número de estimadores.

Además, se observa que, al aumentar el número de estimadores, también se incrementa ligeramente la separación entre las curvas de entrenamiento y validación. Aunque esta diferencia no es muy elevada y debe interpretarse con cautela, podría indicar una leve tendencia al sobreajuste en configuraciones con un mayor número de árboles.



(a) Comparación de la F1 en entrenamiento y validación



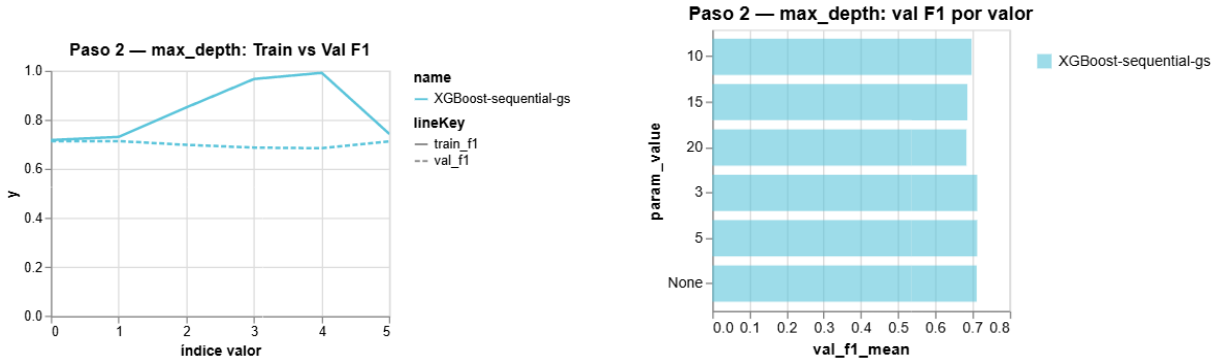
(b) Media de la F1 para cada valor del hiperparámetro

Figura 7.47: Análisis del hiperparámetro n_estimators en XGBoost

max_depth

En la Figura 7.48 se observa que los mejores valores de F1 se obtienen con las profundidades máximas más bajas. A medida que aumenta la profundidad de los árboles, la F1 tiende a disminuir, lo que sugiere que árboles demasiado profundos no mejoran la capacidad de generalización del modelo. Seleccionamos max_depth=3.

Del mismo modo, con los valores más bajos de max_depth no se observan indicios claros de sobreajuste. Sin embargo, conforme aumenta la profundidad, las curvas de entrenamiento y validación se separan de manera notable, indicando una mayor tendencia al sobreajuste.



(a) Comparación de la F1 en entrenamiento y validación

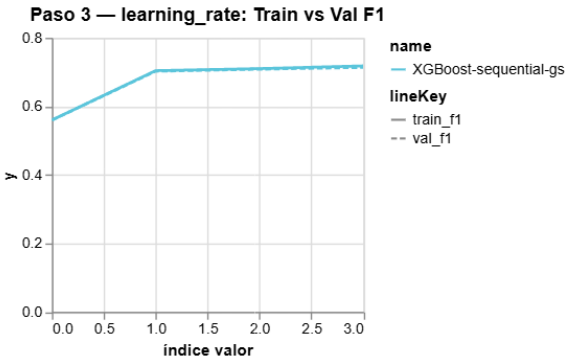
(b) Media de la F1 para cada valor del hiperparámetro

Figura 7.48: Análisis del hiperparámetro max_depth en XGBoost

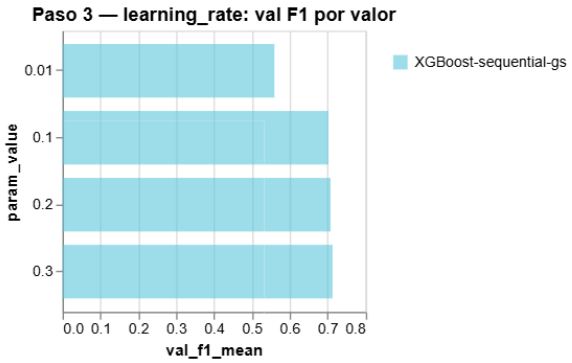
learning_rate

Como se observa en la Figura 7.49, el valor más pequeño de learning_rate obtiene el peor resultado. Conforme aumenta la tasa de aprendizaje, la F1 mejora progresivamente, alcanzando el mejor valor con learning_rate = 0.3. Sin embargo, este ajuste no consigue superar la F1 obtenida en el paso anterior.

En esta etapa, gracias a que la profundidad máxima fue fijada previamente, las curvas de entrenamiento y validación se mantienen próximas, por lo que no se observan indicios claros de sobreajuste.



(a) Comparación de la F1 en entrenamiento y validación



(b) Media de la F1 para cada valor del hiperparámetro

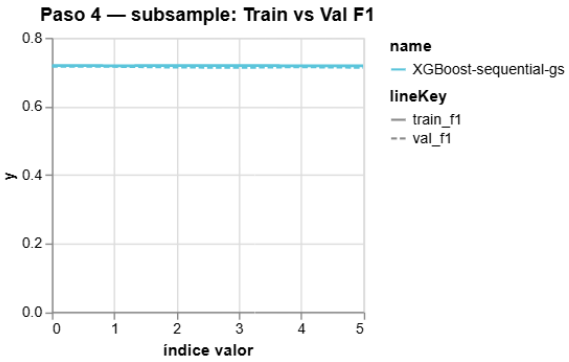
Figura 7.49: Análisis del hiperparámetro learning_rate en XGBoost

subsample

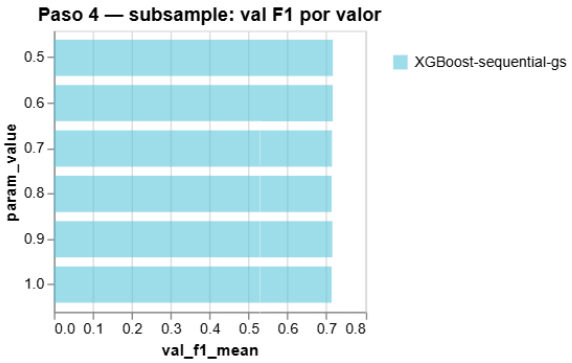
El hiperparámetro subsample permite definir la proporción de instancias que utiliza cada árbol durante el entrenamiento.

Como se observa en la Figura 7.50, los resultados obtenidos son bastante similares. El mejor rendimiento se alcanza con subsample = 0.5. A medida que aumenta la proporción de instancias utilizadas, la F1 disminuye ligeramente, con una diferencia de unas milésimas entre el mejor y el peor resultado.

En esta etapa tampoco se aprecian indicios claros de sobreajuste, ya que las curvas de entrenamiento y validación se mantienen próximas entre sí.



(a) Comparación de la F1 en entrenamiento y validación



(b) Media de la F1 para cada valor del hiperparámetro

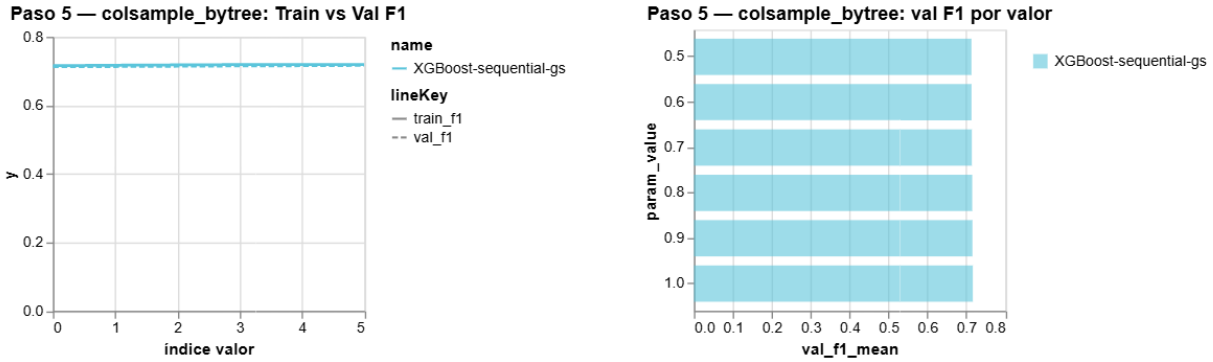
Figura 7.50: Análisis del hiperparámetro subsample en XGBoost

colsample_bytree

El hiperparámetro colsample_bytree permite definir la proporción de variables predictoras que utiliza cada árbol durante el entrenamiento.

Como se observa en la Figura 7.51, los resultados obtenidos son bastante similares entre los distintos valores evaluados. En este caso se aprecia que utilizar una fracción de las variables predictoras no mejora el rendimiento frente al uso de todas ellas. De hecho, la F1 tiende a reducirse ligeramente conforme disminuye la proporción de variables utilizadas, aunque las diferencias son pequeñas. Esto podría sugerir que, para este problema, cada árbol se beneficia de disponer de toda la información predictiva disponible. Seleccionamos colsample_bytree= 1.

En esta etapa final tampoco se aprecia un sobreajuste notorio, ya que las curvas de entrenamiento y validación se mantienen próximas.



(a) Comparación de la F1 en entrenamiento y validación

(b) Media de la F1 para cada valor del hiperparámetro

Figura 7.51: Análisis del hiperparámetro colsample_bytree en XGBoost

Finalmente, este modelo representa una solución fiable y robusta, ya que no presenta indicios claros de sobreajuste y obtiene un buen rendimiento en términos de F1. Además, su rapidez y eficiencia durante el entrenamiento lo convierten en una alternativa muy adecuada para el problema tratado.

CatBoost

Los valores considerados para los diferentes hiperparámetros pueden consultarse en la Tabla 7.10.

Tabla 7.10: Hiperparámetros evaluados para el modelo CatBoost

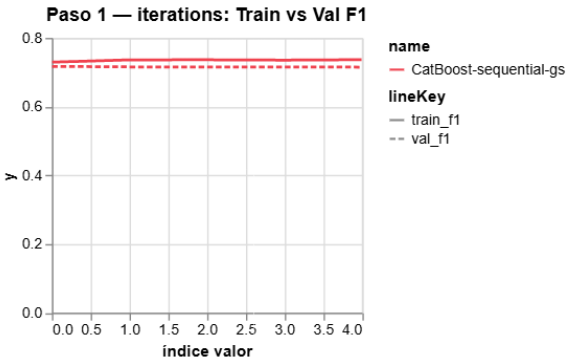
Hiperparámetro	Valores evaluados
iterations	100, 200, 500, 800, 1000
depth	4, 6, 8, 10
learning_rate	0.01, 0.03, 0.05, 0.1, 0.2
l2_leaf_reg	1, 3, 5, 7, 9
border_count	32, 64, 128, 254
bagging_temperature	0, 0.5, 1, 2

iterations

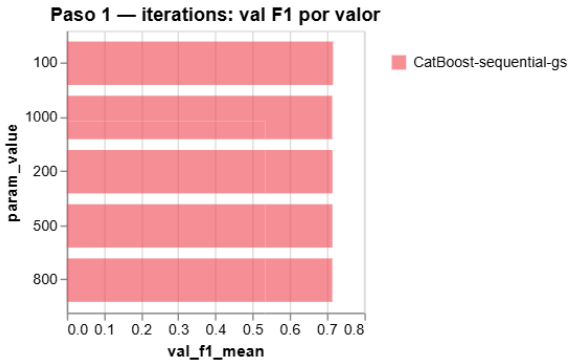
El hiperparámetro iterations permite definir el número de árboles creados durante el proceso de boosting.

Como se observa en la Figura 7.52, los resultados obtenidos son bastante similares entre los distintos valores evaluados. En este caso, el mejor rendimiento se alcanza con iterations = 100. A partir de este valor, existe una ligera disminución de la F1 conforme aumenta el número de iteraciones, aunque las diferencias son de apenas unas pocas milésimas. También cabe destacar el buen rendimiento del modelo en esta primera etapa, ya que obtiene una F1 superior a la alcanzada por otros modelos tras completar todo su proceso de ajuste.

Además, no se aprecia un sobreajuste notorio, ya que las curvas de entrenamiento y validación se mantienen próximas entre sí.



(a) Comparación de la F1 en entrenamiento y validación



(b) Media de la F1 para cada valor del hiperparámetro

Figura 7.52: Análisis del hiperparámetro iterations en CatBoost

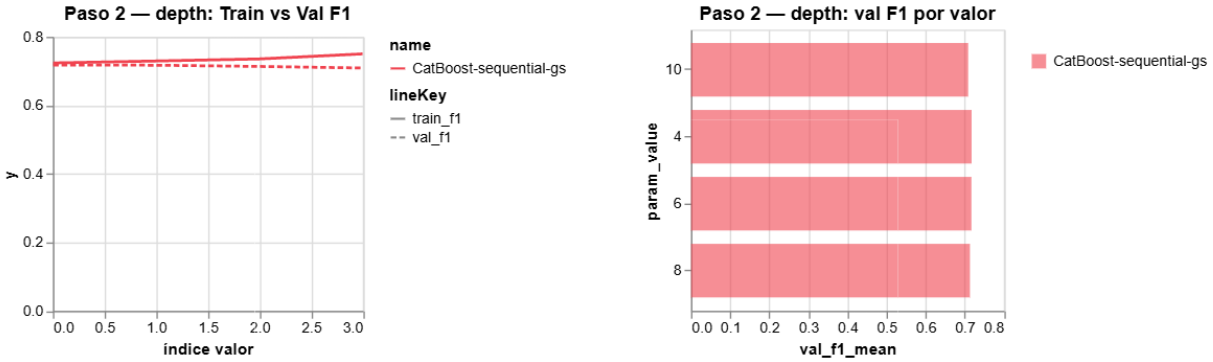
depth

El hiperparámetro depth permite fijar la profundidad máxima de cada árbol.

Como se observa en la Figura 7.53, los resultados obtenidos son bastante similares entre los distintos valores evaluados. El mejor rendimiento se alcanza con depth = 4 y, al igual que

ocurría con los modelos anteriores, a medida que aumenta la profundidad de los árboles la F1 disminuye ligeramente.

Respecto a las curvas de entrenamiento y validación, la distancia entre ellas es reducida, por lo que no hay una tendencia al sobreajuste en esta etapa.



(a) Comparación de la F1 en entrenamiento y validación

(b) Media de la F1 para cada valor del hiperparámetro

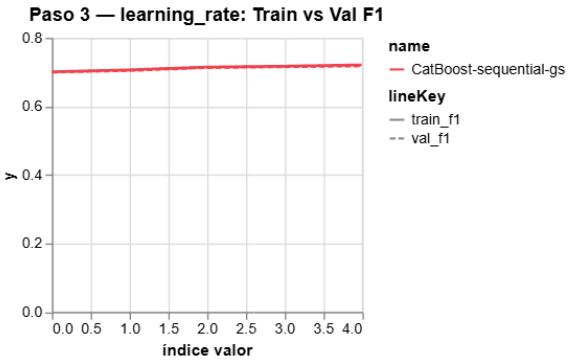
Figura 7.53: Análisis del hiperparámetro depth en CatBoost

learning_rate

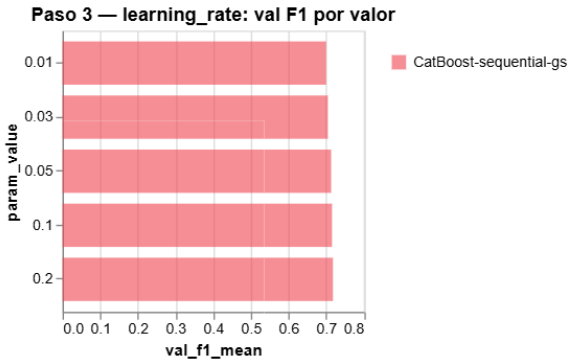
El hiperparámetro learning_rate permite establecer la intensidad con la que cada nuevo árbol corrige los errores cometidos por los árboles anteriores.

Como se observa en la Figura 7.54, existe una tendencia creciente en la F1 conforme aumenta el valor de learning_rate. El mejor resultado se obtiene con learning_rate = 0.2, lo que sugiere que valores demasiado bajos producen un aprendizaje más lento y no permiten mejorar el rendimiento del modelo en este caso.

Tras fijar la profundidad en el paso anterior, en esta etapa no se aprecian diferencias relevantes entre las curvas de entrenamiento y validación, por lo que no se observan indicios claros de sobreajuste.



(a) Comparación de la F1 en entrenamiento y validación



(b) Media de la F1 para cada valor del hiperparámetro

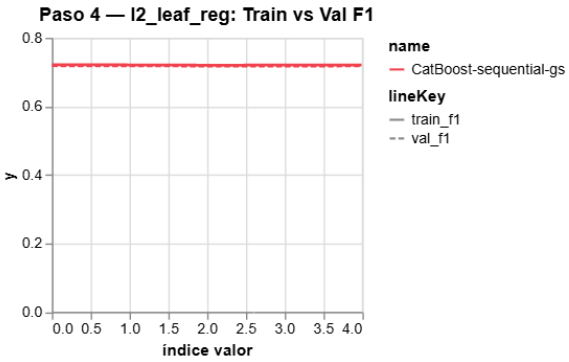
Figura 7.54: Análisis del hiperparámetro learning_rate en CatBoost

l2_leaf_reg

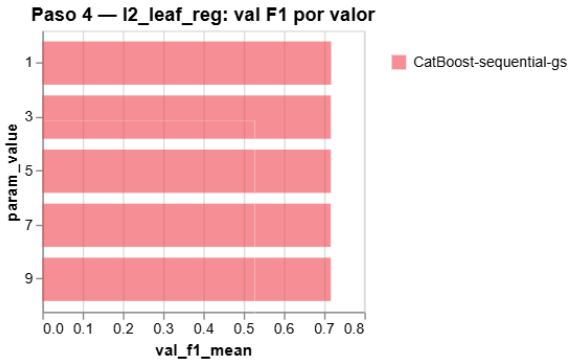
El hiperparámetro l2_leaf_reg establece una penalización de regularización que permite controlar la complejidad del modelo y reducir el riesgo de sobreajuste.

Como se observa en la Figura 7.55, los distintos valores evaluados producen prácticamente el mismo rendimiento. En este caso, no se aprecia una tendencia clara de mejora o empeoramiento de la F1 al aumentar la regularización. El mejor resultado se obtiene con l2_leaf_reg = 1.

En esta etapa no se observan indicios claros de sobreajuste, ya que las curvas de entrenamiento y validación se mantienen prácticamente unidas.



(a) Comparación de la F1 en entrenamiento y validación



(b) Media de la F1 para cada valor del hiperparámetro

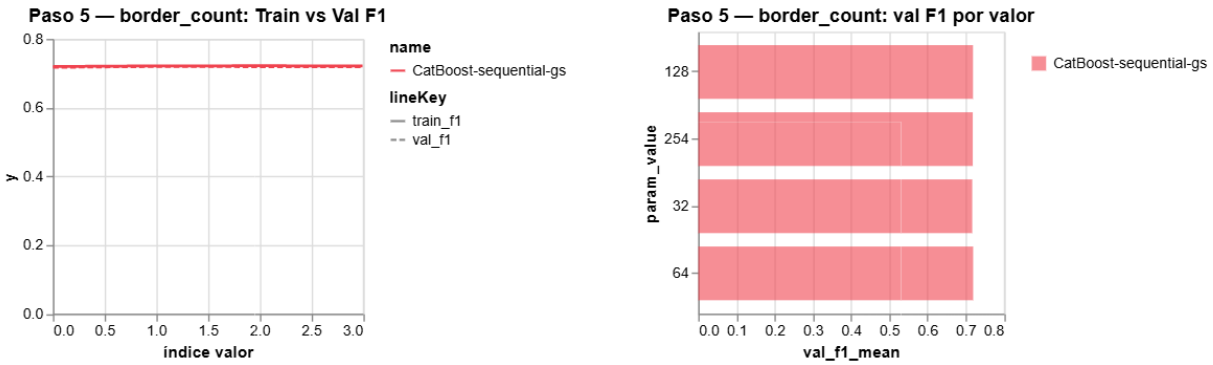
Figura 7.55: Análisis del hiperparámetro l2_leaf_reg en CatBoost

border_count

El hiperparámetro border_count tiene una función análoga a max_bins en HistGradientBoosting, ya que controla el número máximo de divisiones utilizadas para discretizar las variables numéricas continuas.

Como se observa en la Figura 7.56, los valores evaluados apenas producen diferencias en el rendimiento. El mejor resultado se obtiene con border_count = 64, mientras que valores inferiores y superiores presentan una F1 ligeramente menor, con variaciones de apenas unas pocas diezmilésimas.

El modelo mantiene un comportamiento estable, sin cambios apreciables en la distancia entre las curvas de entrenamiento y validación.



(a) Comparación de la F1 en entrenamiento y validación

(b) Media de la F1 para cada valor del hiperparámetro

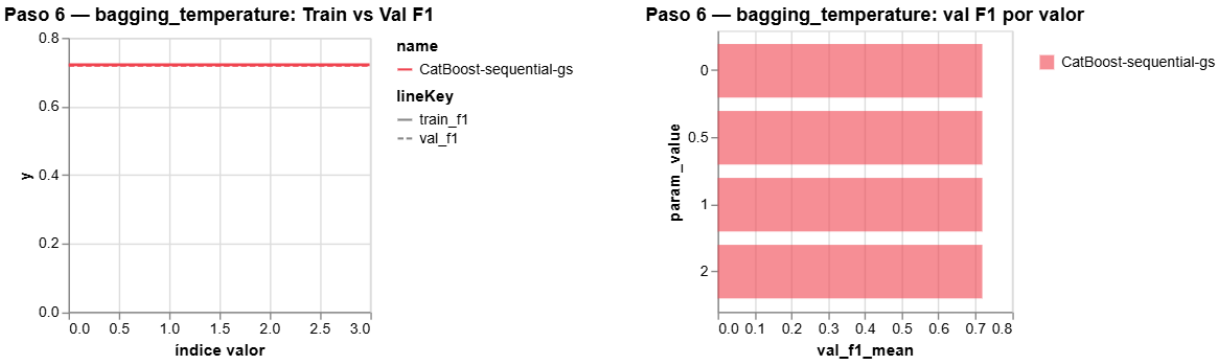
Figura 7.56: Análisis del hiperparámetro border_count en CatBoost

bagging_temperature

El hiperparámetro bagging_temperature controla el grado de aleatoriedad aplicado al muestreo de instancias durante el entrenamiento. Valores más altos introducen mayor aleatoriedad.

Como se observa en la Figura 7.57, los valores evaluados obtienen la misma F1 y no mejoran el mejor resultado alcanzado en el paso anterior. Esto podría indicar que este hiperparámetro no tiene una influencia relevante en la mejora del modelo, bien porque se trata de la última etapa del ajuste secuencial y su efecto queda condicionado por los hiperparámetros seleccionados previamente, o bien porque la aleatoriedad en el muestreo de instancias no afecta de forma significativa al rendimiento en este problema.

Al no producir cambios en el rendimiento, tampoco se observan variaciones entre las curvas de entrenamiento y validación.



(a) Comparación de la F1 en entrenamiento y validación

(b) Media de la F1 para cada valor del hiperparámetro

Figura 7.57: Análisis del hiperparámetro bagging_temperature en CatBoost

Finalmente, CatBoost se posiciona como el mejor modelo evaluado hasta el momento, al obtener el mayor valor de F1. Además, resulta especialmente adecuado para este problema debido a la elevada presencia de variables categóricas en el conjunto de datos, ya que el modelo está diseñado específicamente para trabajar con este tipo de variables sin requerir transformaciones complejas adicionales.

Asimismo, el modelo mantiene un comportamiento estable durante todo el proceso de ajuste, sin mostrar indicios claros de sobreajuste.

7.1.5. Métodos de combinación de modelos

Voting Classifier

Los valores considerados para los diferentes hiperparámetros pueden consultarse en la Tabla 7.11.

Tabla 7.11: Hiperparámetros evaluados para el modelo XGBoost

Hiperparámetro	Valores evaluados
voting	hard, soft

Para la construcción de este nuevo modelo se ha partido de los modelos basados en boosting, ya que fueron los que presentaron mejores resultados en las fases anteriores. En concreto, se han utilizado AdaBoost, CatBoost, XGBoost e HistGradientBoosting.

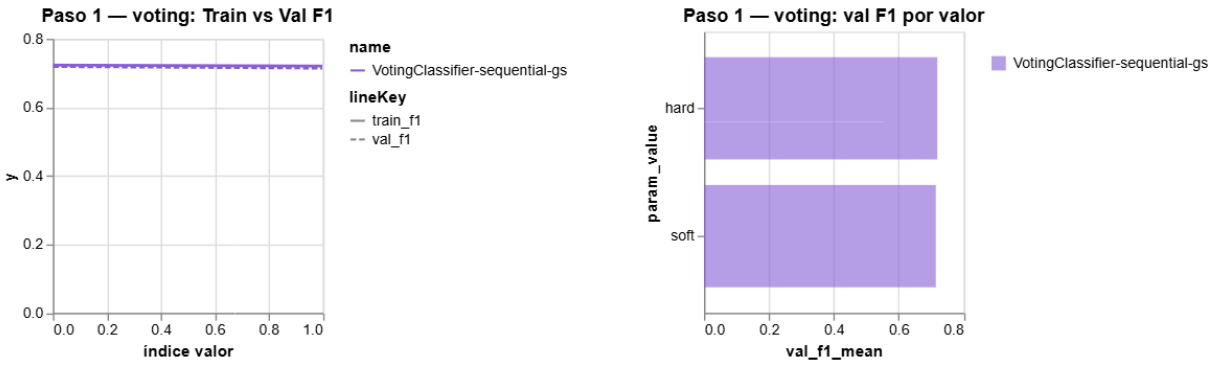
voting

El hiperparámetro voting permite definir la forma en que se combinan las predicciones de los modelos base. Con el método hard, cada modelo emite un voto y se selecciona la clase

mayoritaria. En cambio, con el método soft, se promedian las probabilidades predichas por los modelos y se escoge la clase con mayor probabilidad media.

Como se observa en la Figura 7.58, el método hard obtiene una mejora muy ligera, del orden de unas milésimas, frente a soft. Esto indica que, en este caso, el esquema de un modelo, un voto, ofrece un rendimiento ligeramente superior.

Además, no se aprecian indicios claros de sobreajuste, ya que se parte de modelos base robustos y estables durante sus respectivos procesos de entrenamiento.



(a) Comparación de la F1 en entrenamiento y validación

(b) Media de la F1 para cada valor del hiperparámetro

Figura 7.58: Análisis del hiperparámetro voting en Voting Classifier

Este modelo también presenta un buen rendimiento en términos de F1, además de no mostrar indicios evidentes de sobreajuste. Por tanto, se presenta como una alternativa recomendable una vez entrenados los modelos base que conforman el ensemble.

Stacking Classifier

Los valores considerados para los diferentes hiperparámetros pueden consultarse en la Tabla 7.12.

Tabla 7.12: Hiperparámetros evaluados para el modelo Stacking Classifier

Hiperparámetro	Valores evaluados
iterations	25, 50, 100, 200, 500
depth	4, 6, 8
learning_rate	0.03, 0.1, 0.2, 0.3, 0.4, 0.5
l2_leaf_reg	1, 3, 5
border_count	32, 64, 128

Para la construcción de este nuevo modelo se ha partido de los modelos basados en boosting, como AdaBoost, XGBoost e HistGradientBoosting, ya que fueron los que presentaron

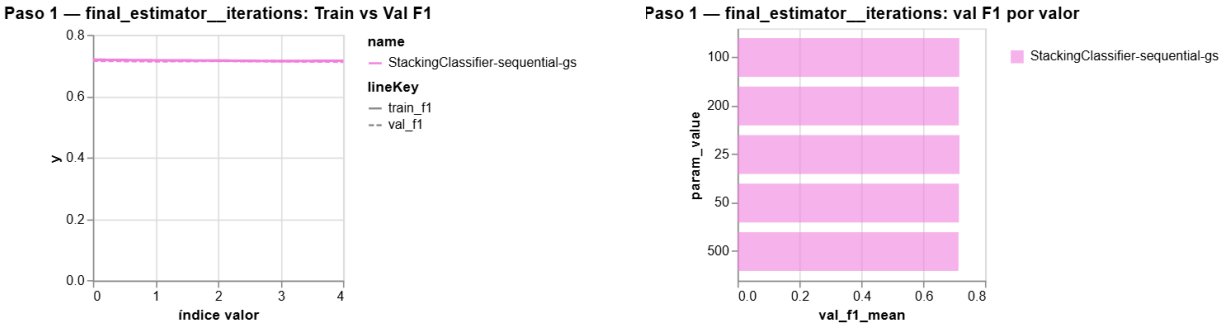
mejores resultados en las fases anteriores. Como metamodelo se ha utilizado CatBoost, al ser el modelo individual con mejor rendimiento.

Por tanto, los hiperparámetros evaluados corresponden al estimador final CatBoost. Se mantienen los mismos hiperparámetros analizados previamente para este modelo, excepto bagging_temperature, ya que en el ajuste anterior no tuvo una influencia relevante sobre el rendimiento y se ha omitido con el objetivo de reducir el coste computacional del entrenamiento.

iterations

Como se puede observar en la Figura 7.59, los resultados obtenidos para los distintos valores evaluados apenas presentan diferencias apreciables, variando únicamente en unas pocas diezmilésimas. El mejor rendimiento se alcanza con iterations = 25. A partir de este valor, la métrica F1 presenta pequeñas fluctuaciones, aunque no consigue superar dicho resultado.

Además, al partir de modelos base robustos y estables durante su entrenamiento, las curvas de entrenamiento y validación se mantienen prácticamente unidas, por lo que no se observan indicios claros de sobreajuste.



(a) Comparación de la F1 en entrenamiento y validación

(b) Media de la F1 para cada valor del hiperparámetro

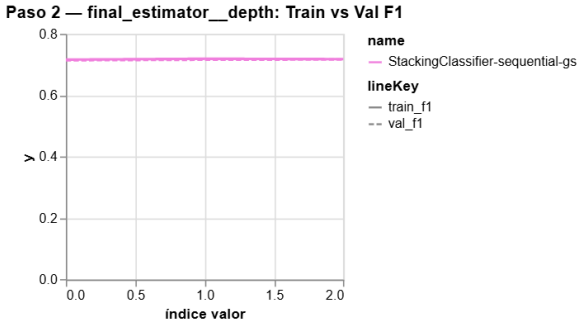
Figura 7.59: Análisis del hiperparámetro iterations en Stacking Classifier

depth

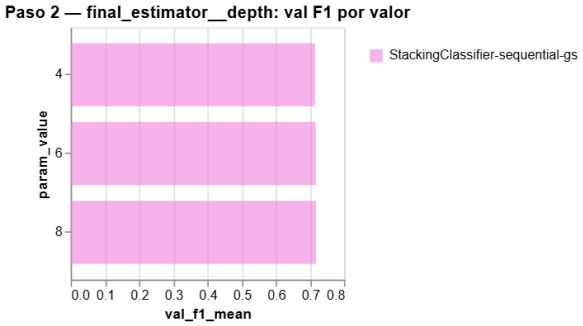
En la Figura 7.60 se puede observar que los resultados obtenidos para los distintos valores evaluados son muy cercanos, aunque varían en unas pocas milésimas. El mejor rendimiento se alcanza con depth = 8. En modelos anteriores se había observado que aumentar la profundidad podía incrementar el riesgo de sobreajuste y reducir el rendimiento. Sin embargo, en este caso ocurre lo contrario: una mayor profundidad mejora ligeramente la F1.

Este comportamiento podría sugerir que el metamodelo necesita una mayor capacidad de representación para aprender mejor las interacciones entre las predicciones generadas por los modelos base.

En esta etapa, la distancia entre las curvas de entrenamiento y validación no varía, por lo que no se observan indicios claros de sobreajuste por ahora.



(a) Comparación de la F1 en entrenamiento y validación



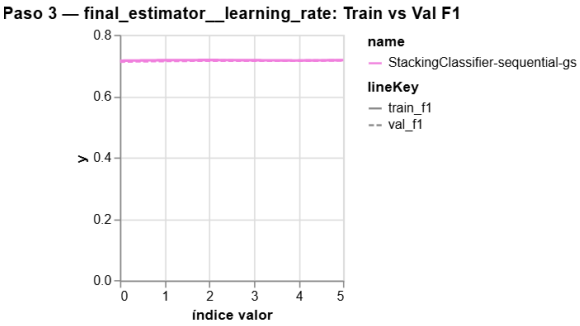
(b) Media de la F1 para cada valor del hiperparámetro

Figura 7.60: Análisis del hiperparámetro depth en Stacking Classifier

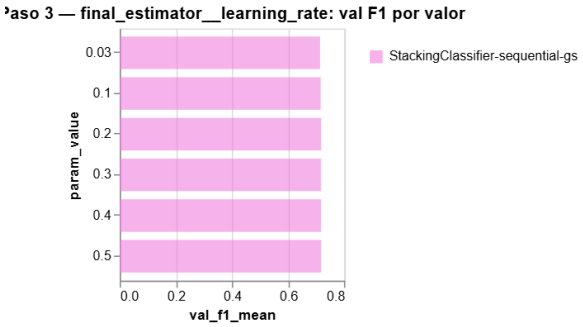
learning_rate

Como se observa en la Figura 7.61, los resultados obtenidos para los distintos valores evaluados presentan diferencias muy reducidas. El mejor rendimiento se alcanza con learning_rate = 0.5, que corresponde al valor más alto considerado. No obstante, la diferencia respecto al resto de configuraciones es de apenas unas pocas diezmilésimas y no se aprecia una tendencia clara de mejora conforme aumenta el valor de learning_rate.

Respecto a las curvas de entrenamiento y validación, estas se mantienen prácticamente unidas, sin cambios apreciables entre ellas. Por tanto, no se observan indicios claros de sobreajuste en esta etapa.



(a) Comparación de la F1 en entrenamiento y validación



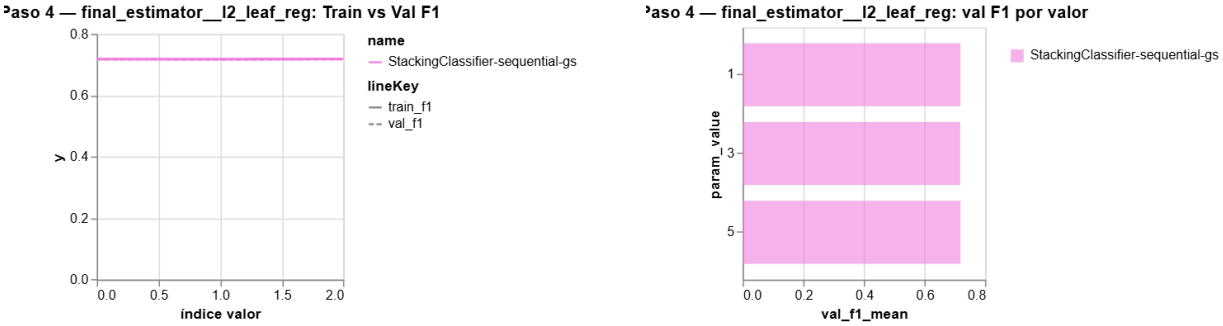
(b) Media de la F1 para cada valor del hiperparámetro

Figura 7.61: Análisis del hiperparámetro learning_rate en Stacking Classifier

l2_leaf_reg

En la Figura 7.62 se observa que los resultados obtenidos para los distintos valores evaluados son muy parejos. El mejor rendimiento se alcanza con l2_leaf_reg = 5. No obstante, la diferencia respecto al resto de valores es residual, de apenas unas pocas diezmilésimas, aunque permite mejorar ligeramente la F1 obtenida en la etapa anterior.

Respecto a las curvas de entrenamiento y validación, no se aprecian cambios relevantes respecto a la etapa anterior, manteniéndose ambas prácticamente unidas.



(a) Comparación de la F1 en entrenamiento y validación

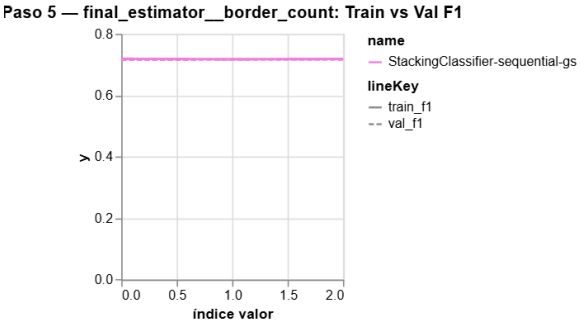
(b) Media de la F1 para cada valor del hiperparámetro

Figura 7.62: Análisis del hiperparámetro l2_leaf_reg en Stacking Classifier

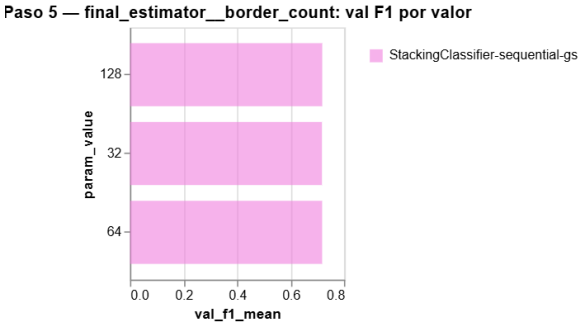
border_count

En este caso, el hiperparámetro border_count no permite mejorar la F1 obtenida en la etapa anterior. Esto puede deberse a que el modelo Stacking trabaja principalmente con las salidas generadas por los modelos base, y no directamente con las variables predictoras originales. Por tanto, la discretización de variables continuas tiene un impacto limitado en este metamodelo. Finalmente, se selecciona border_count = 128.

Como se observa en la Figura 7.63, los valores evaluados ofrecen resultados prácticamente equivalentes, aunque existen pequeñas variaciones entre ellos. Respecto a las curvas de entrenamiento y validación, ambas se encuentran prácticamente superpuestas, por lo que no se aprecian indicios de sobreajuste.



(a) Comparación de la F1 en entrenamiento y validación



(b) Media de la F1 para cada valor del hiperparámetro

Figura 7.63: Análisis del hiperparámetro border_count en Stacking Classifier

Finalmente, una vez entrenados los modelos base, este modelo resulta sencillo de implementar y consigue un muy buen rendimiento en términos de F1. Además, mantiene un comportamiento muy estable a lo largo de todo el proceso de entrenamiento, sin presentar indicios claros de sobreajuste.

7.1.6. Redes Neuronales

Las configuraciones consideradas para la arquitectura, regularización y entrenamiento de la red neuronal pueden consultarse en la Tabla 7.13.

Tabla 7.13: Configuraciones evaluadas para la red neuronal Keras

Parámetro	Valores evaluados
units_1	64, 128, 256
units_2	32, 64, 128
units_3	16, 32, 64
dropout_1	0.2, 0.3, 0.4
dropout_2	0.2, 0.3, 0.4
l2_reg	10^{-5}, 10^{-4}, 10^{-3}
learning_rate	10^{-4}, $5 \cdot 10^{-4}$, 10^{-3}
batch_size	32, 64, 128
epochs	120, 200

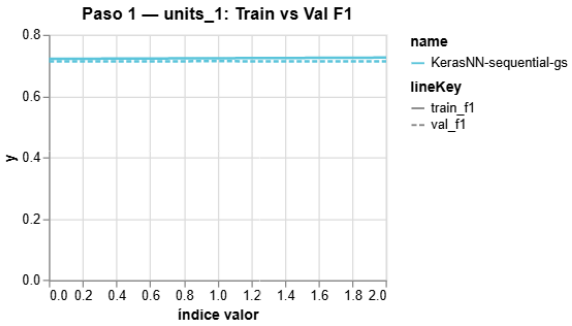
units_1

Respecto a units_1, este parámetro permite definir el número de neuronas que tendrá la primera capa de la red neuronal.

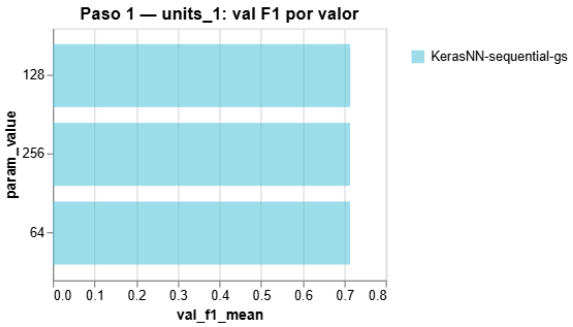
Como se puede observar en la Figura 7.64, los resultados no presentan diferencias visibles entre los valores evaluados, ya que varían únicamente en unas pocas diezmilésimas. El

mejor rendimiento se obtiene con units_1 = 128, valor que finalmente es seleccionado. Cabe destacar que, al tratarse del primer parámetro evaluado, la red neuronal ya obtiene un rendimiento superior al alcanzado por otros modelos tras completar todo su proceso de ajuste, en términos de F1.

Respecto a las curvas de entrenamiento y validación, estas se mantienen prácticamente unidas, por lo que no se observan indicios claros de sobreajuste en esta primera etapa.



(a) Comparación de la F1 en entrenamiento y validación



(b) Media de la F1 para cada valor del hiperparámetro

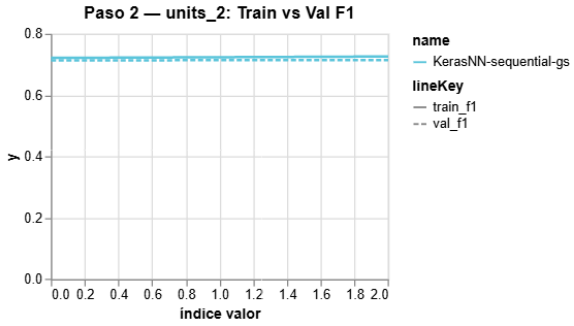
Figura 7.64: Análisis del parámetro units_1 en la Red Neuronal

units_2

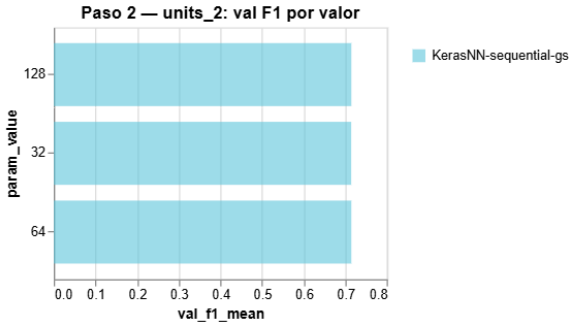
El parámetro units_2 permite definir el número de neuronas que tendrá la segunda capa de la red neuronal.

En la Figura 7.65 se observa que los resultados obtenidos son prácticamente idénticos entre los distintos valores evaluados, con variaciones muy reducidas. En esta etapa se mejora de forma residual el rendimiento alcanzado en el paso anterior. El mejor resultado se obtiene con units_2 = 128, por lo que se selecciona este valor.

Respecto a las curvas de entrenamiento y validación, no se aprecian cambios relevantes respecto a la etapa anterior, por lo que no se observan indicios claros de sobreajuste.



(a) Comparación de la F1 en entrenamiento y validación



(b) Media de la F1 para cada valor del hiperparámetro

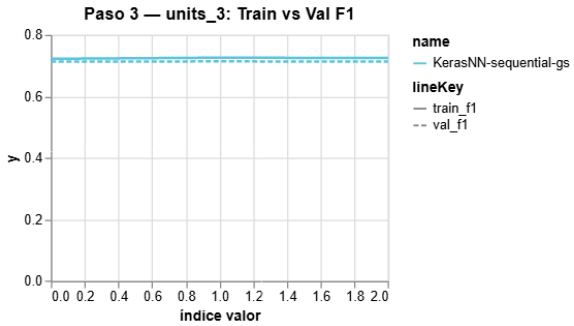
Figura 7.65: Análisis del parámetro units_2 en la Red Neuronal

units_3

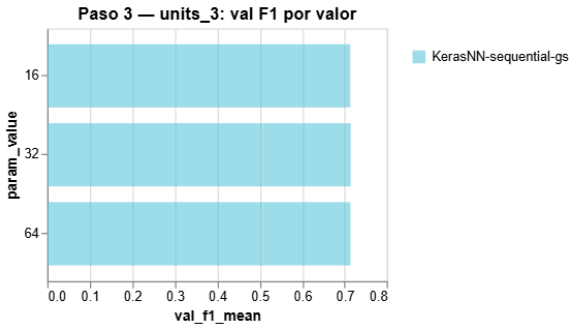
El parámetro units_3 permite definir el número de neuronas de la tercera capa de la red neuronal.

Los resultados mostrados en la Figura 7.66 indican que no existen diferencias apreciables entre los valores evaluados, con variaciones de apenas unas pocas diezmilésimas. Además, esta etapa no aporta una mejora respecto al rendimiento obtenido en el paso anterior. El mejor resultado se obtiene con units_3 = 32, por lo que se selecciona este valor.

En esta etapa tampoco se observan indicios claros de sobreajuste, ya que la pequeña distancia entre las curvas de entrenamiento y validación se mantiene prácticamente constante.



(a) Comparación de la F1 en entrenamiento y validación



(b) Media de la F1 para cada valor del hiperparámetro

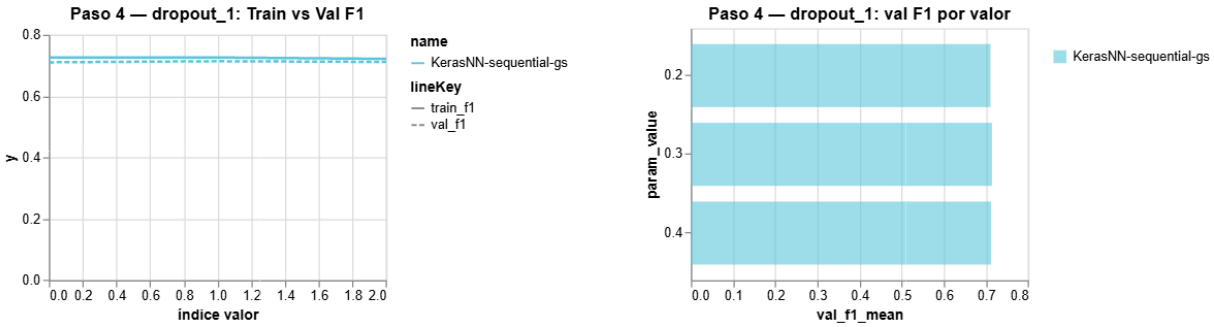
Figura 7.66: Análisis del parámetro units_3 en la Red Neuronal

dropout_1

El parámetro dropout permite desactivar aleatoriamente un porcentaje de neuronas durante el entrenamiento con el objetivo de reducir el sobreajuste y mejorar la capacidad de generalización de la red neuronal. En este caso, se aplica sobre la primera capa oculta.

Como se observa en la Figura 7.67, los resultados obtenidos presentan valores muy próximos entre sí, con variaciones de apenas unas pocas milésimas. En esta etapa tampoco se consigue mejorar el rendimiento alcanzado en las configuraciones anteriores. El mejor resultado se obtiene con dropout_1 = 0.3.

Respecto a las curvas de entrenamiento y validación, estas continúan manteniéndose prácticamente unidas, por lo que no se observan indicios claros de sobreajuste.



(a) Comparación de la F1 en entrenamiento y validación

(b) Media de la F1 para cada valor del hiperparámetro

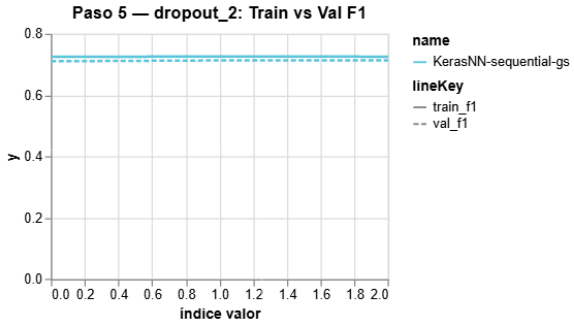
Figura 7.67: Análisis del parámetro dropout_1 en la Red Neuronal

dropout_2

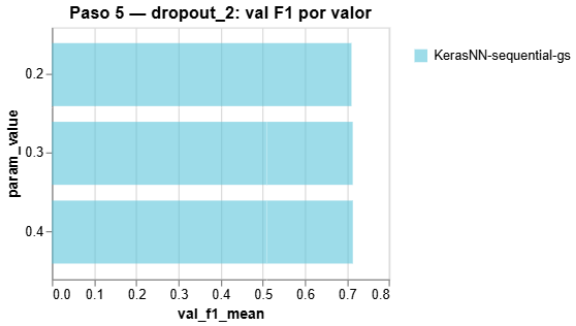
El parámetro dropout permitía desactivar aleatoriamente un porcentaje de neuronas durante el entrenamiento. En este caso, se aplica sobre la segunda capa oculta.

En la Figura 7.68 se muestran los resultados obtenidos, los cuales presentan diferencias mínimas entre los distintos valores evaluados. En esta etapa se empeora ligeramente el mejor rendimiento alcanzado en el paso anterior. El mejor resultado se obtiene con dropout_2 = 0.4.

El modelo mantiene un comportamiento bastante estable, ya que la distancia entre las curvas de entrenamiento y validación apenas varía y continúa siendo muy reducida.



(a) Comparación de la F1 en entrenamiento y validación



(b) Media de la F1 para cada valor del hiperparámetro

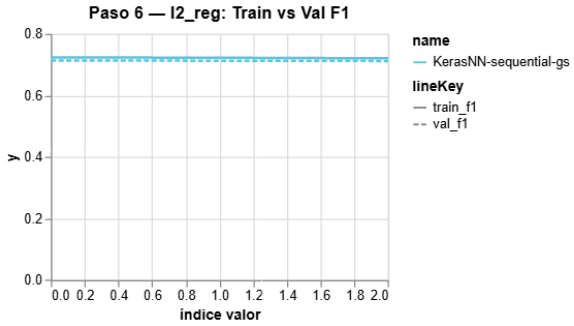
Figura 7.68: Análisis del parámetro dropout_2 en la Red Neuronal

l2_reg

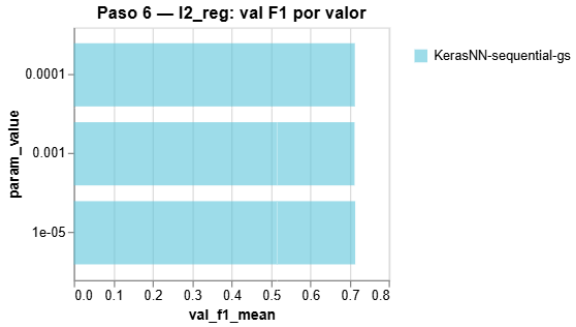
El parámetro l2_reg establece una penalización sobre los pesos de las neuronas de la red neuronal con el objetivo de reducir el sobreajuste y mejorar la capacidad de generalización del modelo.

En la Figura 7.69 se aprecia que los distintos valores evaluados producen rendimientos prácticamente equivalentes, con variaciones muy reducidas entre ellos. En esta etapa se consigue una mejora residual respecto al mejor resultado obtenido anteriormente. El mejor rendimiento se alcanza con l2_reg = 1e-5.

Respecto al comportamiento del modelo, las curvas de entrenamiento y validación continúan manteniéndose muy próximas, sin cambios relevantes respecto a las etapas anteriores.



(a) Comparación de la F1 en entrenamiento y validación



(b) Media de la F1 para cada valor del hiperparámetro

Figura 7.69: Análisis del parámetro l2_reg en la Red Neuronal

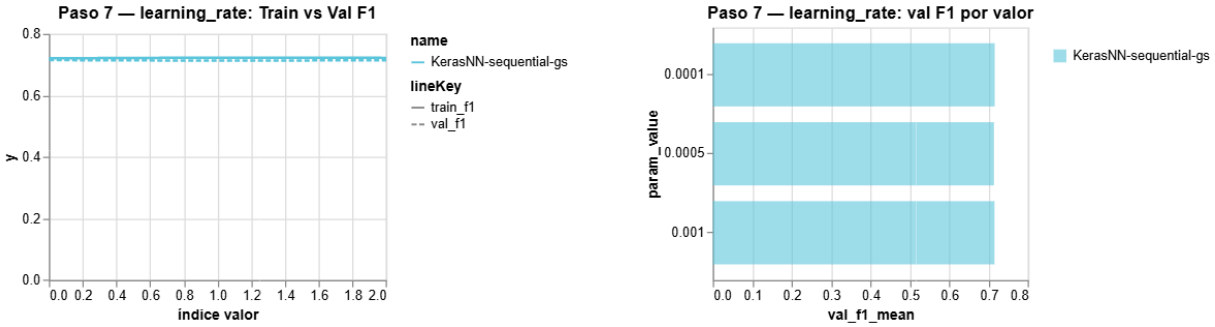
learning_rate

El parámetro learning_rate define la tasa de aprendizaje del modelo. Valores bajos producen un aprendizaje más lento y gradual, mientras que valores demasiado altos pueden

provocar que el modelo no converja correctamente hacia una solución óptima al realizar correcciones excesivamente grandes durante el entrenamiento.

Los resultados recogidos en la Figura 7.70 muestran pequeñas variaciones de unas pocas milésimas entre los valores evaluados. El mejor rendimiento se alcanza con el valor más bajo, learning_rate = 1e-4. Esto podría indicar que una tasa de aprendizaje más reducida favorece un entrenamiento más estable y una mejor capacidad de generalización del modelo.

Respecto a las curvas de entrenamiento y validación, estas mantienen un comportamiento prácticamente idéntico al observado en las etapas anteriores, sin apreciarse cambios relevantes.



(a) Comparación de la F1 en entrenamiento y validación

(b) Media de la F1 para cada valor del hiperparámetro

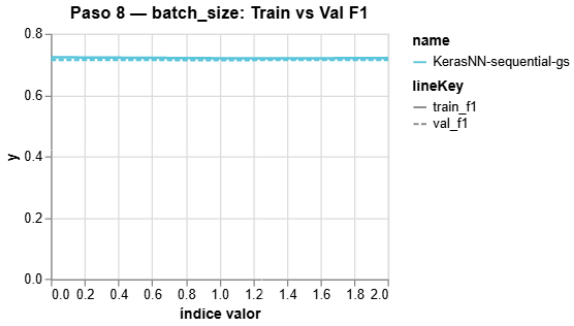
Figura 7.70: Análisis del parámetro learning_rate en la Red Neuronal

batch_size

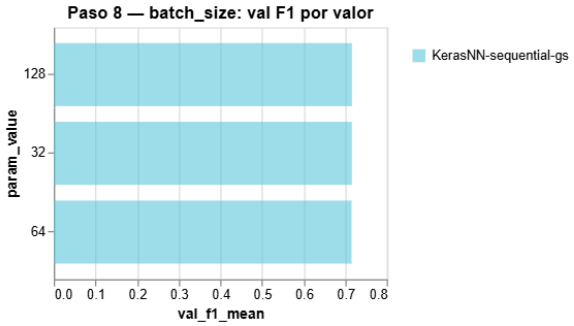
El parámetro batch_size define el número de instancias que se procesan antes de actualizar los pesos de la red neuronal.

Los resultados recogidos en la Figura 7.71 presentan diferencias mínimas entre los valores evaluados, variando únicamente en unas pocas diezmilésimas. El mejor rendimiento se obtiene con batch_size = 32.

Respecto a las curvas de entrenamiento y validación, no se aprecian cambios relevantes respecto a las etapas anteriores, manteniéndose ambas prácticamente unidas.



(a) Comparación de la F1 en entrenamiento y validación



(b) Media de la F1 para cada valor del hiperparámetro

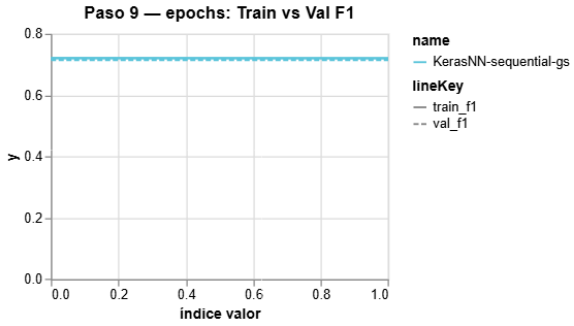
Figura 7.71: Análisis del parámetro batch_size en la Red Neuronal

epochs

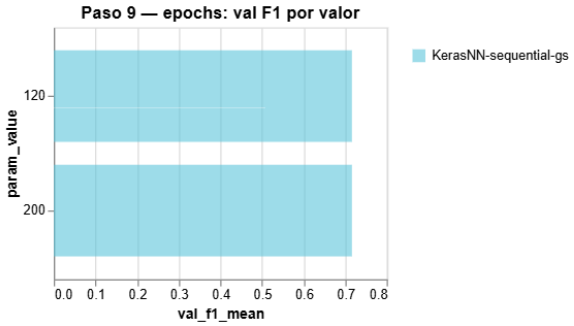
El parámetro epochs define el número de veces que la red neuronal recorrerá completamente el conjunto de datos de entrenamiento durante el proceso de aprendizaje.

La Figura 7.72 recoge el comportamiento del modelo para los valores evaluados de epochs. Ambos presentan un rendimiento prácticamente equivalente, aunque epochs = 120 obtiene un resultado ligeramente superior, por lo que se selecciona esta configuración.

En cuanto a las curvas de entrenamiento y validación, no se aprecian cambios relevantes respecto a las etapas anteriores, manteniéndose ambas muy próximas entre sí.



(a) Comparación de la F1 en entrenamiento y validación



(b) Media de la F1 para cada valor del hiperparámetro

Figura 7.72: Análisis del parámetro epochs en la Red Neuronal

Finalmente, este modelo presenta unos resultados muy destacados, situándose entre los mejores modelos evaluados y demostrando el potencial de las redes neuronales para este problema. Además, mantiene un comportamiento robusto y estable, ya que no ha mostrado indicios claros de sobreajuste durante todo el proceso de entrenamiento. De hecho, los parámetros orientados a reducir el sobreajuste apenas han tenido un impacto relevante sobre el rendimiento del modelo.

Por tanto, la red neuronal se presenta como una alternativa muy adecuada para el problema planteado.

7.2. Hiperparámetros

Esta tabla ofrece a modo de resumen la configuración final de los hiperparámetros para cada modelo estudiado 7.14:

Tabla 7.14: Mejores hiperparámetros obtenidos para cada modelo

Modelo	**Mejores hiperparámetros**
Bagging	`n_estimators=100`, `max_samples=0.5`, `max_features=1.0`, `bootstrap=True`, `bootstrap_features=False`
AdaBoost	`n_estimators=500`, `learning_rate=0.05`
CatBoost	`iterations=100`, `depth=4`, `learning_rate=0.2`, `l2_leaf_reg=1`, `border_count=64`, `bagging_temperature=0`
Decision Tree	`criterion=gini`, `max_depth=5`, `min_samples_split=2`, `min_samples_leaf=1`, `max_features=None`, `class_weight=None`
Extra Trees	`n_estimators=300`, `max_depth=15`, `min_samples_split=20`, `min_samples_leaf=1`, `max_features=0.7`, `criterion=entropy`, `class_weight=None`
HistGradientBoosting	`max_iter=100`, `learning_rate=0.2`, `max_depth=3`, `min_samples_leaf=1`, `max_leaf_nodes=15`, `max_bins=64`
KNN	`n_neighbors=11`, `weights=uniform`, `metric=minkowski`
Regresión logística	`C=0.1`, `solver=saga`, `class_weight=None`, `max_iter=200`
Red neuronal	`units_1=128`, `units_2=128`, `units_3=32`, `dropout_1=0.3`, `dropout_2=0.4`, `l2_reg=1e-05`, `learning_rate=0.0001`, `batch_size=32`, `epochs=120`
Random Forest	`n_estimators=500`, `max_depth=10`, `min_samples_split=20`, `min_samples_leaf=8`, `max_features=0.5`, `criterion=gini`, `class_weight=None`
Stacking	`final_estimator__iterations=25`, `final_estimator__depth=8`, `final_estimator__learning_rate=0.5`, `final_estimator__l2_leaf_reg=5` `final_estimator__border_count=128`
Voting Classifier	`voting=hard`
XGBoost	`n_estimators=25`, `max_depth=3`, `learning_rate=0.3`, `subsample=0.5`, `colsample_bytree=1.0`

7.3. Análisis Global

En esta sección se analizarán los resultados obtenidos por los distintos modelos durante el test. Además, se comparará el rendimiento alcanzado por la red neuronal frente a los

diferentes modelos de machine learning tradicionales. Los resultados obtenidos se muestran en la Tabla 7.15 y Figura 7.73.

Tabla 7.15: Resultados obtenidos en test para los distintos modelos

Modelo	Accuracy	Precision weighted	F1 weighted	ROC-AUC
AdaBoost	0.7467	0.7329	0.7287	0.7734
Bagging	0.7403	0.7255	0.7155	0.7589
CatBoost	0.7439	0.7295	0.7234	0.7749
Decision Tree	0.7380	0.7223	0.7141	0.7509
Extra Trees	0.7340	0.7172	0.7131	0.7555
HistGradientBoosting	0.7456	0.7315	0.7265	0.7756
KNN	0.7102	0.6885	0.6890	0.7150
Logistic Regression	0.7283	0.7098	0.7017	0.7438
MLP	0.7360	0.7203	0.7017	0.7626
Random Forest	0.7420	0.7280	0.7163	0.7711
Stacking	0.7465	0.7329	0.7249	0.7759
SVM	0.7342	0.7240	0.6935	0.7239
Voting	0.7448	0.7306	0.7267	–
XGBoost	0.7456	0.7315	0.7264	0.7747

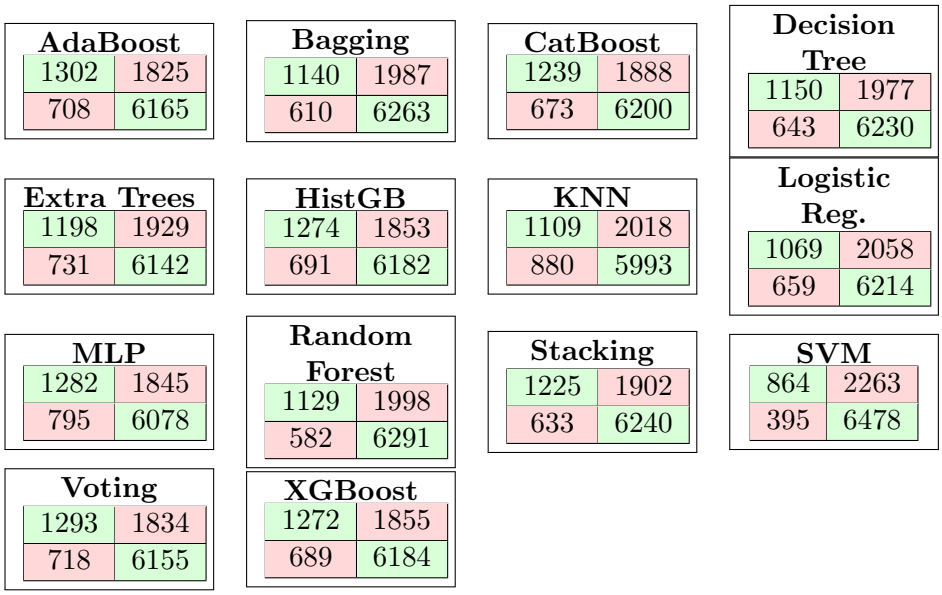


Figura 7.73: Matrices de confusión obtenidas por los distintos modelos.

Nota: Cada matriz sigue el formato $\begin{bmatrix} VN & FP \\ FN & VP \end{bmatrix}$.

De manera general, los modelos con mejor rendimiento son los modelos boosting basados en árboles. En concreto, AdaBoost obtiene el mejor rendimiento global. Muy cerca de este modelo se encuentran XGBoost e HistGradientBoosting, cuyos resultados son prácticamente equivalentes y presentan tam-

bién valores muy elevados de ROC-AUC. De hecho, HistGradientBoosting obtiene uno de los mejores valores de ROC-AUC, indicando una gran capacidad para diferenciar entre clases.

Por otro lado, aunque CatBoost había obtenido el mejor resultado durante validación, en el conjunto de test otros modelos como AdaBoost, XGBoost o HistGradientBoosting logran un rendimiento ligeramente superior. Sin embargo, las diferencias entre los mejores modelos son reducidas.

Los modelos ensemble también muestran un rendimiento muy competitivo. En particular, Stacking obtiene el mayor valor de ROC-AUC, lo que indica que es el modelo con mejor capacidad de separación entre clases. Voting también presenta un rendimiento muy elevado, alcanzando métricas similares a las obtenidas por algunos modelos boosting.

En cuanto a los modelos basados en árboles tradicionales, todos presentan resultados relativamente similares entre sí, siendo Random Forest el modelo que obtiene el mejor comportamiento global dentro de esta familia. Esto sugiere que la utilización de subconjuntos aleatorios de variables ayuda a mejorar la capacidad de generalización del modelo. Sin embargo, técnicas como la aleatorización adicional de umbrales utilizada en Extra Trees no parecen aportar una mejora significativa en este problema, obteniendo resultados ligeramente inferiores incluso a modelos más simples como Decision Tree o Bagging.

Por otro lado, los modelos más tradicionales (a excepción de Decision Tree) como Logistic Regression, SVM o KNN muestran un rendimiento inferior respecto al resto de enfoques estudiados. Esto podría indicar que el problema presenta relaciones complejas y no lineales entre las variables, favoreciendo especialmente a los modelos basados en árboles y ensembles. Además, el menor rendimiento obtenido por KNN puede deberse a la presencia de ruido y a la complejidad de las fronteras de decisión.

Gracias a la matriz de confusión podemos observar qué modelo es capaz de detectar un mayor número de pacientes con cáncer y cuál minimiza en mayor medida los falsos negativos. Este aspecto resulta especialmente importante en aplicaciones médicas, ya que es preferible diagnosticar erróneamente a un paciente sano como enfermo antes que enviar a casa a un paciente con cáncer sin detectar la enfermedad. En este sentido, el modelo SVM es el que obtiene el menor número de falsos negativos y mayor número de aciertos de pacientes con cáncer.

Finalmente, aunque las redes neuronales representan un enfoque muy potente para problemas de clasificación, en este caso no parecen ser la alternativa más adecuada. Su diseño y entrenamiento requieren tener en cuenta un mayor número de factores, como la arquitectura de la red, el número de capas,

las funciones de activación, el optimizador, la tasa de aprendizaje o técnicas de regularización como el dropout. Además, este mayor coste de implementación y ajuste no se ve compensado por un mejor rendimiento, ya que la red neuronal obtiene resultados inferiores a los alcanzados por varios modelos boosting, ensemble y basados en árboles.

Por lo tanto, considerando el contexto clínico, resulta más crítico no detectar a un paciente con cáncer de pulmón que generar una falsa alarma en un paciente sano. Bajo este criterio, SVM se posiciona como el modelo más adecuado, ya que es el que consigue reducir en mayor medida el número de falsos negativos y maximizar los verdaderos positivos en el conjunto de datos analizado. Aunque esta elección penaliza métricas globales como el Accuracy o el F1, desde una perspectiva clínica puede considerarse preferible priorizar la seguridad del paciente. Un falso positivo implicaría la realización de pruebas diagnósticas adicionales, mientras que un falso negativo podría retrasar el diagnóstico y el inicio del tratamiento.

7.4. Coherencia de los resultados con la literatura

Para comprobar si la información aportada por los modelos Random Forest 7.1.4 y Extra Trees 7.1.4, así como el estudio estadístico de las variables con mayor capacidad predictiva 4.4, resulta coherente con la literatura médica, se comparan los resultados obtenidos con distintos estudios sobre los principales factores de riesgo del cáncer de pulmón. Entre ellos, destaca el informe elaborado por Cancer Australia sobre los factores de riesgo asociados a esta enfermedad, del cual se extraen las siguientes Figuras (7.74 , 7.75, 7.76, 7.77) relacionadas con el riesgo de desarrollar cáncer de pulmón según distintos factores [32].

En primer lugar, el tabaquismo aparece como uno de los factores más relevantes. Aproximadamente el 80 % de las muertes por cáncer de pulmón se relacionan con el consumo de tabaco [33]. Además, en la Figura 7.74 se observa cómo fumar más de 20 cigarrillos al día incrementa notablemente el riesgo de desarrollar esta enfermedad.

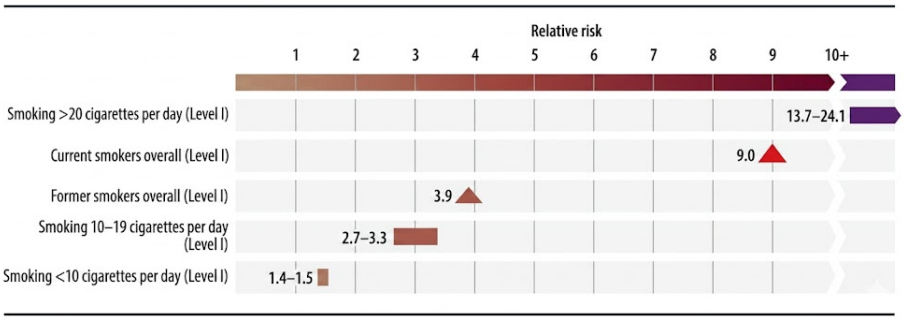


Figura 7.74: Riesgo de desarrollar cáncer de pulmón asociado al tabaquismo

Respecto a la exposición al radón, esta se considera una de las principales causas ambientales del cáncer de pulmón. En la Figura 7.75 puede observarse que la exposición en minas de uranio incrementa notablemente el riesgo de desarrollar la enfermedad.

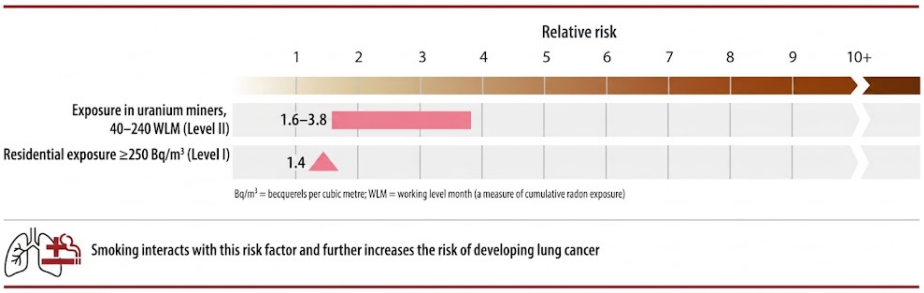


Figura 7.75: Riesgo de desarrollar cáncer de pulmón por exposición al radón

Asimismo, se ha demostrado la importancia del asbesto como factor de riesgo, no solo por la exposición directa de los trabajadores, sino también por la posible exposición indirecta de familiares a través del transporte de fibras en la ropa o en el entorno doméstico [34]. En la Figura 7.76, proporcionada por el informe del gobierno australiano, se observa que la exposición laboral a esta sustancia constituye un factor de riesgo importante.

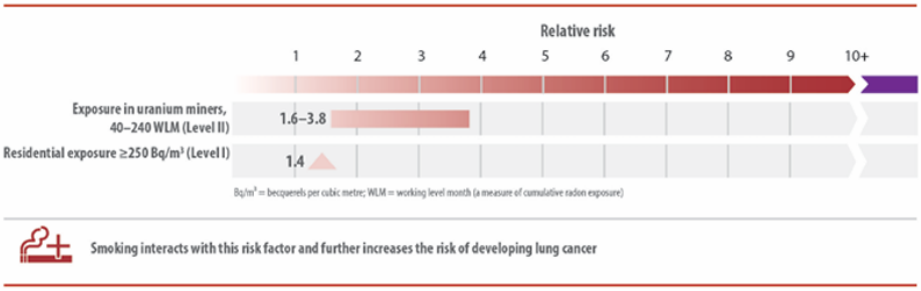


Figura 7.76: Riesgo de desarrollar cáncer de pulmón por exposición al asbesto

Otra variable relevante son los antecedentes familiares. Aunque desde un punto de vista estadístico esta variable parecía aportar menor capacidad predictiva que otras, tanto Random Forest como Extra Trees la sitúan entre las variables más importantes. Además, en la Figura 7.77, proporcionada por el informe del gobierno australiano, puede observarse que el historial familiar constituye un factor que debe ser considerado.

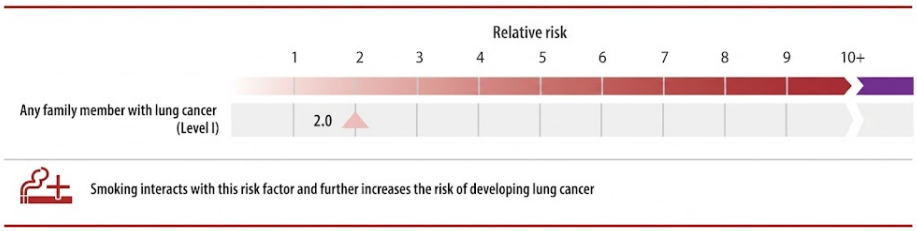


Figura 7.77: Riesgo de desarrollar cáncer de pulmón asociado a antecedentes familiares

La variable *copd_diagnosis* también resulta relevante, ya que diversos estudios han asociado la EPOC con un mayor riesgo de desarrollar cáncer de pulmón [35].

La edad constituye otro factor importante. Según la American Cancer Society, la mayoría de los casos de cáncer de pulmón se diagnostican en personas de 65 años o más, mientras que los diagnósticos en menores de 45 años son mucho menos frecuentes. Por tanto, una mayor edad puede asociarse con una mayor probabilidad de desarrollar esta enfermedad.

Por último, aunque en nuestro análisis la variable *gender* no muestra una capacidad predictiva destacable desde el punto de vista estadístico, las estimaciones de SEOM y REDECAN para España en 2026 indican que el número de nuevos casos de cáncer de pulmón en hombres será casi el doble

que en mujeres, con 23.079 casos estimados frente a 11.829, respectivamente [1].

En conjunto, estos resultados muestran que las variables consideradas más relevantes por los modelos coinciden, en términos generales, con los principales factores de riesgo descritos en la literatura médica.

Parte IV

Conclusiones

Capítulo 8

Conclusiones y trabajo futuro

8.1. Conclusiones

Finalmente, puede concluirse que los objetivos planteados al inicio del proyecto 1.8 han sido cumplidos satisfactoriamente. A continuación, se presentan las principales conclusiones obtenidas.

Los resultados alcanzados pueden considerarse positivos, obteniéndose métricas competitivas en términos F1 o ROC-AUC. Sin embargo, todavía existe margen de mejora, especialmente debido a las limitaciones propias del conjunto de datos utilizado. Variables como la edad, el tabaquismo o la exposición a determinados agentes son factores relevantes que pueden aumentar el riesgo de desarrollar cáncer de pulmón y, por tanto, aportan información útil para la predicción. Sin embargo, existen otros factores que pueden explicar la aparición de la enfermedad.

El desarrollo del cáncer de pulmón también depende de factores genéticos y biológicos propios de cada persona. Algunas personas pueden presentar una mayor capacidad de reparación del ADN o una menor predisposición a desarrollar mutaciones, mientras que otras pueden mostrar una mayor vulnerabilidad genética incluso con exposiciones aparentemente menores. Esto puede provocar que pacientes con factores de riesgo claros no desarrollen la enfermedad y, por el contrario, que otros pacientes sí la desarrollen pese a no presentar valores especialmente elevados en las variables recogidas [36].

Por otro lado, aunque el conjunto de datos presenta cierto desbalanceo entre clases, el análisis realizado muestra que este no parece ser el principal factor limitante del rendimiento. Esto puede observarse, por ejemplo, en que el

uso de *class_weight* apenas modifica los resultados en varios modelos. No obstante, sí se aprecia una mayor tendencia de los modelos a identificar correctamente la clase positiva frente a la negativa, lo que indica que la detección de pacientes sin cáncer resulta más compleja en este conjunto de datos.

Finalmente, y en línea con los objetivos planteados al inicio del trabajo, se ha comprobado que, aunque las redes neuronales son modelos muy potentes y ampliamente utilizados en numerosos problemas de clasificación, en este caso no han sido capaces de obtener los mejores resultados. Por el contrario, otros modelos de aprendizaje automático más sencillos de implementar y con una mayor capacidad de interpretación han mostrado un rendimiento superior.

8.2. Limitaciones

Una de las limitaciones encontradas es el dataset, ya que no incluye otros factores potencialmente relevantes, como la contaminación ambiental [37], sustancias radiactivas u otros agentes externos que podrían influir en el riesgo real de padecer cáncer de pulmón. A esto se suma la presencia de ruido en los datos y distribuciones relativamente similares entre algunas variables, lo que dificulta que los modelos encuentren patrones claramente diferenciadores entre ambas clases.

Otra limitación importante es la naturaleza de las variables utilizadas. El conjunto de datos contiene principalmente variables categóricas y únicamente dos variables numéricas. Muchos modelos de aprendizaje automático suelen rendir mejor con variables continuas, ya que estas pueden aportar una mayor cantidad de información y permitir detectar relaciones más complejas entre los datos. Por tanto, esta limitación puede haber afectado al rendimiento global de los modelos desarrollados.

A pesar de estas limitaciones, los resultados obtenidos demuestran que las técnicas de aprendizaje automático pueden constituir una herramienta de apoyo útil en problemas relacionados con la detección de cáncer de pulmón, especialmente como sistemas complementarios de ayuda a la decisión clínica.

8.3. Trabajo futuro

Como trabajo futuro, como hemos comentado anteriormente en las Limitaciones 8.2, sería recomendable ampliar el conjunto de datos incorporando

nuevos factores potencialmente relevantes, como la contaminación ambiental, la exposición laboral a sustancias químicas o radiactivas, o información clínica más detallada.

Además, resultaría interesante estudiar el problema desde una perspectiva genética, analizando cómo podrían incorporarse variables relacionadas con la predisposición genética o la capacidad de reparación del ADN. Para ello, una primera opción sería comprobar la existencia de bases de datos que ya incluyan este tipo de información y, en caso contrario, estudiar la posibilidad de recopilarla a partir de una muestra lo más diversa posible.

Otra posible línea de mejora consistiría en ampliar la información asociada a las variables categóricas, incorporando más grados de exposición o niveles en determinados factores de riesgo y enfermedades. De esta forma, los modelos podrían disponer de información más amplia y detallada sobre cada paciente.

Además de variables como el sexo o la edad, sería de utilidad disponer de información demográfica adicional que permitiera comprobar si distintos sectores poblacionales están correctamente representados. No sería la primera vez que una herramienta médica basada en inteligencia artificial presentase sesgos hacia determinados sectores poblacionales. Por ello, resulta importante garantizar que distintos grupos de población estén correctamente representados en el conjunto de datos, evitando que los modelos aprendan patrones excesivamente centrados en perfiles concretos y mejorando así su capacidad de generalización.

Otra posible ampliación del trabajo sería incorporar imágenes médicas al dataset y desarrollar modelos multimodales capaces de combinar información clínica y factores de riesgo junto con imágenes como radiografías de tórax o tomografías computarizadas (TC torácico). Esto podría permitir capturar patrones más complejos y mejorar la capacidad predictiva de los modelos.

Bibliografía

[1] Red Española de Registros de Cáncer (REDECAN). Las cifras del cáncer en españa 2026. Technical report, Red Española de Registros de Cáncer, España, 2026.

[2] Niki Najar Najafi, Helia Hajihassani, and Maryam Azimzadeh Irani. The impact of artificial intelligence on cancer diagnosis and treatment: A review. *Cancer Informatics*, 24:11769351251371273, 2025.

[3] Huakang Tu, Yunfeng Zhao, Jiameng Cui, Wanzhu Lu, Gege Sun, Xiaohang Xu, Qingfeng Hu, Kejia Hu, Ming Wu, and Xifeng Wu. Improving lung cancer risk prediction using machine learning: A comparative analysis of stacking models and traditional approaches. *Cancers*, 17(10):1651, 2025.

[4] Glassdoor. Junior data scientist sueldos en españa, 2026. Consultado el 8 de junio de 2026.

[5] Naciones Unidas. Objetivo 3: Garantizar una vida sana y promover el bienestar para todos en todas las edades. `https://www.un.org/sustainabledevelopment/es/health/`, 2025. Consultado el 15 de junio de 2026.

[6] Pacto Mundial de la ONU España. Ods 10: Reducción de las desigualdades. `https://www.pactomundial.org/ods/10-reduccion-de-las-desigualdades/`, 2025. Consultado el 15 de junio de 2026.

[7] Elias Dritsas and Maria Trigka. Lung cancer risk prediction with machine learning models. *Big Data and Cognitive Computing*, 6(4):139, 2022.

[8] Shahid Mohammad Ganie and Pijush Kanti Dutta Pramanik. Interpretable lung cancer risk prediction using ensemble learning and xai based on lifestyle and demographic data. *Computational Biology and Chemistry*, 117:108438, 2025.

[9] Towhidul Islam, Safa Asgar, and Sajjad Mahmood. Lung cancer prediction using machine learning on structured clinical data: A systematic review of diagnosis, risk, and survival models. *Preprints*, 2025. Preprint.

[10] Aurélien Géron. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. O'Reilly Media, 2019.

[11] Dave Bergmann and IBM. ¿qué es el machine learning (ml) o aprendizaje automático?, 2025.

[12] Olivier Chapelle, Bernhard Schölkopf, and Alexander Zien, editors. *Semi-Supervised Learning*. MIT Press, Cambridge, MA, 2006.

[13] Gareth James, Daniela Witten, Trevor Hastie, and Robert Tibshirani. *An Introduction to Statistical Learning*. Springer, 2013.

[14] Christopher M. Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006.

[15] Thomas G. Dietterich and Ghulum Bakiri. Solving multiclass learning problems via error-correcting output codes. *Journal of Artificial Intelligence Research*, 2:263–286, 1995.

[16] Francisco Herrera, Francisco Charte, Antonio J. Rivera, and María J. del Jesus. *Multilabel Classification: Problem Analysis, Metrics and Techniques*. Springer International Publishing, Cham, 1 edition, 2016.

[17] Antonio Mucherino, Petraq J. Papajorgji, and Panos M. Pardalos. k-nearest neighbor classification. In *Data Mining in Agriculture*, volume 34 of *Springer Optimization and Its Applications*. Springer, New York, NY, 2009.

[18] Li-Yeh Hu, Min-Wei Huang, Shih-Wei Ke, and Chih-Fong Tsai. The distance function effect on k-nearest neighbor classification for medical datasets. *SpringerPlus*, 5(1):1304, 2016.

[19] Vikramaditya Jakkula. Tutorial on support vector machine (svm), 2006. Tutorial técnico.

[20] Jiang Su and Harry Zhang. A fast decision tree learning algorithm. In *Proceedings of the Twenty-First National Conference on Artificial Intelligence*, AAAI, pages 500–505. AAAI Press, 2006.

[21] S. B. Kotsiantis, G. E. Tsekouras, and P. E. Pintelas. Bagging model trees for classification problems. In *Advances in Informatics*, volume 3746 of *Lecture Notes in Computer Science*, pages 328–337, Berlin, Heidelberg, 2005. Springer.

[22] Anjaneyulu Babu Shaik and Sujatha Srinivasan. A brief survey on random forest ensembles in classification model. In *International Conference on Innovative Computing and Communications*, volume 56 of *Lecture Notes in Networks and Systems*, pages 253–260. Springer, Singapore, 2019.

[23] Zhi-Hua Zhou. *Ensemble Methods: Foundations and Algorithms*. Chapman and Hall/CRC, 2012.

[24] Picard2024. Hist gradient boosting classifier (scikit-learn). `https://datasciencepythonblog.net/hist-gradient-boosting-classifier-scikit-learn/`, 2024. CFD Institute. Accedido el 22 de abril de 2026.

[25] IBM. ¿qué es xgboost? `https://www.ibm.com/es-es/think/topics/xgboost`, 2026. Consultado el 22 de mayo de 2026.

[26] XGBoost developers. *XGBoost Parameters*. XGBoost Documentation, 2025.

[27] Abdullahi A. Ibrahim, Raheem L. Ridwan, Muhammed M. Muhammed, Rabiat O. Abdulaziz, and Ganiyu A. Saheed. Comparison of the catboost classifier with other machine learning methods. *International Journal of Advanced Computer Science and Applications*, 11(11), 2020.

[28] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.

[29] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.

[30] Haibo He and Yunqian Ma, editors. *Imbalanced Learning: Foundations, Algorithms, and Applications*. John Wiley & Sons, Hoboken, New Jersey, 2013.

[31] Draken-Sensei. Lung-cancer-risk-dataset, 2025.

[32] Cancer Australia. Risk factors for lung cancer: An overview of the evidence. `https://www.canceraustralia.gov.au`, 2014. Consultado el 22 de mayo de 2026.

[33] Y. Alduais, H. Zhang, F. Fan, J. Chen, and B. Chen. Non-small cell lung cancer (nsclc): A review of risk factors, diagnosis, and treatment. *Medicine*, 102(8):e32899, 2023.

[34] Steven B. Markowitz, Stephen M. Levin, Albert Miller, and Alfredo Morabia. Asbestos, asbestosis, smoking, and lung cancer. new findings from the north american insulator cohort. *American Journal of Respiratory and Critical Care Medicine*, 188(1):90–96, 07 2013.

[35] C. Qi, S. W. Sun, and X. Z. Xiong. From copd to lung cancer: Mechanisms linking, diagnosis, treatment, and prognosis. *International Journal of Chronic Obstructive Pulmonary Disease*, 17:2603–2621, 2022.

[36] Yohani García Mederos, Larisa Zamora Matamoros, and Nelsa Sagaró del Campo. Análisis estadístico implicativo en la identificación de factores de riesgo en pacientes con cáncer de pulmón. *MEDISAN*, 19(8):947, 2015.

[37] G. Luo, Y. Zhang, H. Rumgay, et al. Estimated worldwide variation and trends in incidence of lung cancer by histological subtype in 2022 and over time: a population-based study. *The Lancet Respiratory Medicine*, 13:348–363, 2025.